Brief cho Claude Code – SDC 2026 Midterm01 Localization

Tổng hợp: PDF giải thích repo SDC-midterm-1- (commit 0ae197f) · ghi chú tay, đánh máy, đánh dấu vàng của Thu Hang · các câu trả lời đã thảo luận · danh sách nhiệm vụ. Ngày 22/10/2026. Phụ lục cuối file = nguyên bản PDF có ghi chú tay.

Gửi Claude Code – đọc trước khi làm. Mục tiêu: trả lời bằng **dữ liệu và thí nghiệm** các câu hỏi trong tài liệu này, ưu tiên việc giảm **sai số vị trí** (chiếm ~78 % điểm Track01, ~82 % Track02). Bắt đầu bằng đọc `CLAUDE.md`, `docs/HUONG-DAN-CHAY.md` và `docs/nhat-ky/08-gicp-va-scale.md`. Làm theo thứ tự ưu tiên ở mục 7. Mọi câu trả lời trong tài liệu có nhãn *(cần kiểm chứng)* là giả thuyết – hãy xác nhận hoặc bác bỏ bằng số liệu và ghi lại.

1. Luật bắt buộc (không được vi phạm)

- **Không dò bảng điểm.** Không tạo file lệch có chủ đích để đo đáp án ẩn (probe_*, bump*, yawproj_*, posproj_*, lb_posyaw_*, corr_r*, dao động cosin). TA cấm từ 20/10. Dùng điểm Kaggle để *so sánh pipeline / chỉnh tham số bình thường* thì được.
- Mọi file nộp phải là **output trực tiếp của pipeline**. Bài chấm cuối = file pipeline sạch, đánh dấu "Use for Final Score".
- Tối đa ~3 file mỗi lần nộp, 15 lượt/ngày/track. Ưu tiên thí nghiệm **offline** (không tốn lượt) để lọc trước.
- Một thí nghiệm chỉ đổi **một** tham số. Ghi trước giả thuyết, sau đó ghi kết quả vào `docs/nhat-ky/`.
- Không làm Track03. Không commit token Kaggle hay dữ liệu cuộc thi. Luôn commit + push sau mỗi bước xong.
- Hạn: Kaggle 29/10/2026 09:00 (UTC+8); thuyết trình 29/10; E3 (code, result_1/2.csv, report PDF tiếng Anh, README) 05/11 23:59.

2. Bối cảnh tóm tắt

Mục	Track01	Track02
Số frame / thời lượng / quãng đường	3011 / 109.7 s / 1052.9 m	3299 / 118.5 s / 1247.4 m
Baseline TA (Kaggle)	1.79075	1.92587
Pipeline thuần tốt nhất đã có điểm	1.25338 (phys_yawb_1)	0.40778 (rts_heading_2)
Cùng phương pháp, lượt cũ (lai)	0.78914 = 0.616 m + 0.2×0.865°	0.27654 = 0.227 m + 0.2×0.249°
Bài chấm sạch (chưa có điểm lúc viết)	clean_cfz1_tau7.csv, dự kiến ~0.79	clean_cfz2_tau15.csv, dự kiến ~0.28
Điểm file dò (KHÔNG dùng chấm)	0.46	0.19
Độ giật GNSS	0.142 m	0.052 m

Điểm = RMSE vị trí (m) + 0.2 × RMSE yaw (độ). **Pipeline sạch** (`tools/run_clean_pipeline.sh`): GNSS → Kalman CTRV + RTS (nhiễu chọn bằng maximum likelihood) → [chỉ Track01] GICP point cloud 3 camera (MapAnything) vào bản đồ voxel 0.3 m, tỉ lệ = 1.646/h(frame), 2 lượt, cổng chặn ≤2 m/3°, trung vị 5 + Gauss 2 s, --demean → yaw = bộ lọc bù gyro trục z + hướng chạy (τ 7 s Track01, 15 s Track02).

File ngoài git cần cho pipeline: lượt NDT C++ `c3s08_*` (Track01) / `c3s10T2_*` (Track02) gồm `*_result.txt` và `result/chunks/` (chỉ để lấy z, roll, pitch); `map_voxel_0.3m.pcd`; `phys_yawb_1.csv`, `rts_heading_2.csv`. Lưu ý: c3s08 chạy với prior `yawproj_v1_1.csv` (tổ hợp chọn theo bảng điểm) → pipeline chưa sạch tuyệt đối.

3. Bảng tổng hợp ghi chú của người dùng (34 mục)

Cột “Nhiệm vụ” trỏ tới mục 7. A = khái niệm, B = nguồn sai số, C = thuật toán, D = hai track / tham số, E = dò bảng điểm.

Tr.	Ghi chú gốc	Câu hỏi thật sự	Nhóm	Ưu tiên	Nhiệm vụ
6	Vàng: tỉ lệ ×3 thay vì ×9, trôi ~100 m	Vì sao sai tỉ lệ gây trôi lớn	B	TB	T5
6	Vàng: GNSS gần đáp án; độ cao mặt đường; gyro IMU	Ba phát hiện chính – nói rõ khi thuyết trình	–	Cao	T9
6	Khoanh: (Track01), 2 lượt, lọc bù gyro, (làm xấu đi)	Vì sao chỉ Track01 dùng GICP, vì sao 2 lượt	D	Cao	T6
6	Tay: phần lớn sai số do RMSE – RMSE phụ thuộc chỉ số nào	Thành phần nào của RMSE lớn nhất	B	Cao	T3
8	Tay: 3 luồng h265 là gì; 4 Hz là gì	Định dạng video, tần số GNSS	A	Thấp	–
8	Tay: LoGeR là gì, MapAnything là gì	Hai mô hình dựng point cloud	A	TB	T9
8	Tay: pose ban đầu có cộng nhiễu – tại sao, từ đâu, Track2 lệch	Nguồn gốc nhiễu init_guess	B	Thấp	–
9	Tay: đổi radian → độ gây sai số bao nhiêu	Đổi đơn vị có mất chính xác không	B	Thấp	–
9	Tay: phạt nặng sai lớn → xét lại file đã nộp, khi nào sai lớn	Sai lớn nằm ở đoạn nào	B	Cao	T3
9	Tay: Kaggle chỉ cho tổng điểm → phân tách cho kết luận gì, có gì phi vật lý	Kết luận từ tách điểm	E	Cao	T8
9	Vàng+tay: chỉ trình bày như phân tích → viết vào report, mô tả rõ dò như nào	Mục dò bảng điểm trong report	E	Cao	T8
13	Tay: có cộng dồn sai số không; làm sao giảm (bias 25–30°)	Trôi gyro và cách khử	B	TB	–
13	Tay: yaw đã sửa là cho track nào (Hình 6)	Hình 6 thuộc track nào	A	Thấp	–
13	Tay: chia nhỏ (voxel) gây sai số bao nhiêu	Sai số giảm mẫu	B	TB	T4
14	Tay: việc nhân (scale) gây sai số gì / bao nhiêu	Sai số do tỉ lệ	B	Cao	T2, T4
14	Vàng+tay: h ≈ 2.06 m nhưng camera 1.65 m – tại sao	Vì sao MapAnything sai tỉ lệ	B	TB	T7
14	Tay: từ dò điểm có suy ra số ≠ 0.8 không	Có tỉ lệ tốt hơn 0.8 không	D	TB	T4, T6
15	Vàng+tay: điểm khởi đầu tốt – có khởi đầu xấu gây sai số không	Prior đủ tốt cho ICP không	B	TB	T3
16	Tay: khác nhau của 3 kết quả khi dùng 3 thuật toán	PLANAR / point-to-plane / GICP	C	TB	T6
16	Tay: nói rõ khác biệt GICP và NDT	GICP vs NDT	C	Cao	T9
17	Tay: tại sao cần Kalman	Lý do dùng Kalman	C	TB	T9
18	Vàng+tay: độ trôi do đâu, làm sao fix; 2 đồ thị liên hệ ra sao	Đọc Hình 15	C	TB	–
26	Tay: baseline thấp thì trông như nào	Hình dạng baseline	A	Thấp	T10
27	Vàng+tay: NDT C++ cũ c3s08/c3s10T2 → z, roll, pitch … là sao	Vai trò lượt NDT cũ	A	Cao	T0, T7
27	Tay: gyro là gì; voxel là gì	Khái niệm	A	Thấp	–

Tr.	Ghi chú gốc	Câu hỏi thật sự	Nhóm	Ưu tiên	Nhiệm vụ
27	Tay: tại sao Track1 và Track2 khác nhau	Khác biệt pipeline hai track	D	Cao	T6, T11
40	Tay: tìm bài toán tương tự, họ giải như nào	Tài liệu tham khảo	C	TB	T9
41	Tay: tự tính lại kết quả, so với Kaggle	Kiểm chứng offline	D	TB	T1
41	Khoanh: ngẫu nhiên (vài mm), lọc không khử được → cần cách tính khác	Khử lỗi trôi chậm GNSS	D	Cao	T6, T12
42	Đánh máy: có sai số làm tròn không	Sai số số học	B	Thấp	–
42	Vàng+đánh máy: tại sao lệch nhiều, chỉnh thông số như nào cho ít sai số nhất	Nguồn lệch GICP, chiến lược chỉnh	D	Cao	T2, T4, T6
43	Vàng cả mục 6.6 + viết vào report như phần thú vị	Mục dò bảng điểm	E	Cao	T8
45	Đánh máy: lỗi vị trí và yaw cần thu nhỏ	Hướng giảm điểm	D	Cao	T4, T6
48	Vàng 0.78914/0.27654; khoanh Hình 21: lúc đầu dẫn, rồi tụt đột ngột, đứng yên	Đọc Hình 21	E	TB	T8
chat	AI dựng 3D sai hay thuật toán sai? Lỗi ở đâu?	Tách sai số đầu vào vs thuật toán	B	Cao	T2
chat	Tại sao độ giật GNSS Track01 cao như vậy?	Nguyên nhân jitter	D	Cao	T11

4. Câu trả lời đã thảo luận – nhóm A (khái niệm)

- **h265:** H.265/HEVC là chuẩn nén video; rosbag có 3 luồng nén, mỗi luồng một camera. Repo dùng ảnh đã giải nén sẵn trong Track1_images/.
- **4 Hz:** GNSS 4 bản tin/s (0.25 s/mẫu); camera ~28 frame/s → ~7 frame giữa 2 mẫu GNSS phải nội suy.
- **LoGeR / MapAnything:** mô hình AI dựng 3D từ ảnh do TA chạy sẵn. MapAnything (Meta, 2025) cho kết quả gần đơn vị mét (cần tỉ lệ ~0.8). LoGeR cho camera trước, đơn vị tuỳ ý (cần ×9). *(cần kiểm chứng thông tin LoGeR từ slide TA)*
- **Hình 6:** Track02 (yaw GNSS kẹt −15.2° 13 s đầu).
- **Gyro:** con quay hồi chuyển trong IMU, đo tốc độ quay; gyro z = tốc độ đổi yaw. **Voxel:** khối lập phương cạnh L, giảm mẫu = giữ trọng tâm mỗi khối.
- **Baseline:** local trôi 91–110 m (nút rối rồi trôi xa), nhưng điểm Kaggle baseline chỉ 1.79 → file baseline TA nộp có lẽ không phải lượt trôi này. *(cần kiểm chứng – T10)*
- **c3s08 / c3s10T2:** lượt NDT C++ cũ trên 3cameras_ply (s08 = tỉ lệ 0.8, s10 = 1.0, T2 = Track02). Pipeline chỉ lấy z, roll, pitch để GICP có pose 3D đầy đủ; x, y, yaw từ prior Kalman. Rủi ro: mất file (ngoài git) và prior của c3s08 là file tổ hợp theo bảng điểm → T0, T7.

5. Câu trả lời – nhóm B, C, D, E

B. Nguồn sai số

- **RMSE:** RMSE² = (lệch trung bình)² + (độ dao động)². Phụ thuộc vào bias dọc/ngang, dao động, và vài frame sai rất lớn (bình phương phóng đại). --demean cắt phần bias của GICP. Cần tách lỗi theo dọc / ngang / yaw và theo đoạn đường.

- **Vùng nghi có sai lớn:** đầu Track02 (yaw kẹt, ~4.6° sai trong 13 s); khúc cua (yaw GNSS trễ 1.1–1.4 s); lệch thời gian (dịch ±0.1 s làm Track01 1.26 → 1.68–1.74, vì 0.1 s × 9.6 m/s ≈ 0.96 m và √(0.905²+0.96²) ≈ 1.32 m); frame GICP bị cổng chặn loại.
- **Radian → độ, làm tròn:** nhân hằng số, không sai. CSV 6 chữ số → ≤0.00003°. float64 không đáng kể. Chỗ quan trọng duy nhất là float32 với toạ độ −5000 m (~0.5 mm) – đã xử lý bằng trừ gốc toạ độ trong gicp_refine.py.
- **Gyro cộng dồn:** bias tăng tuyến tính (25–30° sau 100 s), nhiễu trắng tăng theo √t. Khử: khớp tuyến tính c + b·t, bộ lọc bù, có thể thêm zero-velocity update hoặc bias trong trạng thái Kalman.
- **Voxel:** trên mặt phẳng gần như không lệch; cạnh/góc tối đa ~L/2 (0.15 m). Ảnh hưởng chính là mất chi tiết → đo bằng --map-res 0.2/0.3/0.5 (T4).
- **Tỉ lệ:** sai Δs/s làm điểm ở khoảng cách d lệch Δs/s·d (5 % ở 40 m = 2 m). Camera chỉ nhìn phía trước → lệch cùng chiều → có thể tạo bias dọc đường +0.17 m. *(cần kiểm chứng – T2, T4)*
- **h 2.06 vs 1.65 m:** mô hình độ sâu không biết chiều cao camera, tỉ lệ mét chỉ xấp xỉ, ảnh fisheye méo. 1.65 m cũng chỉ là suy luận → cần đo độc lập (T7).
- **init_guess nhiễu:** TA cố ý; pipeline cuối không dùng → không ảnh hưởng kết quả.
- **Điểm khởi đầu:** prior lệch ~0.9 m (T1) / ~0.23 m (T2), trong vùng hội tụ (--max-corr 1.0 m). Rủi ro ở đoạn prior lệch > 1 m → thống kê % hội tụ / % bị loại theo đoạn (T3).

C. Thuật toán

Phương pháp (Track01, lượt 1, cùng prior)	Lỗi vị trí (m)	Điểm
Point-to-plane (small_gicp)	0.681	0.99504
GICP	0.714	1.02813
PLANAR (tự viết, x/y/yaw)	0.719	1.03311
GICP 2 lượt + demean (pipeline)	0.616	0.92775 (yaw cũ)

- Ba thuật toán chênh ≤ 0.04 m → lỗi còn lại chủ yếu từ point cloud. So sánh chưa công bằng: chưa có point-to-plane 2 lượt + demean + yaw gyro (T6).
- **GICP vs NDT:** NDT so point cloud với lưới ô 4–8 m (mỗi ô một phân bố chuẩn), không ghép cặp, nhạy kích thước ô, ~14 s/frame, 0.836 m. GICP ghép cặp điểm gần nhất, mỗi điểm hai phía có hiệp phương sai từ 20 lân cận, ~0.06 s/frame, 0.803 m.
- **Vì sao Kalman:** GNSS 4 Hz vs camera 28 Hz; GNSS Track01 giật; RTS dùng dữ liệu tương lai. Track02 0.473 → 0.408; Track01 gần không đổi vì lỗi là trôi chậm.
- **Hình 15 (giả lập):** trên = 3 đường yaw (hướng chạy nhiễu, gyro trôi, thật); dưới = sai số của hướng chạy thô và bộ lọc bù. Bộ lọc bù lấy hình dạng gyro, kéo mức trung bình về hướng chạy.
- **Bài toán tương tự** *(từ trí nhớ, cần tra lại)*: Caselitz et al., Monocular camera localization in 3D LiDAR maps (IROS 2016); Cattaneo et al., CMRNet (ITSC 2019); Koide, hdl_localization (NDT + UKF + IMU); Autoware NDT scan matcher + EKF. Điểm chung: scan matching là **phép đo** đưa vào bộ lọc.

D. Hai track, độ lệch GICP, chỉnh tham số

- **Track01 vs Track02:** chỉ trộn một nguồn khi nó chính xác hơn cái nó sửa. GICP nhiễu ~0.5 m; prior Track01 lệch ~0.9 m (GICP giúp), Track02 ~0.23 m (GICP làm xấu). 2 lượt tốt vì lượt 2 bắt đầu gần hơn (0.714 → 0.679); 5 lượt kéo theo méo (0.801).
- **Độ lệch GICP** dọc +0.17 m, ngang −0.14 m: có khả năng do point cloud méo có hệ thống (tỉ lệ, gốc camera MapAnything, độ sâu xa). Demean chỉ bỏ phần hằng số.
- **Chỉnh tham số:** một lần một tham số, ưu tiên lý do vật lý, lọc offline trước. Thứ tự: --max-range 20/30/40 → point-to-plane 2 lượt + demean → tỉ lệ 0.75/0.85 → --sigma-s 1/2/4 → info_fuse (if_iso / if_aniso).
- **Lỗi trôi chậm GNSS:** lọc không khử được; cần nguồn tuyệt đối độc lập (scan matching) và mô hình lỗi GNSS là trạng thái trôi chậm (Gauss–Markov) trong Kalman với độ sửa GICP làm phép đo (T12). info_fuse.py là phiên bản bình phương tối thiểu của ý này.

- **Tự tính lại:** không có đáp án nên chỉ tính được chênh lệch giữa file (compare_csv.py) → dự đoán điểm, rồi so với Kaggle (T1).

E. Dò bảng điểm và Hình 21

- **Kết luận hợp lệ từ tách điểm:** Track01 = 0.905 m + 1.77° (3 cặp độc lập, chênh 0.0014); GNSS không lệch ngang (+0.025 m); không lệch thời gian (−3.6 ms); yaw đáp án là yaw thân xe.
- **Phi vật lý:** trọng số tổ hợp (ví dụ −4.58) và tổ hợp dao động cosin → "đo lại đáp án", không phải năng lực thuật toán.
- **Hình 21:** mỗi đường = điểm tốt nhất của một loại file nên chỉ đi xuống. Xanh dương đứng ở 1.253 từ 17/10 (file sạch chưa nộp). Cam giảm 18–19/10 theo cải tiến thật rồi đứng ở 0.789 (thời gian dồn vào dò). Xanh lá tụt đúng hai đợt dò 20/10 (0.555) và 21/10 (0.46), rồi đứng vì đã dừng.

6. Hai câu hỏi mới (thảo luận trong chat)

6.1 Sai số do mô hình AI hay do thuật toán của tôi?

Hai nguồn khác nhau: **sai số đầu vào** (point cloud do AI dựng: sai tỉ lệ, méo theo khoảng cách, lệch gốc camera) và **sai số thuật toán** (không hội tụ, cực tiểu địa phương, ghép cặp sai, tham số chưa tốt). Bằng chứng hiện có nghiêng về đầu vào: (1) ba thuật toán khác nhau cho lỗi gần như nhau 0.681–0.719 m; (2) độ sửa GICP lệch một hằng số – đặc trưng của méo có hệ thống; (3) Hessian không dự báo nhiễu từng frame (tương quan ≈ 0). Kết luận cần chứng minh bằng thí nghiệm T2 (point cloud hoàn hảo cắt từ bản đồ → đo sai số thuần thuật toán; thêm lỗi giả lập → xem có tái tạo bias +0.17 m không) và T3b (raw_ply vs 3cameras_ply cùng thuật toán). Cách nói khi thuyết trình: lỗi gốc ở point cloud AI, nhiệm vụ của thuật toán là phát hiện và bù (tỉ lệ theo mặt đường, giới hạn tầm, demean); phần méo thay đổi theo frame thì chưa bù được.

6.2 Tại sao độ giật GNSS Track01 cao (0.142 m so với 0.052 m)?

Repo chưa tìm nguyên nhân. Giả thuyết theo thứ tự khả năng: (1) **che chắn / multipath** – Track01 nhiều cây lớn và nhà cao hai bên (Hình 2); (2) **bộ định vị đổi chế độ** (topic là localizer_node đã xử lý, z hằng số; chuyển RTK fix ↔ float tạo bước nhảy); (3) **cách chạy** – chậm hơn (9.6 vs 10.5 m/s), dừng/rẽ nhiều hơn. Hệ quả: GNSS Track01 kém hơn nên GICP có chỗ để sửa. Kiểm chứng ở T11.

7. Nhiệm vụ cho Claude Code (theo thứ tự ưu tiên)

Mỗi nhiệm vụ ghi: mục tiêu, cách làm, đầu ra, tiêu chí hoàn thành, và có tốn lượt Kaggle không. Mọi script Python chạy trong image `sdc-reg` như `run_clean_pipeline.sh`. Kết quả ghi vào `docs/nhat-ky/10-tra-loi-cau-hoi.md` (file mới) + `docs/nhat-ky/data/`.

Nhiệm vụ	Kaggle?	Cách làm	Hoàn thành khi
T0 – Sao lưu file ngoài git	Không	Kiểm tra tồn tại và sao lưu c3s08_*, c3s10T2_*, result/chunks/, map_voxel_0.3m.pcd (2 track), phys_yawb_1.csv, rts_heading_2.csv vào ~/SDC-midterm01/backup/ kèm sha256sum. Ghi danh sách vào nhật ký.	Có file backup + checksum; nhật ký liệt kê file nào thiếu.
T1 – Điểm file sạch, dự đoán vs thật	Không (đã xếp hàng)	Đọc điểm Kaggle của clean_cfz1_tau7 / clean_cfz2_tau15 và if_iso_* / if_aniso_*. Với mỗi file, dự đoán điểm từ chênh lệch (compare_csv.py) so với file đã có điểm; lập bảng dự đoán vs thật. Nhắc người dùng chọn “Use for Final Score” (việc làm tay trên web).	Bảng trong nhật ký; người dùng đã chọn bài chấm.
T2 – Thí nghiệm “point cloud hoàn hảo” (AI vs thuật toán)	Không	Viết tools/synthetic_gicp.py: với ~200 frame Track01 rải đều, cắt điểm bản đồ trong vùng camera nhìn thấy quanh pose prior (FLU, x > 0, tầm ≤ 40 m, góc ngang theo FOV 3 camera), đưa về hệ camera, giảm mẫu như source_cloud. (a) Dời pose khởi đầu một lượng biết trước (dọc 0.5 m, ngang 0.3 m, yaw 2°), chạy đúng cấu hình GICP của pipeline → sai số thuần thuật toán. (b) Thêm lỗi giả lập: tỉ lệ ±3 % / ±5 %, nhiễu độ sâu $\sigma = k \cdot d^2$, lệch gốc camera 0.3 m → đo độ sửa trung bình dọc/ngang. (c) So với độ sửa thật (+0.17 / −0.14 m).	Bảng (a) sai số thuần thuật toán, (b) lỗi giả lập nào tái tạo được bias thật; kết luận câu hỏi 6.1; 1 hình cho report.
T3 – Ngân sách sai số và vùng sai lớn	Không	Dùng file debug GICP lượt 2 + segment_analysis.py: tách độ sửa theo dọc/ngang; tính trung bình, độ lệch chuẩn, % hội tụ, % bị cổng chặn loại theo 10 đoạn; đánh dấu đoạn đầu xe đứng, khúc cua (độ cong), đoạn có độ giật GNSS cao. (b) Chạy cùng GICP trên raw_ply ×(1.62/h) và 3cameras_ply để so đầu vào.	Bảng theo đoạn + nhận xét đoạn nào nghi sai lớn; so sánh raw_ply vs 3cameras.
T4 – Quét tham số offline	Không	Chỉ đổi một tham số mỗi lần, cùng prior: --map-res 0.2/0.3/0.5; --max-range 20/30/40; tỉ lệ ×0.95/×1.0/×1.05 (tức 0.76/0.8/0.84 quy đổi). Với mỗi cấu hình ghi: độ sửa trung bình dọc/ngang, độ lệch chuẩn sau làm mượt, % bị loại, map_fit.py (chỉ để loại cấu hình hỏng). Chọn ≤ 3 ứng viên.	Bảng quét; danh sách ≤ 3 ứng viên kèm lý do vật lý.
T5 – Giải thích sai tỉ lệ ×3 cho report	Không	Dùng docs/nhat-ky/data/baseline_partial/ và Hình 8: viết 1 đoạn + 1 hình cho thấy point cloud ×3 nhỏ hơn bản đồ và NDT trôi từ frame 17.	Đoạn văn tiếng Anh + hình.
T6 – Ứng viên Kaggle (pipeline trực tiếp)	Có, ≤ 3 file/đợt	Từ T4 chọn: (1) --max-range tốt nhất; (2) point-to-plane 2 lượt + demean + yaw gyro τ 7 (so công bằng với GICP); (3) tỉ lệ 0.75 hoặc 0.85. Tạo bằng run_clean_pipeline.sh có tham số, không chỉnh tay CSV. Ghi giả thuyết trước khi nộp.	3 file + điểm + nhật ký; cập nhật bài chấm nếu tốt hơn.
T7 – Làm sạch z/roll/pitch, đo chiều cao camera	Không (1 lượt nộp nếu khác biệt)	Thay nguồn z/roll/pitch từ c3s08: z = độ cao mặt đất bản đồ dưới vị trí xe + chiều cao camera; roll/pitch từ gia tốc kế IMU (hướng trọng lực) hoặc pháp tuyến mặt đường bản đồ. So với giá trị c3s08. Đo chiều cao camera độc lập (ground_scale.py đã in độ cao mặt đất bản đồ dưới GNSS). Chạy lại pipeline, compare_csv.py với clean_*.	Pipeline không còn phụ thuộc c3s08; RMS chênh vị trí; con số chiều cao camera đo độc lập.
T8 – Mục “Leaderboard analysis” cho report	Không	Viết tiếng Anh, ~nửa trang, trong Problems and Solutions: vấn đề (chỉ có tổng điểm) → cách dò cặp ± ($P(+L)^2 + P(-L)^2 - 2P^2 = 2L^2$) → 4 kết luận hợp lệ và thay đổi thuật toán chúng dẫn tới → giới hạn (tổ hợp phi vật lý, 0.46/0.19) → quyết định dừng 21/10, 85/152 lần nộp là dò. Câu chốt: all graded submissions come directly from the localization pipeline.	Đoạn report + giải thích Hình 21.

Nhiệm vụ	Kaggle?	Cách làm	Hoàn thành khi
T9 – Tài liệu cho thuyết trình	Không	6–8 slide (4–6 phút): 3 phát hiện chính; pipeline (Hình 17) + vì sao Track02 không dùng GICP; GICP vs NDT; kết quả theo Hình 20 với điểm sạch; minh bạch về dò bảng điểm và dùng AI. Tra và xác nhận 4 tài liệu tham khảo mục C, thông tin LoGeR/MapAnything.	Dàn slide + câu trả lời dự phòng (vì sao demean, vì sao 0.8, vì sao 2 lượt).
T10 – Câu hỏi baseline 1.79 vs trôi 91 m	Không	Tính điểm của baseline_partial so với GNSS (track_error.py) để cho thấy điểm local phải là hàng chục mét; soạn câu hỏi ngắn cho TA hoặc ghi chú trong report.	Con số + câu hỏi cho TA.
T11 – Nguyên nhân độ giật GNSS Track01	Không	Viết tools/gnss_jitter.py: độ giật theo cửa sổ 2 s (phần dư sau làm mượt / sai phân bậc 2) cho cả hai track; vẽ theo thời gian; phát hiện bước nhảy rời rạc; tương quan với tốc độ, độ cong; tính mức che chắn từ map.pcd (số điểm cao > 3 m trong ±20 m ngang quanh vị trí) và tương quan với độ giật.	Hình + bảng; xếp hạng 3 giả thuyết bằng số liệu.
T12 – Kalman kết hợp GICP + IMU (nếu còn thời gian)	Có, 1–2 file	Trạng thái thêm lỗi GNSS dạng Gauss–Markov (bx, by); GNSS đo vị trí + lỗi; độ sửa GICP (đã demean) đo vị trí thật với σ từ dữ liệu; gyro đo ω. Tham số chọn bằng maximum likelihood như rts_smoother.py. So với fuse_to_csv (Gauss 2 s).	Script + điểm; kết luận cho câu "cần cách tính khác".
T13 – Dọn repo	Không	Cập nhật README (điểm cũ), sửa reproduce_submissions.sh chỉ so 34 file, thống nhất thư mục build, thêm kiểm tra lỗi đầu vào cho gicp_refine.py (--pose-source thiếu).	Commit dọn dẹp.

8. Checklist cuối

- [ ] Trước 29/10 09:00: nộp file sạch (hoặc bản sạch tốt hơn từ T6/T7) và chọn "Use for Final Score" cho mỗi track.
- [ ] 29/10: thuyết trình 4–6 phút, dùng điểm sạch, không dùng 0.46/0.19.
- [ ] 05/11 23:59: E3 = code (không data), result_1.csv, result_2.csv, report PDF tiếng Anh (Pipeline, Contributions, Problems and Solutions, Workload Distribution), README.
- [ ] Report khai báo tham số chọn bằng Kaggle: tỉ lệ 0.8, số lượt 2, τ, trục gyro z, tắt hiệu chỉnh trượt, demean (có dựa vào probe_lat).

Phụ lục A (các trang tiếp theo): nguyên bản "Giải thích toàn bộ repo SDC-midterm-1-" kèm ghi chú tay, đánh máy và đánh dấu vàng của người dùng. Số trang trong bảng mục 3 là số trang của phụ lục này.

Giải thích toàn bộ repo SDC-midterm-1-

Cuộc thi Midterm01 – Localization (Kaggle)
Môn Self-Driving Cars (SDC) 2026

Tài liệu dành cho người **chưa biết gì** về lập trình, định vị hay xe tự hành. Mọi file, mọi hàm, mọi tham số – kể cả phần tạm, phần chưa xong và phần có lỗi.

Repo: github.com/mochihat/SDC-midterm-1- (private) · nhánh main · commit 0ae197f (21/10/2026 19:11)
Viết ngày 21/10/2026 để chuẩn bị thuyết trình ngày 29/10/2026

Mục lục

Đọc tài liệu này thế nào 4

Thứ tự đọc gợi ý 4

Cách tài liệu được viết và mức độ minh bạch 4

Tóm tắt một trang 6

Chương 1. Bài toán: định vị xe trong cuộc thi Midterm01 7

1.1 Localization (định vị) là gì? 7

1.2 Vì sao xe tự hành cần định vị chính xác? 7

1.3 Cuộc thi yêu cầu gì? 7

1.4 Dữ liệu đầu vào 8

1.5 Đầu ra: file nộp Kaggle 8

1.6 Cách chấm điểm 9

1.7 Luật quan trọng: không được "dò đáp án" qua bảng điểm 9

Chương 2. Kiến thức nền tảng (chỉ những gì repo thực sự dùng) 10

2.1 Toạ độ và hệ toạ độ (coordinate frame) 10

2.2 Pose: x, y, yaw – và ma trận 4×4 11

2.3 Các cảm biến 12

2.4 Point cloud, file PLY / PCD, và giảm mẫu voxel 13

2.5 Độ sâu từ ảnh và vấn đề tỉ lệ (scale) 14

2.6 Bản đồ LiDAR 14

2.7 Scan matching: ICP, point-to-plane, GICP, NDT 15

2.8 Lọc theo thời gian: bộ lọc Kalman, CTRV, RTS smoother, maximum likelihood 17

2.9 Bộ lọc bù (complementary filter) cho yaw 18

2.10 Nội suy, làm mượt Gauss, lọc trung vị 19

2.11 Phần mềm: ROS 2, rosbag, PCL, small_gicp, Docker 19

Chương 3. Cấu trúc repo: từng thư mục, từng file 20

3.1 Thư mục gốc 20

3.2 docker/ 21

3.3 src/localization/ – gói ROS 2 (code C++ của TA, đã mở rộng) 22

3.4 tools/ – 30 công cụ 23

3.5 docs/ 24

3.6 video/ – bộ dựng video bài giảng 24

Chương 4. Luồng chạy tổng thể (pipeline): từ dữ liệu thô tới file nộp 26

4.1 Pipeline gốc của TA (baseline) 26

4.2 Pipeline cuối – "sạch" (dùng làm bài chấm) 26

4.3 Những dữ liệu trung gian quan trọng 28

Chương 5. Giải thích chi tiết từng file code 30

5.1 Node C++ localization (src/localization/) . . . 30
5.2 processing_codes/ – chuyển đổi và ghép (Python) . . . 33
5.3 Các script shell ở thư mục gốc . . . 34
5.4 tools/ – từng công cụ . . . 34
5.5 docs/report/make_report.py và video/ . . . 39
Chương 6. Toán học và thuật toán phía sau . . . 40
6.1 Scan matching như một bài toán tối ưu . . . 40
6.2 Bộ lọc Kalman mở rộng với mô hình CTRV và RTS smoother . . . 40
6.3 Bộ lọc bù cho yaw . . . 41
6.4 Hessian, thông tin theo hướng và bộ ghép bình phương tối thiểu . . . 41
6.5 Vì sao trừ độ lệch trung bình (--demean) . . . 42
6.6 Toán của việc "dò bảng điểm" (chỉ để hiểu – không được dùng chấm) . . . 43
Chương 7. Các tham số quan trọng và ảnh hưởng của chúng . . . 44
7.1 Pipeline sạch . . . 44
7.2 Số lượt GICP, phương pháp khớp, τ, trục gyro (Track01, chỉ đổi đúng một thứ) . . . 45
7.3 Track02: τ của bộ lọc bù yaw . . . 45
7.4 Tham số của node C++ (NDT) . . . 45
Chương 8. Cài đặt và chạy lại từ đầu, từng bước . . . 46
8.1 Chuẩn bị (một lần) . . . 46
8.2 Build và (tuỳ chọn) xem bằng RViz . . . 46
8.3 Bước 1 – GNSS + Kalman (trong container ROS) . . . 46
8.4 Bước 2 – Chuẩn bị cho GICP . . . 47
8.5 Bước 3 – Pipeline sạch: một lệnh (trên máy) . . . 47
8.6 Bước 4 – Kiểm tra và nộp . . . 47
Chương 9. Kết quả, điểm mạnh, hạn chế và hướng cải thiện . . . 48
9.1 Kết quả hiện tại (21/10/2026 tối) . . . 48
9.2 Điểm mạnh . . . 49
9.3 Hạn chế, phần chưa xong, lỗi đã biết . . . 49
9.4 Hướng cải thiện (đã có cơ sở trong repo) . . . 50
9.5 Việc cần làm trước 29/10 (checklist) . . . 50
Phụ lục A – Thuật ngữ Việt – Anh . . . 51
Phụ lục B – Danh sách toàn bộ file CSV đã nộp / định nộp . . . 53
Track01 . . . 53
Track02 . . . 54
Phụ lục C – Lịch sử commit . . . 55

Đọc tài liệu này thế nào

Tài liệu này giải thích **mọi thứ** trong repo **SDC-midterm-1-** – mã nguồn và nhật ký làm bài cho cuộc thi **Midterm01 Localization** trên Kaggle của môn Self-Driving Cars. Mục tiêu: một người không biết lập trình, không biết định vị, không biết xe tự hành vẫn đọc hiểu được bài toán là gì, code làm gì, vì sao làm vậy, và kết quả ra sao.

Thứ tự đọc gợi ý

- **Chỉ có 15 phút:** đọc mục *Tóm tắt một trang* ngay sau đây, Chương 1 và Chương 4 (sơ đồ pipeline).
- **Muốn hiểu nền tảng:** Chương 2 giải thích từ đầu tọa độ, cảm biến, point cloud, ICP/NDT/GICP, bộ lọc Kalman… luôn đi từ ví dụ đời thường trước rồi mới tới công thức.
- **Muốn biết từng file:** Chương 3 (cây thư mục, vai trò từng file) và Chương 5 (giải thích từng file code, từng hàm).
- **Muốn hiểu toán:** Chương 6. **Muốn chạy lại:** Chương 8. **Muốn biết kết quả và hạn chế:** Chương 9.
- **Gặp từ lạ:** tra Phụ lục A (thuật ngữ Việt – Anh).

Cách tài liệu được viết và mức độ minh bạch

Trước khi viết, toàn bộ repo đã được đọc: 286 file (code C++, Python, shell, Dockerfile, launch file, cấu hình RViz, 9 file nhật ký tiếng Việt, các file dữ liệu phân tích, 111 file CSV đã nộp Kaggle, bộ công cụ làm video) và lịch sử 36 commit. Số liệu trong tài liệu lấy từ chính repo (chủ yếu từ `docs/nhat-ky/` và `CLAUDE.md`). Tài liệu **không** chạy lại code (máy viết tài liệu không có dữ liệu 50 GB của cuộc thi), nên mọi con số là con số đã được ghi lại trong repo.

Có năm loại khung màu xuất hiện xuyên suốt:

Ví dụ đời thường
Một ví dụ quen thuộc để hiểu khái niệm trước khi gặp thuật ngữ.

Ý chính
Điều quan trọng nhất của một mục, nên nhớ.

Ghi chú
Thông tin bổ sung, chi tiết kỹ thuật.

Minh bạch: điểm cần biết
Hack, đường tắt, phần chưa xong, lỗi, chỗ không nhất quán, hoặc phần dính tới luật của TA. Tài liệu không giấu những chỗ này.

Không chắc chắn
Chỗ người viết tài liệu không chắc code/kết quả làm gì – được ghi rõ là không chắc.

Minh bạch: ai làm, làm bằng gì

Repo ghi (`CLAUDE.md`, `docs/nhat-ky/06`) rằng bài được làm **một mình** (đội Kaggle tên **"11"**, 1 thành viên), và phần lớn code được viết **với sự hỗ trợ của AI** (Claude). Khi thuyết trình nên nói rõ hai điều này; mục "Workload Distribution" của báo cáo = 100 % một người.

Đề bài của bạn nói "code của nhóm mình" – nếu thực tế có thêm thành viên thì repo hiện chưa ghi nhận điều đó.

Tóm tắt một trang

Tóm tắt một trang

Bài toán: cho một chiếc xe chạy ~110–120 giây trong thành phố. Với mỗi khung hình camera (3011 frame ở Track01, 3299 frame ở Track02), phải đoán vị trí **x, y** (mét) và hướng mũi xe **yaw** (radian) trên một bản đồ 3D quét bằng LiDAR. Điểm = sai số vị trí (m) + 0.2 × sai số hướng (độ); càng thấp càng tốt.

Code mẫu của TA khớp point cloud dựng từ ảnh vào bản đồ bằng thuật toán NDT, nhưng dùng sai tỉ lệ point cloud (×3 thay vì ≈ ×9) nên trôi ~100 m. Điểm baseline: 1.79075 (Track01), 1.92587 (Track02).

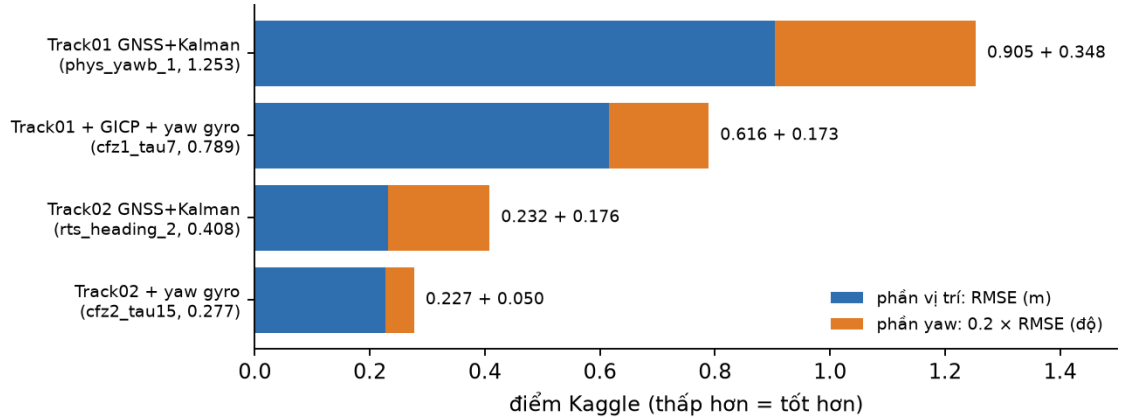
Phát hiện chính: (1) rosbag có sẵn một topic GNSS rất gần đáp án; (2) tỉ lệ point cloud đúng phải đo từ độ cao mặt đường; (3) yaw tốt nhất lấy từ con quay hồi chuyển (gyro) của IMU, không phải từ GNSS.

Pipeline cuối ("sạch", dùng để chấm): GNSS → bộ lọc Kalman CTRV + RTS smoother → (Track01) GICP khớp point cloud 3 camera vào bản đồ, 2 lượt, trừ độ lệch trung bình → yaw = bộ lọc bù gyro + hướng chạy. Track02 không dùng scan matching (làm xấu đi).

Điểm: cùng phương pháp đã đạt 0.78914 (Track01) và 0.27654 (Track02). Bản dựng lại "sạch" (`clean_cfz1_tau7.csv`, `clean_cfz2_tau15.csv`) **chưa được nộp**, dự kiến ~0.79 / ~0.28.

Điểm đang hiện trên bảng xếp hạng (0.46 / 0.19, hạng 2) đến từ việc **dò đáp án qua bảng điểm** – TA đã nói (20/10) cách này **không được** dùng cho bài chấm. Repo đã dừng việc này và tách riêng các file đó.

Việc phải làm trước 29/10 09:00: nộp hai file sạch và chọn chúng là "Use for Final Score" trên Kaggle.



Hình 1. Điểm Kaggle tách thành phần vị trí và phần yaw (số liệu từ docs/nhat-ky/05 và 08). Yaw từ gyro cắt giảm mạnh phần màu cam ở cả hai track.

Note : phần lớn sai số do RMSE

Tìm xem RMSE phụ thuộc vào các chỉ số nào

Chương 1. Bài toán: định vị xe trong cuộc thi Midterm01

1.1 Localization (định vị) là gì?

> **Ví dụ đời thường**
> Bạn bị bịt mắt, đưa tới một góc phố quen rồi mở mắt ra. Bạn nhìn quanh: "bên trái là tiệm bánh, phía trước là cột đèn, bên phải có hàng cây" – rồi so với **tấm bản đồ trong đầu** và biết ngay mình đứng ở đâu, quay mặt về hướng nào. Đó chính là **localization**: so cái mình đang nhìn thấy với bản đồ có sẵn để suy ra vị trí và hướng của mình.

Trong xe tự hành, **localization** (định vị) là bài toán trả lời câu hỏi: "*Xe đang ở đâu trên bản đồ, và mũi xe đang chĩa về hướng nào?*" Câu trả lời gọi là **pose** (tư thế). Trong cuộc thi này pose chỉ gồm 3 số: toạ độ **x, y** (mét) và góc **yaw** (góc quay của mũi xe quanh trục thẳng đứng).

1.2 Vì sao xe tự hành cần định vị chính xác?

- Một làn đường chỉ rộng ~3–3.5 m. GPS điện thoại thường sai 3–10 m → không biết xe đang ở làn nào.
- Xe tự hành lên kế hoạch đường đi trên bản đồ có độ chính xác cỡ cm (vạch kẻ, lề, đèn tín hiệu). Nếu không biết chính xác mình ở đâu trên bản đồ đó thì bản đồ vô dụng.
- Vì vậy xe kết hợp nhiều nguồn: GNSS (vệ tinh), IMU (cảm biến quán tính), và **so khớp cảm biến với bản đồ** (scan matching) – chính là chủ đề của cuộc thi.

1.3 Cuộc thi yêu cầu gì?

Đề: **dùng point cloud dựng từ ảnh camera, khớp vào bản đồ LiDAR có sẵn để định vị xe** ("Design your own scan matching algorithm"). Được phép dùng thêm ảnh các camera khác, IMU và GNSS. Có hai track (hai đoạn đường khác nhau) chấm riêng trên hai trang Kaggle:

	Track01	Track02
Trang Kaggle	`sdc-2026-midterm-01-track-01`	`sdc-2026-localization-track-02`
Số frame phải nộp	3011	3299
Thời lượng / quãng đường (từ GNSS)	109.7 s / 1052.9 m	118.5 s / 1247.4 m
Tốc độ trung bình	9.60 m/s (~35 km/h)	10.53 m/s (~38 km/h)
Điểm baseline của TA	1.79075	1.92587

- Hạn Kaggle: **29/10/2026 09:00** (giờ UTC+8). Tối đa **15 lần nộp/ngày/track** (ngày đổi lúc 08:00 giờ máy).
- Nộp E3 trước **05/11 23:59**: code (không kèm dữ liệu), `result_1.csv`, `result_2.csv`, báo cáo PDF (tiếng Anh, 4 mục: Pipeline, Contributions, Problems and Solutions, Workload Distribution), README.
- Thuyết trình 4–6 phút ngày 29/10 (20 điểm). Báo cáo 30 điểm. Xếp hạng Kaggle: hạng 1/2/3 = 56/52/48 × r, top 50 % = 42r, top 75 % = 36r, trên baseline = 30r, với r = 0.3 cho Track01/02 (theo `CLAUDE.md`).

> **Không chắc chắn**
> Phần thang điểm môn học có hai phiên bản trong repo: `docs/nhat-ky/01` ghi "mỗi track: hạng 1: 28, hạng 2: 26..." còn `CLAUDE.md` (mới hơn, theo slide cập nhật 21/10) ghi "56/52/48 × r, r = 0.3". Tài liệu này không có slide gốc để kiểm tra; nên tin bản mới hơn và xem lại slide của TA.

1.4 Dữ liệu đầu vào

Mỗi track là một thư mục (không nằm trong git vì quá lớn, ~25 GB/track) gồm:

Thành phần	Là gì	Kích thước (Track01)
`map.pcd`	Bản đồ 3D quét bằng LiDAR: ~60 triệu điểm (x, y, z, cường độ), dạng chữ (ASCII)	2.70 GB
`Track1.db3` + `metadata.yaml`	Rosbag (bản ghi cảm biến ROS 2): IMU 100 Hz, 3 luồng video h265, topic GNSS `/localizer_node/gnss_pose` 4 Hz	250 MB
`raw_ply/`	3011 point cloud dựng từ **ảnh camera trước** bằng mô hình AI **LoGeR**; tên file chứa thời điểm chụp (nano-giây); ~20–41 nghìn điểm/frame, có màu	5.09 GB
`3cameras_ply/`	3011 point cloud dựng từ **3 camera** bằng mô hình AI **MapAnything**; ~330 nghìn điểm/frame; có `index.csv` (số thứ tự, thời điểm, tên file)	14.75 GB
`Track1_images/`	Ảnh 3 camera (trước, trước-trái, trước-phải), fisheye 1440×928, + `camera_info/` (thông số ống kính, vị trí tương đối 3 camera, mặt nạ nắp capo `hood_mask.npy`)	–
`init_guess.txt`	Ma trận 4×4: pose ban đầu **có cộng nhiễu** (Track02 lệch yaw 14.85°)	–



Hình 2. Ảnh từ camera trước (fisheye) của Track01 – lấy từ video/assets/figures/cam_f.jpg trong repo.

1.5 Đầu ra: file nộp Kaggle

Một file CSV, mỗi frame một dòng, theo đúng thứ tự thời gian, 4 cột: `idx,x,y,yaw`. `x, y` tính bằng mét trong hệ toạ độ của bản đồ; `yaw` tính bằng **radian** (trong khoảng −π...π). Ví dụ (3 dòng đầu của một file Track01):

```
idx,x,y,yaw
0,-4800.720977,2274.626087,2.323874
1,-4800.725199,2274.632284,2.323874
2,-4800.729455,2274.638531,2.323874
```

Ý nghĩa: ở frame 0 xe ở x = −4800.72 m, y = 2274.63 m, mũi xe quay 2.3239 rad ≈ 133.2° so với trục x của bản đồ (tức là hướng gần Tây-Bắc).

1.6 Cách chấm điểm

Gọi Δp_i là khoảng cách (m) giữa vị trí mình đoán và vị trí thật ở frame i, $\Delta\theta_i$ là sai số yaw tính bằng **độ**. Điểm của một file là:

$$E = \sqrt{\frac{1}{n}\sum_{i=1}^{n} \|\Delta p_i\|^2} \; + \; 0.2 \cdot \sqrt{\frac{1}{n}\sum_{i=1}^{n} \Delta\theta_i^2}$$

RMSE vị trí (m) + 0.2 × RMSE yaw (độ)

Ví dụ đời thường

RMSE (root mean square error – căn bậc hai của trung bình bình phương sai số) giống "sai số trung bình" nhưng phạt nặng những lần sai lớn. Ví dụ 3 frame sai 0.1 m, 0.1 m và 1.0 m: trung bình thường = 0.4 m, nhưng RMSE = √((0.01 + 0.01 + 1)/3) ≈ 0.58 m.
Hệ số 0.2 nghĩa là **sai 1° yaw bị phạt bằng sai 0.2 m vị trí**. Một file sai vị trí 0.6 m và sai yaw 0.9° được 0.6 + 0.18 = 0.78 điểm.

Ghi chú

Kaggle chỉ cho biết **tổng điểm**, không cho biết riêng phần vị trí và phần yaw. Từ 21/10 Kaggle chỉ hiển thị 2 chữ số thập phân và **cắt bớt** (0.789 → 0.78), không làm tròn. Điểm 5 chữ số cũ được lưu trong `docs/nhat-ky/data/kaggle_submissions.csv` và `precise_scores.json`.

1.7 Luật quan trọng: không được "dò đáp án" qua bảng điểm

Ngày 20/10 TA trả lời (và slide "mid01 Updates & Adjustments" ngày 21/10 nhắc lại):

- Dùng điểm Kaggle để **chọn mô hình, chỉnh tham số bình thường** (so sánh pipeline) – **được**.
- Nộp có hệ thống các file làm lệch có kiểm soát để suy ra lỗi của đáp án ẩn rồi sửa quỹ đạo theo đó ("**leaderboard-based fitting**"), kể cả theo từng đoạn đường – **không được** dùng cho bài chấm/xếp hạng. Chỉ được trình bày trong báo cáo như một phân tích.
- Phải chỉ định một bài nộp **tạo trực tiếp từ pipeline định vị** làm bài chấm.

Minh bạch: điểm cần biết

Trước khi có trả lời này, repo **đã** làm khá nhiều việc dò bảng điểm (file `probe_*`, `yawproj_*`, `posproj_*`, `lb_posyaw_*`, `corr_r*`) – xem Chương 6.6 và Chương 9. Những file này cho điểm tốt nhất đang hiện trên bảng (0.46 / 0.19, hạng 2 cả hai track) nhưng **không được dùng để chấm**. Sau 21/10 repo đã dừng việc này, rút file khỏi hàng đợi nộp, và dựng lại bài chấm "sạch".

Khi thuyết trình, cần nói điểm "thật" của pipeline (~0.79 / ~0.28 – khi đã nộp và có điểm), không nói 0.46 / 0.19.

Chương 2. Kiến thức nền tảng (chỉ những gì repo thực sự dùng)

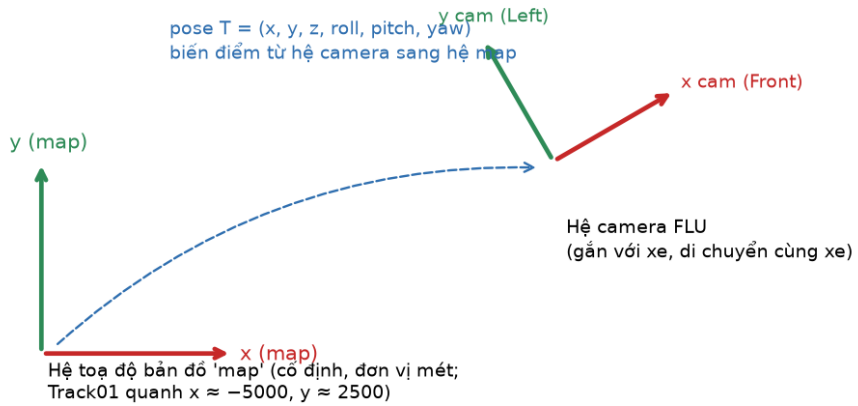
Chương này giải thích từng khái niệm theo thứ tự: ví dụ đời thường → ý tưởng → (nếu cần) công thức. Mỗi mục ghi rõ khái niệm được dùng ở file nào trong repo.

2.1 Toạ độ và hệ toạ độ (coordinate frame)

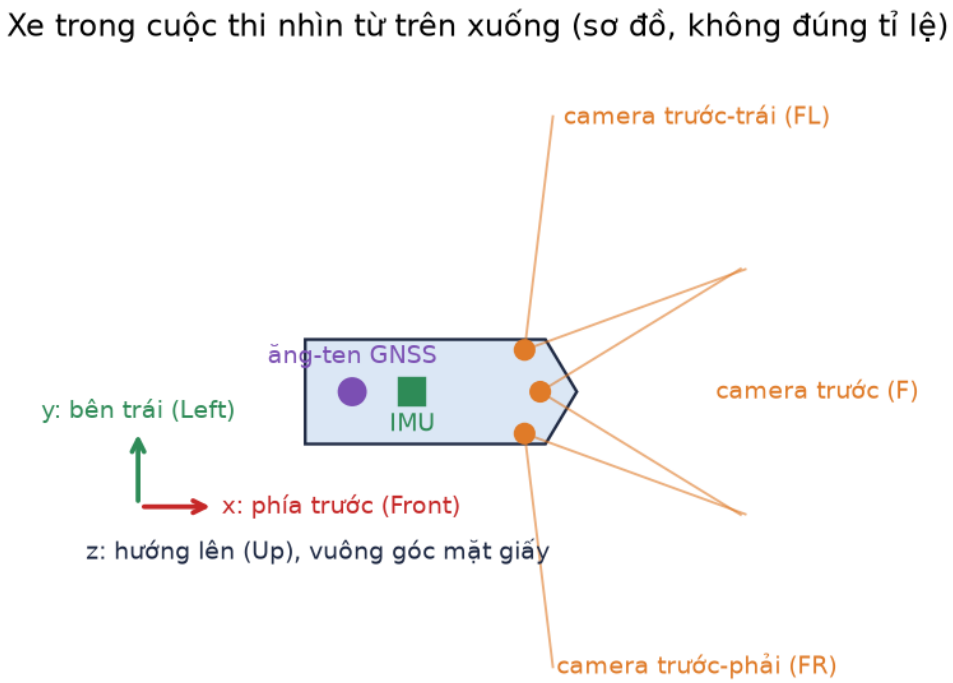
> **Ví dụ đời thường**
> Nói "quán cà phê cách nhà mình 200 m về phía Bắc" chỉ có nghĩa khi người nghe biết "nhà mình" ở đâu và "phía Bắc" là hướng nào. Một **hệ toạ độ** chính là một gốc (điểm 0) cộng với các hướng trục.

Repo dùng hai loại hệ toạ độ:

- **Hệ bản đồ "map"**: cố định, đơn vị mét. Bản đồ LiDAR, GNSS và đáp án đều dùng hệ này. Toạ độ rất lớn (Track01 quanh x ≈ −5000, y ≈ 2500; độ cao z ≈ 108–110) vì gốc toạ độ nằm ở xa.
- **Hệ camera FLU** (Front-Left-Up): gắn vào xe, gốc ở camera trước; trục x chĩa về **phía trước**, y sang **trái**, z **lên trên**. Point cloud dựng từ ảnh nằm trong hệ này (header của file `3cameras_ply` ghi rõ `coordinate_frame=front_camera_FLU_x_front_y_left_z_up`).



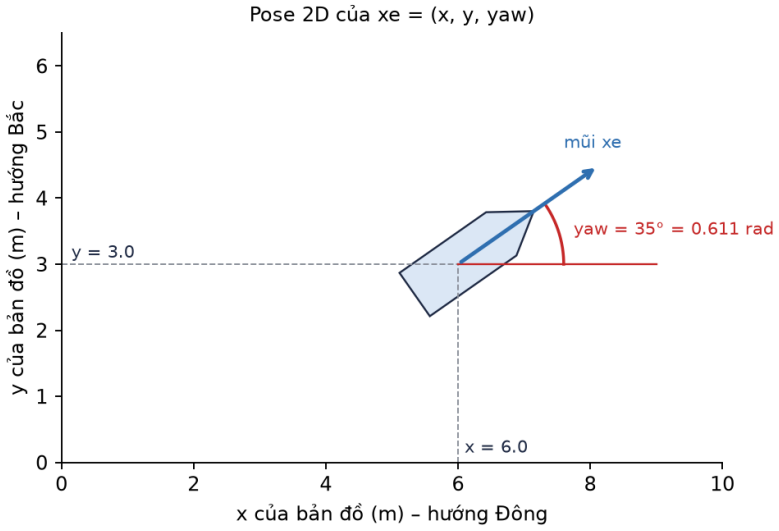
Hình 3. Hai hệ toạ độ. Pose của xe là phép biến đổi đưa một điểm từ hệ camera sang hệ bản đồ.



Hình 4. Các cảm biến trên xe và hệ trục FLU (sơ đồ minh hoạ, không đúng kích thước thật).

2.2 Pose: x, y, yaw – và ma trận 4×4

Pose = vị trí + hướng. Trong không gian 3D đầy đủ có 6 số (6 bậc tự do – 6DoF): x, y, z và ba góc **roll** (lăn – nghiêng sang bên), **pitch** (chúi – ngóc đầu lên/xuống), **yaw** (quay trái/phải). Cuộc thi chỉ chấm x, y, yaw vì xe chạy trên mặt đường gần như phẳng.



Hình 5. Pose 2D. yaw đo từ trục x của bản đồ, ngược chiều kim đồng hồ là dương.

Trong code C++ và trong các file `*_result.txt`, pose được lưu dưới dạng **ma trận biến đổi 4×4**:

```
T = | r11 r12 r13  x |      R = khối 3×3 (r..): phép quay (chứa roll, pitch, yaw)
    | r21 r22 r23  y |      t = (x, y, z): vị trí
    | r31 r32 r33  z |
    |  0   0   0   1 |
```

$$p_{map} = R\,p_{cam} + t$$

R là ma trận quay (chứa roll, pitch, yaw), t là vị trí (x, y, z). Nhân T với một điểm trong hệ camera cho ra điểm đó trong hệ bản đồ. Lấy yaw từ R bằng công thức `yaw = atan2(R[1][0], R[0][0])` – đúng như script của TA `txt_to_idx_x_y_yaw.py` làm. Mỗi dòng của `*_result.txt` là 12 số = 3 hàng đầu của T.

Ghi chú

Radian và độ: 180° = π rad ≈ 3.1416 rad, 1 rad ≈ 57.3°. Kaggle nhận yaw bằng radian, nhưng chấm sai số yaw bằng độ. **Gói góc (wrap):** 179° và −179° chỉ cách nhau 2°, không phải 358°. Mọi file trong repo có hàm `wrap(a) = (a + π) mod 2π − π` để đưa góc về khoảng −π…π; còn `np.unwrap` làm ngược lại (nối liền chuỗi góc, bỏ các bước nhảy 2π) để tính đạo hàm và làm mượt.

Ghi chú

Quaternion: ROS lưu hướng bằng quaternion (4 số x, y, z, w) thay vì 3 góc. Code đổi quaternion sang yaw bằng $yaw = atan2(2(wz + xy), 1 - 2(y^2 + z^2))$ (hàm `read_gnss` trong `gnss_to_csv.py`).

Note : việc này tạo sai số bao nhiêu

2.3 Các cảm biến

2.3.1 Camera

Ba camera fisheye (góc rộng) ở phía trước xe. Ảnh 2D không cho biết khoảng cách. Cuộc thi đã dùng hai mô hình AI (**LoGeR** cho camera trước, **MapAnything** cho 3 camera) để **đoán độ sâu** và dựng sẵn thành point cloud – repo không tự dựng.

có thể dùng tương tự để suy ra cách tính chính xác toán học

2.3.2 LiDAR

Cảm biến bắn tia laser và đo thời gian phản xạ → ra hàng trăm nghìn điểm 3D chính xác cỡ cm. Trong cuộc thi LiDAR chỉ dùng để **làm bản đồ trước** (`map.pcd`); lúc chạy xe không có LiDAR, chỉ có camera.

2.3.3 GNSS

Ví dụ đời thường

GPS trên điện thoại là một loại GNSS (Global Navigation Satellite System – hệ thống định vị vệ tinh, gồm GPS, GLONASS, Galileo, BeiDou…).

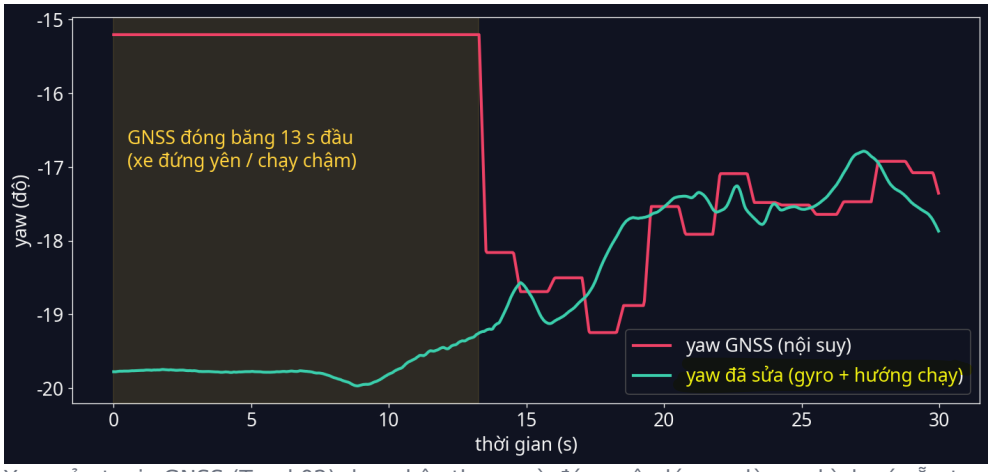
Topic `/localizer_node/gnss_pose` trong rosbag cho pose (x, y, yaw) 4 lần/giây, cùng hệ toạ độ với bản đồ. Phân tích trong repo (`docs/nhat-ky/02`) cho thấy:

- z luôn hằng số (110.132 ở Track01) → đây là **pose phẳng đã qua xử lý** (tên topic là "localizer_node"), không phải GNSS thô. Nó **rất gần đáp án**: chỉ nội suy topic này đã được 1.31 / 0.57 điểm, tốt hơn hẳn baseline.
- Yaw của topic này thực ra được **tính từ chính các vị trí GNSS** (hướng dịch chuyển trong 1 s), chỉ cập nhật ~1.25 s/lần (dạng bậc thang), nên **trễ 1.1–1.4 s** và **đứng yên khi xe dừng** (Track02 giữ −15.2° suốt 13 s đầu, trong khi yaw thật ≈ −19.8°).
- GNSS Track01 "giật" gấp ~3 lần Track02 (0.142 m so với 0.052 m) → Track01 khó hơn.

là sao

việc này có ảnh hưởng gì ko? bằng chứng nó có ảnh hưởng hoặc ko?

tại sao "giật" gấp 3 lần



Hình 6. Yaw của topic GNSS (Track02) dạng bậc thang và đứng yên lúc xe dừng – hình có sẵn trong repo.

2.3.4 IMU: con quay hồi chuyển (gyro) và gia tốc kế

> **Ví dụ đời thường**
> Nhắm mắt ngồi trên xe buýt, bạn vẫn cảm nhận được xe đang rẽ trái hay tăng tốc – nhờ tai trong. IMU (Inertial Measurement Unit) là "tai trong" của xe.

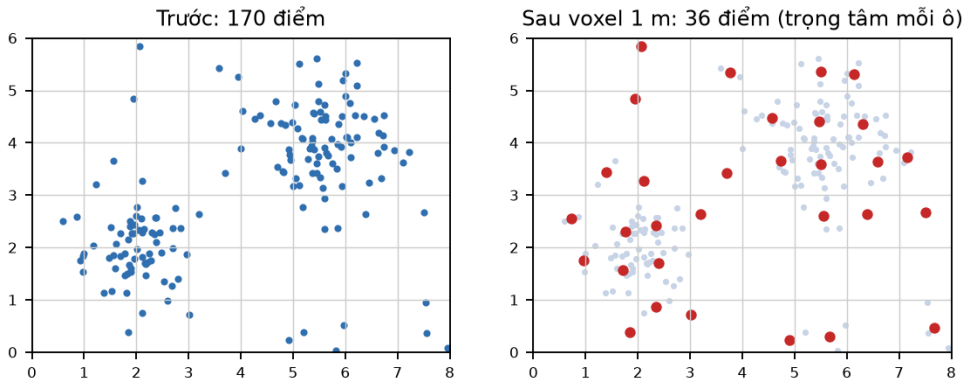
Topic `/imu/data` (100 Hz) cho: **vận tốc góc** (gyro, rad/s, quanh 3 trục) và **gia tốc** (m/s²). Tốc độ quay quanh trục z (gyro z) = tốc độ thay đổi yaw. **Cộng dồn (tích phân)** gyro z theo thời gian → góc đã quay. Vấn đề: gyro có **bias** (độ lệch không đổi) ≈ −0.25…−0.30 °/s ở cả hai track. Sau 100 s, bias này cộng dồn thành 25–30° sai → không dùng gyro một mình được. Lời giải: **bộ lọc bù** (mục 2.9).

2.4 Point cloud, file PLY / PCD, và giảm mẫu voxel

Point cloud (đám mây điểm) = danh sách rất nhiều điểm 3D (x, y, z), có thể kèm màu hoặc cường độ. Hai định dạng file:

- **PLY** (`.ply`): point cloud từ ảnh. `raw_ply` dạng chữ (ASCII), `3cameras_ply` dạng nhị phân.
- **PCD** (`.pcd`): định dạng của thư viện PCL, dùng cho bản đồ.

Bản đồ có ~60 triệu điểm – quá nhiều để tính nhanh. **Giảm mẫu voxel** (voxel grid downsampling): chia không gian thành các khối lập phương cạnh L (ví dụ 1 m hoặc 0.3 m), mỗi khối chỉ giữ lại một điểm là trọng tâm các điểm trong khối. Track01: 59 531 846 điểm → 496 909 điểm với L = 1 m.



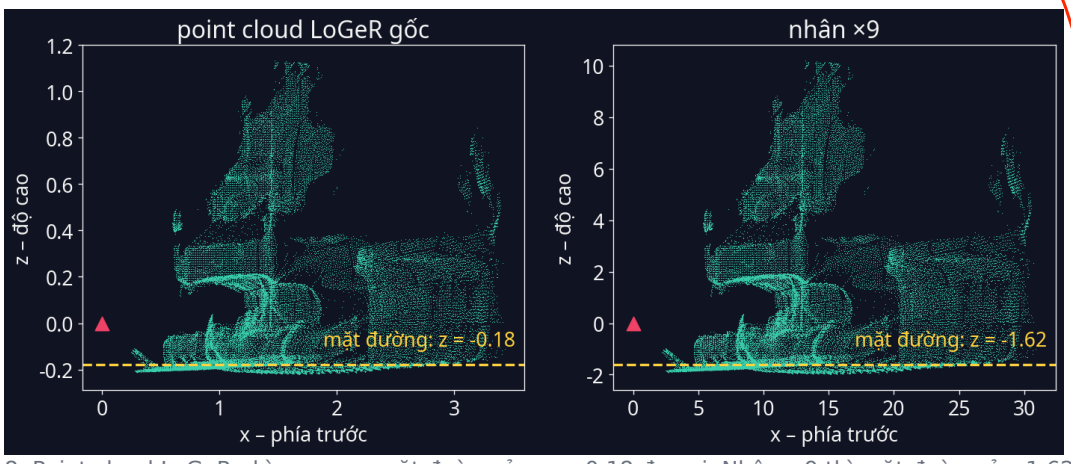
Hình 7. Giảm mẫu voxel (minh hoạ 2D): mỗi ô 1 m chỉ giữ một điểm đỏ.

2.5 Độ sâu từ ảnh và vấn đề tỉ lệ (scale)

> **Ví dụ đời thường**
> Nhìn một bức ảnh chụp một chiếc ô tô, bạn không thể biết đó là xe thật cách 10 m hay xe đồ chơi cách 1 m – ảnh chỉ cho **hình dạng**, không cho **kích thước tuyệt đối**. Mô hình AI đoán độ sâu từ ảnh cũng vậy.

Point cloud từ `raw_ply` (LoGeR) ở **đơn vị tuỳ ý**: frame 0 chỉ trải ra ~4 "đơn vị" trong khi camera thật nhìn xa hàng chục mét. Phải nhân với một **tỉ lệ** (scale) mới khớp với bản đồ (mét). Code mẫu của TA dùng `ply_scale = 3`. Repo đo tỉ lệ bằng **mặt đường**:

1. Trong point cloud, mặt đường là một mặt phẳng nằm dưới camera một khoảng h (đơn vị của point cloud). Đo h bằng cách lấy các điểm thấp, gần, phía trước và tìm độ cao xuất hiện nhiều nhất (histogram) – hàm `ground_height` trong `tools/ground_scale.py`.
2. Camera thật cao H mét so với mặt đường. Vậy tỉ lệ = H / h.
3. Đo trên 21 frame Track01: h ≈ 0.18 (dao động chỉ 5 %). Với H ≈ 1.5–1.8 m → tỉ lệ ≈ 8.4–10, **không phải 3**.

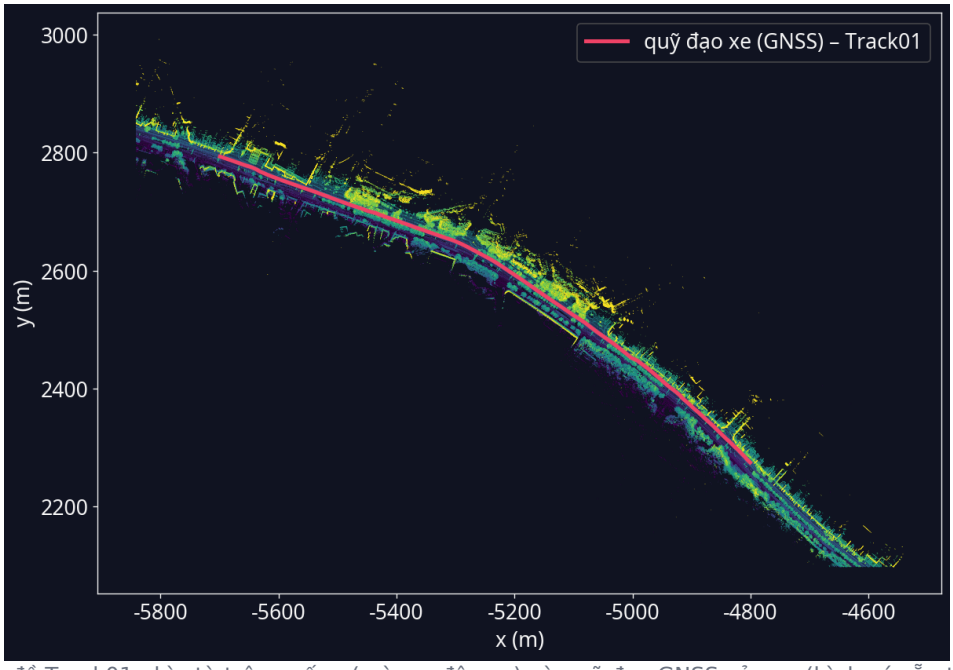


Hình 8. Point cloud LoGeR nhìn ngang: mặt đường ở z = −0.18 đơn vị. Nhân ×9 thì mặt đường ở −1.62 m – đúng độ cao một camera trên xe (hình có sẵn trong repo).

Với `3cameras_ply` (MapAnything), point cloud đã gần đúng mét: mặt đường ở h ≈ 2.06 m dưới camera, nhưng camera thật chỉ cao ~1.65 m → cần tỉ lệ ≈ 0.8. Pipeline cuối dùng **tỉ lệ riêng cho từng frame** = 1.646 / h(frame), với h đo lại từng frame rồi lọc trung vị qua 57 frame (~2 s). Hằng số 1.646 = 0.8 × 2.058 (2.058 = trung vị h của Track01).

> **Minh bạch: điểm cần biết**
> Hằng số 0.8 (và vì thế 1.646) được **chọn bằng điểm Kaggle**: tỉ lệ 1.0 làm điểm xấu đi, 0.8 làm tốt lên (`docs/nhat-ky/03 §8`). Đây thuộc loại "chỉnh tham số bình thường" mà TA cho phép, nhưng nên nói rõ. Lý do vật lý đi kèm: 0.8 ứng với camera cao ~1.65 m, khớp với chiều cao suy ra độc lập từ `raw_ply` ×9 (~1.62 m).

2.6 Bản đồ LiDAR



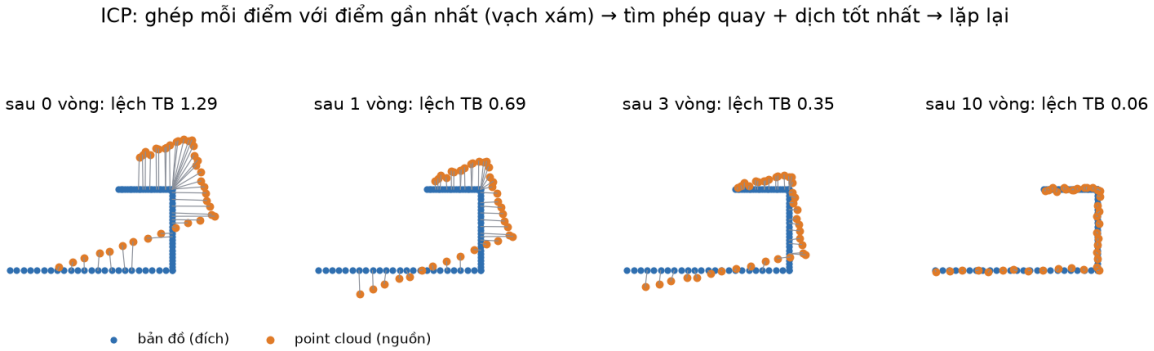
Hình 9. Bản đồ Track01 nhìn từ trên xuống (màu = độ cao) và quỹ đạo GNSS của xe (hình có sẵn trong repo).

2.7 Scan matching: ICP, point-to-plane, GICP, NDT

> **Ví dụ đời thường**
> Bạn có hai tờ giấy bóng kính, mỗi tờ vẽ cùng một ngôi nhà nhưng một tờ bị xê dịch và xoay lệch. Bạn đặt chồng lên nhau rồi xê dịch, xoay tờ trên cho tới khi các nét trùng khít. **Scan matching** (khớp quét) là làm đúng việc đó với point cloud (tờ trên) và bản đồ (tờ dưới). Vị trí, góc xoay cuối cùng của tờ trên chính là pose của xe.

2.7.1 ICP (Iterative Closest Point – lặp điểm gần nhất)

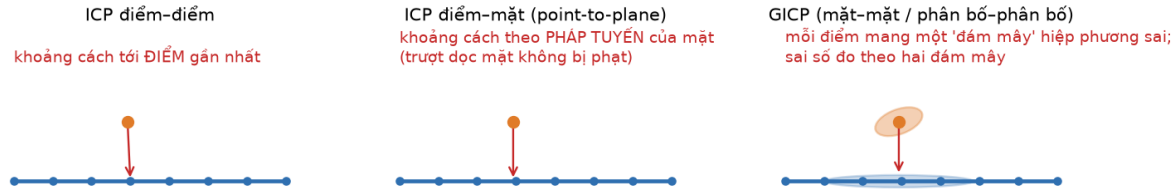
1. Với mỗi điểm của point cloud, tìm điểm gần nhất trên bản đồ (một "cặp").
2. Tìm phép quay + dịch làm tổng bình phương khoảng cách các cặp nhỏ nhất (có công thức đóng).
3. Áp dụng phép biến đổi đó, quay lại bước 1. Dừng khi gần như không còn dịch chuyển.



Hình 10. ICP thật (tính bằng numpy) trên ví dụ 2D: sai lệch giảm dần qua các vòng lặp.

ICP cần **điểm khởi đầu tốt**: nếu bắt đầu quá xa, "cặp điểm gần nhất" sai và ICP rơi vào một **cực tiểu địa phương** (khớp sai nhưng không tự thoát được). Vì vậy repo luôn bắt đầu từ pose GNSS.

2.7.2 Ba biến thể khác nhau ở cách đo "khoảng cách"

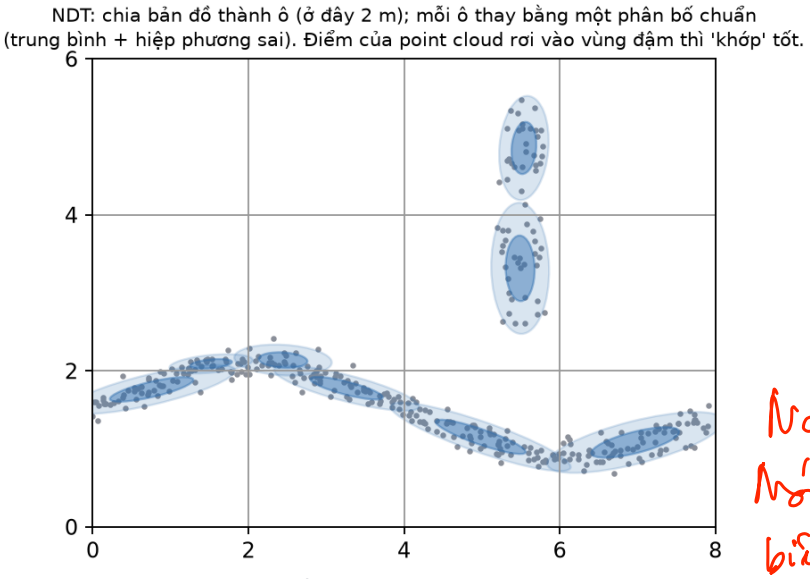


Hình 11. ICP điểm–điểm, điểm–mặt và GICP.

- **Point-to-plane ICP**: chỉ phạt khoảng cách theo pháp tuyến của mặt bản đồ (ví dụ vuông góc với bức tường). Trượt dọc theo tường không bị phạt → hội tụ nhanh hơn trên mặt phẳng. Repo dùng: `--method PLANE_ICP` (thư viện small_gicp) và một bản tự viết chỉ cho x, y, yaw: `--method PLANAR`.
- **GICP** (Generalized ICP, Segal và cộng sự 2009): mỗi điểm được coi là một phân bố chuẩn nhỏ (mô tả hình dạng bề mặt quanh nó, ước lượng từ 20 điểm lân cận). Đây là phương pháp chính của pipeline cuối (`--method GICP`).
- **VGICP**: GICP trên lưới voxel cho nhanh (đã thử, không dùng ở bản cuối).

2.7.3 NDT (Normal Distributions Transform)

Thay vì ghép từng cặp điểm, NDT chia bản đồ thành các ô (ví dụ 4 m hoặc 8 m), mỗi ô tóm tắt bằng một phân bố chuẩn (điểm trung bình + độ trải theo từng hướng). Sau đó tìm pose sao cho các điểm của point cloud rơi vào vùng "đậm" của các phân bố nhiều nhất. Code mẫu của TA dùng NDT của thư viện PCL. Tham số `ndt_resolution` = kích thước ô.



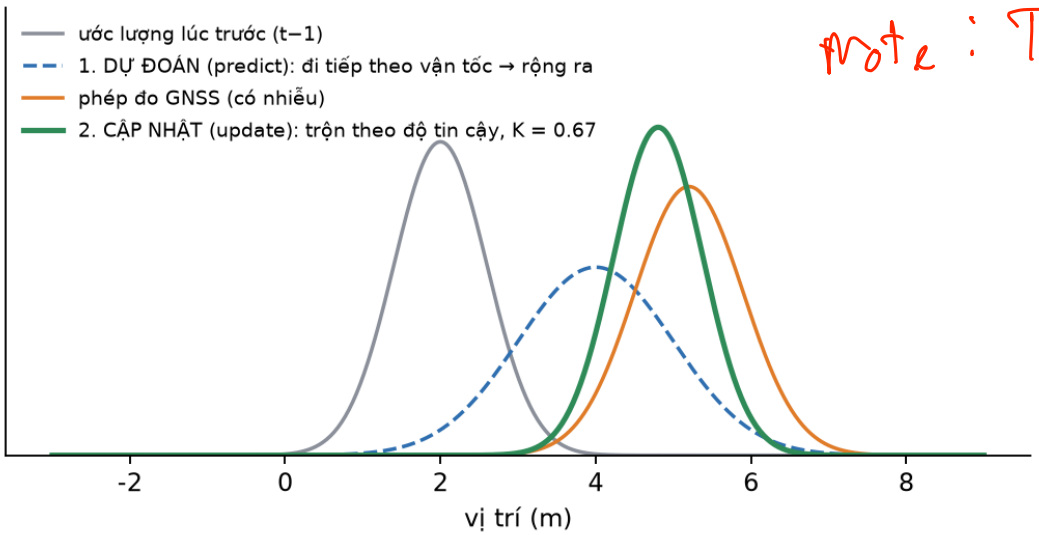
Hình 12. Ý tưởng NDT (minh hoạ 2D).

> **Ý chính**
> Trong repo: NDT (C++, PCL) ổn định sau khi sửa tỉ lệ nhưng kém GNSS; GICP (Python, small_gicp) nhanh hơn ~200 lần (0.06 s/frame so với ~14 s/frame) và chính xác hơn, nên pipeline cuối dùng GICP.

2.8 Lọc theo thời gian: bộ lọc Kalman, CTRV, RTS smoother, maximum likelihood

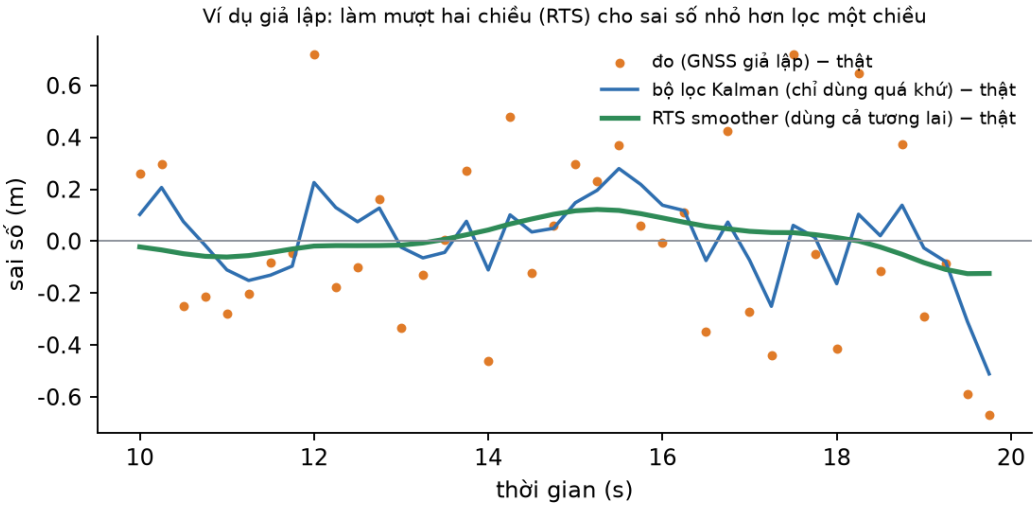
Ví dụ đời thường

Bạn đi bộ trong sương mù, biết mình đi ~1 m/giây về phía trước (dự đoán). Thỉnh thoảng nghe tiếng chuông nhà thờ cho biết đại khái mình ở đâu (đo, nhưng không chính xác). Cách tốt nhất là **trộn**: dùng dự đoán, rồi kéo nhẹ về phía phép đo – kéo nhiều hay ít tuỳ bạn tin dự đoán hay tin tiếng chuông hơn. Đó là bộ lọc Kalman.



Hình 13. Một bước Kalman: dự đoán làm độ không chắc chắn rộng ra; phép đo kéo ước lượng về phía nó; kết quả hẹp hơn cả hai.

- **Trạng thái** (state): các đại lượng cần ước lượng, kèm **hiệp phương sai** P (mức độ không chắc chắn).
- **Mô hình chuyển động CTRV** (Constant Turn Rate and Velocity): giữa hai lần đo, xe đi thẳng theo hướng mũi xe với tốc độ v, và quay đều với tốc độ quay ω; v và ω chỉ thay đổi chậm, ngẫu nhiên. Vì mô hình không tuyến tính (có cos, sin) nên dùng **Kalman mở rộng** (EKF): tuyến tính hoá bằng ma trận đạo hàm F.
- **Nhiễu quá trình** q (xe có thể tăng tốc/rẽ bất ngờ bao nhiêu) và **nhiễu đo** σ (GNSS sai bao nhiêu) quyết định "tin bên nào". Repo chọn các mức nhiễu bằng **maximum likelihood** (hợp lý cực đại): thử nhiều bộ giá trị, chọn bộ làm cho chính chuỗi GNSS quan sát được "có khả năng xảy ra" nhất – không cần đáp án, không cần Kaggle.
- **RTS smoother** (Rauch–Tung–Striebel): sau khi lọc xuôi thời gian, chạy ngược lại để mỗi thời điểm được dùng cả thông tin **tương lai**. Làm được vì bài này xử lý offline (đã có toàn bộ dữ liệu).



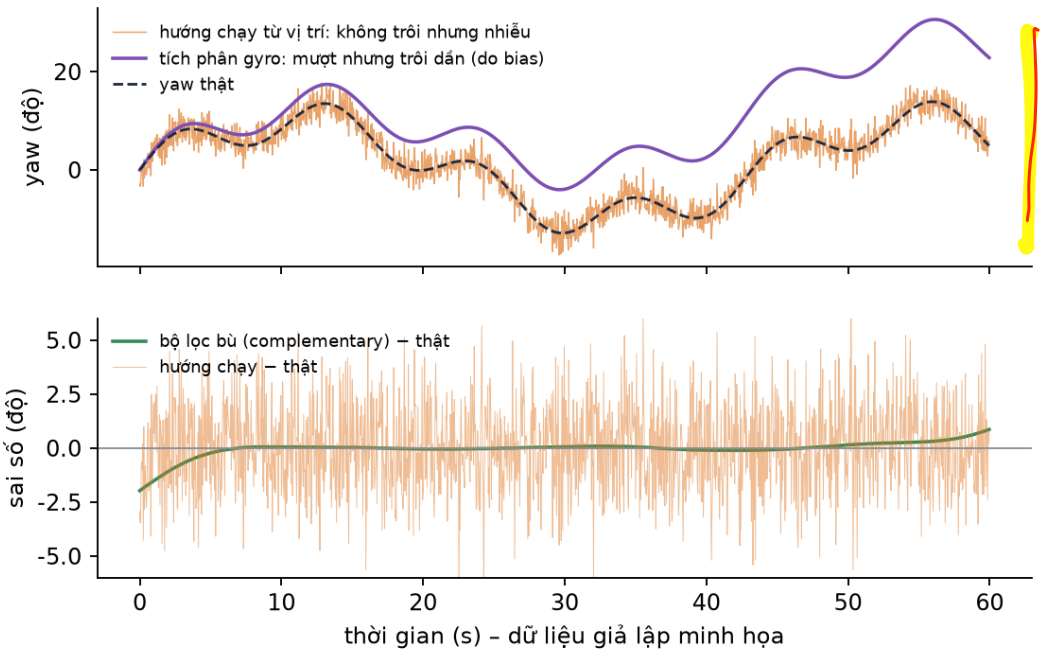
Hình 14. Ví dụ giả lập: lọc xuôi (xanh dương) bám đuôi phép đo; làm mượt hai chiều (xanh lá) sát thật hơn.

2.9 Bộ lọc bù (complementary filter) cho yaw

Ví dụ đời thường

Đồng hồ cơ chạy đều nhưng mỗi ngày nhanh vài giây (trôi). Đồng hồ trên đài phát thanh thì đúng nhưng chỉ báo mỗi giờ một lần và hay bị nhiễu. Cách tốt: **dùng đồng hồ cơ cho từng giây, rồi thỉnh thoảng chỉnh lại theo đài**.

Áp dụng cho yaw: **tích phân gyro** giống đồng hồ cơ (mượt, đúng các thay đổi nhanh, nhưng trôi do bias); **hướng chạy tính từ quỹ đạo** (hướng của vector vận tốc) giống đài (không trôi, nhưng nhiễu và lẫn trễ). Bộ lọc bù lấy phần **nhanh** từ gyro và phần **chậm** (trung bình dài hạn) từ hướng chạy. Ranh giới nhanh/chậm là tham số **τ** (tau, giây). Công thức và code: Chương 6.3, file `tools/yaw_fusion.py`.



Hình 15. Bộ lọc bù trên dữ liệu giả lập: sai số (xanh lá) nhỏ hơn hẳn hướng chạy thô (cam).

2.10 Nội suy, làm mượt Gauss, lọc trung vị

- **Nội suy** (interpolation): GNSS có 4 mẫu/giây, camera ~28 frame/giây → phải ước lượng giá trị ở giữa các mẫu. Nội suy tuyến tính (`np.interp`: nối thẳng) hoặc **spline bậc 3** (đường cong trơn).
- **Làm mượt Gauss** (`gaussian_filter1d`): thay mỗi giá trị bằng trung bình có trọng số của các giá trị lân cận, trọng số hình chuông với độ rộng σ. σ lớn → mượt hơn nhưng mất chi tiết.
- **Lọc trung vị** (`median_filter`): thay mỗi giá trị bằng trung vị của cửa sổ (ví dụ 5 giá trị). Loại bỏ những giá trị "nhảy" bất thường mà không kéo lệch như trung bình.

2.11 Phần mềm: ROS 2, rosbag, PCL, small_gicp, Docker

Công cụ	Là gì (giải thích đơn giản)	Dùng ở đâu trong repo
ROS 2 Humble	Bộ khung phần mềm cho robot: các chương trình ("node") trao đổi dữ liệu qua các kênh có tên ("topic")	node C++ `localization`, launch file
rosbag2 (.db3)	File ghi lại toàn bộ dữ liệu cảm biến trên các topic theo thời gian, để phát lại sau	đọc GNSS/IMU bằng C++ (`rosbag2_cpp`) và Python (`rosbags`)
RViz2	Chương trình xem 3D: bản đồ, point cloud, quỹ đạo	`config/*.rviz`, `launch/*.py`
PCL	Point Cloud Library (C++): đọc/ghi PLY/PCD, voxel, NDT	`localization.cpp`
small_gicp	Thư viện ICP/GICP/VGICP nhanh (Koide), có bản Python	`tools/gicp_refine.py`
numpy / scipy	Thư viện tính toán số của Python	hầu hết `tools/*.py`
Docker	"Hộp" chứa sẵn toàn bộ phần mềm để chạy giống hệt nhau trên mọi máy	`docker/`, `video/docker/`
Kaggle API	Cách nộp bài/đọc điểm bằng lệnh thay vì trình duyệt	bot nộp bài (ngoài repo)

Chương 3. Cấu trúc repo: từng thư mục, từng file

Repo có 286 file (không tính thư mục `.git`), 47 MB. Phần lớn dung lượng là CSV kết quả và một file nhạc 11 MB của bộ làm video. **Dữ liệu cuộc thi không nằm trong repo** – nó ở `~/SDC-midterm01/data/` trên máy (trong Docker: `/root/data/`).

Cây thư mục (rút gọn; danh sách đầy đủ ở các bảng bên dưới)

```
SDC-midterm-1-/                (trên máy: ~/SDC-midterm01/midterm01)
├── README.md  CLAUDE.md  .gitignore  .vscode/settings.json
├── run_baseline.sh  run_fusion.sh  run_chunk.sh  run_image_only.sh
│                                  script chạy node C++ (trong Docker ROS)
├── docker/
│   ├── Dockerfile  docker-compose.yml  ros_entrypoint.sh   → image sdc_ros2_dev
│   └── reg/Dockerfile                                      → image sdc-reg
├── src/localization/              gói ROS 2 (template TA, đã mở rộng)
│   ├── CMakeLists.txt  package.xml
│   ├── include/localization/localization.h
│   ├── src/        localization.cpp  map_modified.cpp  visualization.cpp
│   ├── launch/     localization / map_modified / visualization .launch.py
│   ├── config/     localization.rviz  visualization.rviz
│   ├── processing_codes/  txt_to_idx_x_y_yaw.py  gnss_to_csv.py
│   │                      gnss_smooth_to_csv.py  fuse_to_csv.py
│   └── result/reg/clean/  4 file clean_*.csv + if/ (14 file)
│                          (result/ bị .gitignore; các file này được ép thêm)
├── tools/                         30 công cụ (28 Python + 2 shell)
├── docs/
│   ├── HUONG-DAN-CHAY.md          hướng dẫn chạy
│   ├── nhat-ky/                   nhật ký 01-09 + README
│   │   ├── data/                  số liệu phân tích, lịch sử nộp, debug NDT
│   │   ├── fig/submission_progress.png
│   │   └── submissions/           track1/ (61 CSV)  track2/ (50 CSV)
│   └── report/                    make_report.py  bao_cao_tong_hop.pdf  fig/
└── video/                         bộ dựng video bài giảng (Manim + edge-tts)
    ├── build.py  lib/  scenes/ (20 chương)  docker/
    └── assets/  script.md  research_notes.md  README.md
```

3.1 Thư mục gốc

File	Vai trò
`README.md`	Giới thiệu ngắn (tiếng Anh): cách bố trí repo, cách chạy NDT có prior, bảng tham số mới của node C++. **Lỗi thời một phần:** vẫn ghi "Best so far 1.25338 / 0.40778" (điểm ngày 17/10).
`CLAUDE.md`	"Trạng thái dự án – đọc trước" (cập nhật tối 21/10): luật bắt buộc (không dò bảng điểm, không làm Track03, ~3 file/lần nộp, luôn commit + push), kết quả hiện tại, pipeline, hàng đợi nộp ngày 22/10, việc tiếp theo, môi trường, bẫy. Viết cho phiên AI hoặc người tiếp nhận dự án.
`.gitignore`	Bỏ qua `build*/ install*/ log*/ __pycache__/ src/localization/result/`.
`.vscode/settings.json`	Đường dẫn CMake cho VS Code: `/home/ee904/SDC-midterm01/midterm01/src` – **đường dẫn máy của TA** còn sót lại từ template, không dùng.
`run_baseline.sh`	Chạy node C++ với cấu hình mặc định của TA (không RViz) rồi đổi kết quả sang CSV Kaggle.
`run_fusion.sh`	Chạy NDT có prior GNSS cho một track; ghi `<TAG>_TrackN_result.txt` và `<TAG>_TrackN_debug.csv`.
`run_chunk.sh`	Như trên nhưng chỉ một đoạn frame (`start_frame`, `max_frames`) để chạy song song nhiều lõi.
`run_image_only.sh`	NDT chỉ dùng ảnh (không GNSS), tỉ lệ ×9 – thí nghiệm "scan matching thuần" cho báo cáo.

3.2 docker/

File	Vai trò
`Dockerfile`	Image `sdc_ros2_dev`: `osrf/ros:humble-desktop` + PCL, rosbag2, RViz2, OpenCV, numpy 1.23.5, pandas < 2.1, rosbags. Bản TA có lỗi `pandas<2.1` không có dấu nháy (shell hiểu `<` là chuyển hướng file) – đã sửa thành `"pandas<2.1"`.
`docker-compose.yml`	Mở container có màn hình (X11) để chạy RViz; gắn repo vào `/root/midterm`, dữ liệu vào `/root/data`. Đã điền đường dẫn `/home/thuhang/...` (cứng, máy khác phải sửa) và tắt khối GPU (máy chưa cài nvidia-container-toolkit; NDT chạy CPU).
`ros_entrypoint.sh`	Khi container khởi động: nạp môi trường ROS 2 và workspace (nếu đã build).
`reg/Dockerfile`	Image `sdc-reg`: Python 3.11 + small_gicp 1.0.0, numpy 1.26.4, scipy 1.13.1, rosbags 0.10.4. Dùng cho mọi `tools/*.py` (máy host theo ghi chú không có numpy).

> **Ghi chú**
> Theo `docs/HUONG-DAN-CHAY.md`, `docker-compose.yml` được **chép** ra `~/SDC-midterm01/` (cạnh thư mục repo) rồi mới chạy `docker compose` ở đó; bản trong repo là bản sao để lưu.

3.3 src/localization/ – gói ROS 2 (code C++ của TA, đã mở rộng)

File	Vai trò
`CMakeLists.txt`	Cách build: 3 chương trình `localization`, `map_modified`, `visualization`; liên kết ROS 2, PCL, Eigen, Boost. Không đổi so với TA.
`package.xml`	Khai báo gói ROS 2 và thư viện phụ thuộc (maintainer `ee904@todo.todo`, license TODO – để nguyên của TA).
`include/localization/localization.h`	Khai báo lớp `LocalizationApp`, các struct `PlyFrameInfo`, `PriorPose` (mới), `DB3Frame` và giá trị mặc định của mọi tham số.
`src/localization.cpp`	**Node định vị chính** (825 dòng): đọc bản đồ, liệt kê PLY, đọc GNSS/IMU, chạy NDT từng frame, ghi kết quả, gửi lên RViz. Repo thêm: prior, loại kết quả lệch xa, stride, start_frame, giới hạn tầm, file debug (+132 dòng). Chi tiết: Chương 5.1.
`src/map_modified.cpp`	Chương trình phụ của TA: giảm mẫu `map.pcd` bằng voxel và lưu bản nhị phân. Không sửa.
`src/visualization.cpp`	Chương trình phụ của TA: phát lại bản đồ + quỹ đạo từ file kết quả để xem. **Không được repo dùng**; tên biến "radar" cho thấy code này lấy từ một bài tập cũ (dữ liệu radar) – xem Chương 5.1.4.
`launch/localization.launch.py`	Chạy node định vị + RViz, có các tham số tuỳ chọn. Không có các tham số mới (prior_file…).
`launch/map_modified.launch.py`	Chạy `map_modified`.
`launch/visualization.launch.py`	Chạy `visualization` + RViz.
`config/localization.rviz`	Bố cục RViz 2: bản đồ `/map_pc`, submap, point cloud, pose, quỹ đạo.
`config/visualization.rviz`	Bố cục cho visualization – dùng tên lớp kiểu ROS 1 (`rviz/PointCloud2`); có thể không mở được trong RViz 2 (chưa kiểm chứng).
`processing_codes/txt_to_idx_x_y_yaw.py`	Script TA: đổi `*_result.txt` (12 số/dòng) thành CSV Kaggle.
`processing_codes/gnss_to_csv.py`	Đọc GNSS từ rosbag, nội suy tuyến tính tới thời điểm từng frame → CSV. Viết thêm.
`processing_codes/gnss_smooth_to_csv.py`	Như trên + làm mượt Gauss + spline bậc 3; yaw = yaw GNSS hoặc hướng chạy.
`processing_codes/fuse_to_csv.py`	**Ghép** độ sửa của scan matching (từ file debug) vào prior: lọc, trừ trung bình, làm mượt, nội suy. Dùng trong pipeline cuối.
`result/reg/clean/*.csv`	4 file kết quả sạch: `clean_cfz1_tau7`, `clean_cfz2_tau15` (bài chấm dự kiến), `clean_p2dm_pos_1` (vị trí Track01 sau GICP lượt 2, trước bước yaw), `clean_prior_1` (prior Track01).
`result/reg/clean/if/*.csv`	14 file thử bộ ghép mới `info_fuse.py`; tên mã hoá tham số: `if_g{γ}_s{σ_match}_q{q}_{track}[_cfzτ]`, ví dụ `if_g1_s0.5_q0.02_1_cfz7` = γ 1, σ 0.5 m, q 0.02, Track01, đã thay yaw bằng gyro τ 7 s.

> **Không chắc chắn**
> Tài liệu đoán các file `if_*_cfz7/cfz15.csv` chính là `if_aniso_*` / `if_iso_*` trong `docs/nhat-ky/submissions/` (γ 1 / γ 0 với σ 0.5, q 0.02). Repo không ghi rõ ánh xạ này. Cũng lưu ý: `if_iso_2`/`if_aniso_2` có 130291 byte, khác 1 byte so với các file Track02 khác – có thể do làm tròn số; không ảnh hưởng định dạng.

3.4 tools/ – 30 công cụ

Nhóm theo vai trò. Cột "Trong bài chấm?" cho biết file có nằm trên đường tạo ra `clean_cfz1_tau7` / `clean_cfz2_tau15` hay không.

File	Làm gì	Trong bài chấm?
`rts_smoother.py`	Kalman CTRV + RTS smoother trên GNSS, nhiễu chọn bằng maximum likelihood	**Có** (bước ①)
`csv_ops.py`	Ghép vị trí của file A với yaw của file B; dời ngang / cộng yaw	**Có** (tạo phys_yawb_1)
`yaw_fusion.py`	Yaw = bộ lọc bù gyro + hướng chạy	**Có** (②, ⑤)
`gicp_refine.py`	Khớp point cloud vào bản đồ bằng small_gicp (ICP/GICP/VGICP/PLANE_ICP) hoặc ICP 3 bậc tự do tự viết	**Có** (③)
`ground_scale.py`	Đo độ cao mặt đường trong point cloud → tỉ lệ	**Có** (hàm dùng trong ③)
`map_voxel.py`	Giảm mẫu map.pcd thành bản đồ 0.3 m (đọc theo khối, ít RAM)	**Có** (chuẩn bị)
`run_clean_pipeline.sh`	Chạy toàn bộ pipeline sạch bằng một lệnh	**Có**
`rts_imu_smoother.py`	Kalman có thêm gyro và mô hình yaw GNSS bị trễ	Không (thí nghiệm)
`info_fuse.py`	Ghép bằng bình phương tối thiểu toàn quỹ đạo, trọng số theo Hessian GICP	Không (ứng viên 22/10)
`start_yaw_fix.py`	Dựng lại yaw lúc xe đứng yên bằng gyro	Không
`curvature_yaw.py`	Hiệu chỉnh yaw theo độ cong (điểm đo trước cầu sau)	Không
`generate_result.py`	Tạo một CSV từ cấu hình JSON (GNSS → NDT → ghép)	Không (tự động hoá cũ)
`merge_debug.py`	Gộp các file debug NDT chạy theo đoạn	Gián tiếp (tạo c3s08/c3s10T2)
`pc_io.py`	Đọc PLY/PCD bằng numpy	Gián tiếp
`analyze_gnss_imu.py`	Thống kê GNSS / IMU / init_guess	Phân tích
`analyze_yaw_lag.py`	Đo độ trễ của yaw GNSS	Phân tích
`check_frame_alignment.py`	Thử nhiều tỉ lệ/góc để kiểm tra hệ trục point cloud	Phân tích
`time_offset.py`	Đo lệch thời gian GNSS – IMU (kết luận: không đo được)	Phân tích
`map_fit.py`	Thước đo offline "độ khớp bản đồ" (không cần đáp án)	Phân tích
`three_hat.py`	Thước đo "three-cornered hat" (kết luận: không dùng được)	Phân tích
`segment_analysis.py`	Hessian GICP có dự báo nhiễu không; bảng theo đoạn đường	Phân tích
`track_error.py`	So một quỹ đạo với quỹ đạo tham chiếu, tính "điểm"	Phân tích
`compare_csv.py`	Chênh lệch vị trí/yaw giữa các file nộp	Phân tích
`submission_stats.py`	Bảng + biểu đồ mọi lần nộp, phân loại pipeline/lai/dò → doc 09	Phân tích
`reproduce_submissions.sh`	Tạo lại 34 file nộp đầu tiên và so từng byte	Kiểm tra (lỗi thời, xem 5.4.10)
`analyze_probes.py`	Tách điểm thành phần vị trí / yaw từ các file thăm dò	**Dò bảng điểm** (chỉ phân tích)
`yaw_projection.py`	Tổ hợp tối ưu các hướng thay đổi yaw theo điểm Kaggle	**Dò bảng điểm**
`pos_projection.py`	Tổ hợp tối ưu các hướng thay đổi vị trí theo điểm Kaggle	**Dò bảng điểm**
`make_probes.py`	Sinh file dò dạng dao động cosin	**Dò bảng điểm**
`bump_yaw.py`	Sinh file dò yaw theo cửa sổ thời gian	**Dò bảng điểm**

3.5 docs/

File / thư mục	Nội dung
HUONG-DAN-CHAY.md	Hướng dẫn chạy từng bước, từ build Docker tới nộp bài (Chương 8 dựa trên file này).
nhat-ky/README.md	Mục lục nhật ký, tóm tắt kết quả (một số số liệu đã lỗi thời, ví dụ "18 file Track01").
nhat-ky/01-chuan-bi.md	Thông tin cuộc thi, tải dữ liệu (API Kaggle bị giới hạn → tải zip qua trình duyệt), sửa Docker.
nhat-ky/02-phan-tich-du-lieu.md	Phân tích GNSS, init_guess, point cloud (hệ trục, tỉ lệ), bản đồ, IMU, độ trễ yaw GNSS.
nhat-ky/03-thi-nghiem-ndt.md	Chạy baseline (trôi ~100 m), sửa C++, thí nghiệm NDT ×2…×10, NDT chỉ dùng ảnh, NDT 3 camera.
nhat-ky/04-cac-lan-nop-kaggle.md	Từng lần nộp #1–#35 (17–18/10): cách tạo, khác gì, vì sao, điểm, kết luận.
nhat-ky/05-phan-tich-loi.md	Tách điểm thành lỗi vị trí/yaw bằng file thăm dò; tổ hợp theo điểm (giờ là phân tích).
nhat-ky/06-nhan-xet.md	22 nhận xét, bảng hiệu quả/không hiệu quả, sai lầm và bài học, rủi ro.
nhat-ky/07-tai-tao.md	Lệnh tạo lại từng file nộp đầu tiên (34/34 trùng byte ngày 18/10).
nhat-ky/08-gicp-va-scale.md	GICP, tỉ lệ theo mặt đường, demean, yaw gyro, dò dao động, trả lời TA, cải tiến sau 21/10.
nhat-ky/09-thong-ke-nop-bai.md	Tự sinh: thống kê 152 lần nộp, phân loại pipeline / lai / dò.
nhat-ky/data/	Kết quả phân tích dạng chữ (*.txt), lịch sử nộp (kaggle_submissions*.csv/json, precise_scores.json, submission_stats.csv), ndt_debug/ (12 file debug NDT – đủ để tạo lại file nộp mà không chạy NDT hàng giờ), image_only_test/, baseline_partial/ (kết quả baseline TA chạy dở, bằng chứng trôi).
nhat-ky/submissions/	111 CSV: bản chính xác của các file đã nộp (hoặc định nộp). Danh sách đầy đủ: Phụ lục B.
nhat-ky/fig/submission_progress.png	Biểu đồ điểm tốt nhất theo thời gian, theo loại.
report/make_report.py	Script reportlab tạo báo cáo tổng hợp PDF tiếng Việt cũ (19/10) + 4 hình trong report/fig/.
report/bao_cao_tong_hop.pdf	Báo cáo đó (trước khi TA trả lời → một phần nội dung đã lỗi thời).

3.6 video/ – bộ dựng video bài giảng

Một dự án phụ (18/10): tự động tạo video tiếng Việt ~38 phút giải thích bài, dùng **Manim** (thư viện vẽ animation toán), **edge-tts** (đọc chữ thành giọng nói, giọng vi-VN-HoaiMyNeural), nhạc và hiệu ứng tự tổng hợp bằng numpy. Không liên quan tới kết quả Kaggle.

File	Vai trò
build.py	Một lệnh: đọc lời → render Manim → đặt giọng đúng nhịp → thêm SFX → trộn nhạc (giảm nhạc khi có giọng) → phụ đề → ghép 20 chương.
lib/kit.py	Khai báo chương dạng danh sách "nhịp" (câu đọc + thứ hiện lên: tiêu đề, gạch đầu dòng, ảnh, công thức…).
lib/style.py	Màu, kiểu chữ, thẻ tiêu đề, thời lượng nhịp.
lib/tts.py	Đọc từng câu thành file wav, đo thời lượng; bảng PRONOUNCE viết lại thuật ngữ Anh theo cách đọc Việt ("Kaggle" → "ke gồ"); tự thử lại khi dịch vụ lỗi.
scenes/*.py (20 file)	Mỗi chương một file: c0 mở đầu, c1 bức tranh lớn, c2 vì sao, c3_01…c3_11 kiến thức nền (hệ toạ độ, pose, point cloud, scale, tiền xử lý, ICP – tính ICP thật bằng numpy, NDT, Kalman, ROS, thước đo, Kaggle), c4 code của đề, c5a/c5b đã làm gì, c6 hướng tiếp, c7 checklist, c8 thuật ngữ.
assets/make_figures.py	Vẽ hình từ dữ liệu thật (bản đồ, point cloud, baseline vs GNSS, yaw bậc thang, NDT) → assets/figures/.

File	Vai trò
`assets/make_audio.py`	Tổng hợp nhạc nền 120 BPM và 3 SFX (whoosh, pop, ding) – không dùng nhạc bên ngoài.
`assets/figures/*` (9 file)	Hình dùng cho video; tài liệu này dùng lại 5 hình trong số đó.
`assets/music, sfx, CREDITS.md`	File âm thanh tự tổng hợp và ghi nguồn/giấy phép (CC0).
`script.md, research_notes.md, README.md`	Kịch bản đầy đủ, dữ kiện thu thập (18/10), hướng dẫn build.
`docker/Dockerfile, requirements.txt, .gitignore`	Môi trường render (Python 3.11, Manim 0.18.1, LaTeX, ffmpeg).

Minh bạch: điểm cần biết

Video được làm ngày 18/10, **trước** GICP, yaw gyro và trả lời của TA. Nó nói về điểm 1.79 → 1.05 và có đoạn về "chọn theo điểm Kaggle" – nội dung đã lỗi thời so với hiện trạng. Không nên dùng video làm tài liệu thuyết trình mà không cập nhật.

Chương 4. Luồng chạy tổng thể (pipeline): từ dữ liệu thô tới file nộp

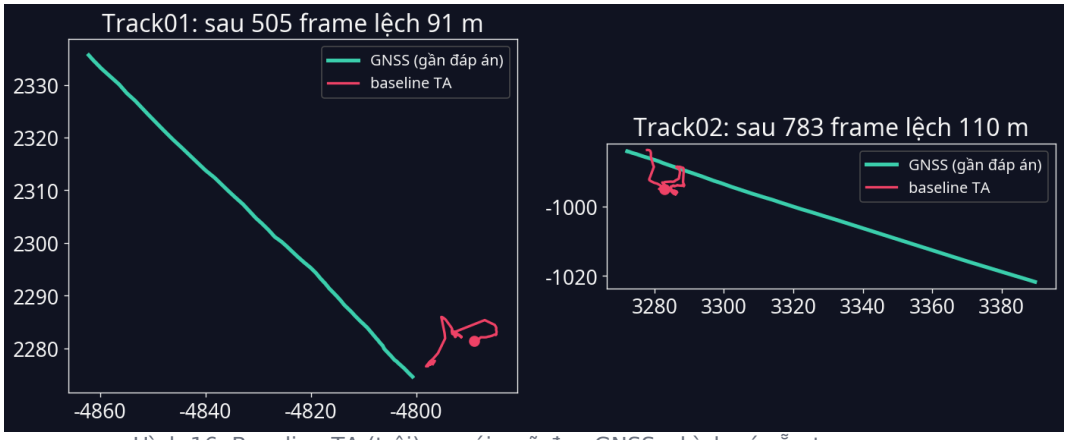
Có hai "thế hệ" pipeline trong repo. Hiểu cả hai giúp hiểu vì sao code có nhiều thứ không còn dùng.

4.1 Pipeline gốc của TA (baseline)

Baseline: mỗi frame bắt đầu từ pose của frame trước, không dùng GNSS/IMU

```
raw_ply/*.ply ──► nhân tỉ lệ ×3 ──► lọc NaN + voxel 0.3 m ─┐
map.pcd ──► voxel 1 m (lưu cache map_downsampled_1m.pcd) ──► cắt submap ±100 m quanh pose cũ ─┤
init_guess.txt (frame 0) / pose frame trước (frame sau) ──────────────────────────────────────► NDT
     (ô 8 m)
   ──► pose 4×4 ──► Track1_result.txt (12 số/dòng) ──► txt_to_idx_x_y_yaw.py ──► CSV Kaggle
```

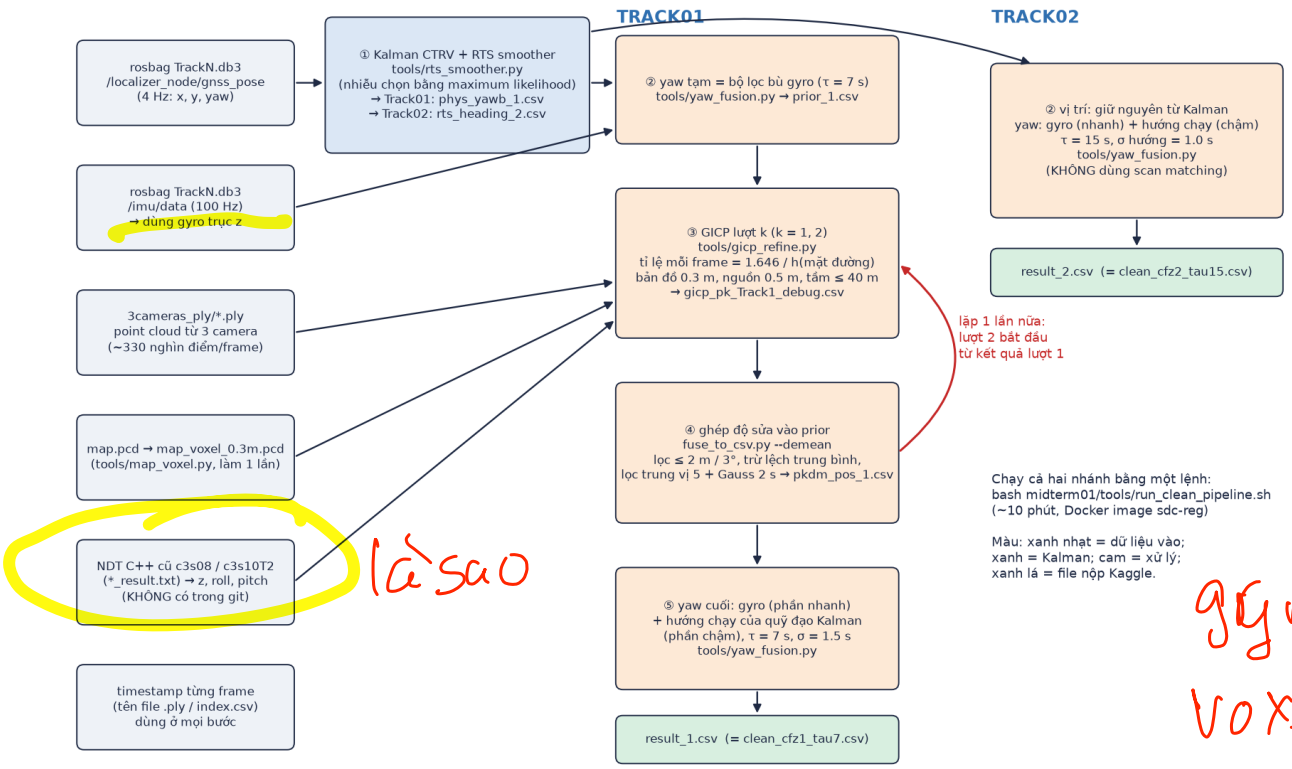
Vấn đề: tỉ lệ ×3 sai (đúng ≈ ×9) nên point cloud nhỏ hơn thật 3 lần; NDT khớp sai ngay từ frame 17 và kéo sai luôn các frame sau. Chạy trên máy: Track01 lệch 91 m sau 504 frame, Track02 lệch 110 m sau 782 frame (`docs/nhat-ky/data/baseline_partial/`).



Hình 16. Baseline TA (trôi) so với quỹ đạo GNSS – hình có sẵn trong repo.

4.2 Pipeline cuối – "sạch" (dùng làm bài chấm)

Note : base line thấp thì sẽ trông như nào



Hình 17. Sơ đồ pipeline sạch (tools/run_clean_pipeline.sh). Số trong vòng tròn là thứ tự bước.

Bước	Track	Đầu vào	Làm gì	Đầu ra	File code
0a	1, 2	map.pcd	Giảm mẫu voxel 0.3 m (làm một lần)	map_voxel_0.3m.pcd	`map_voxel.py`
0b	1, 2	3cameras_ply, GNSS	NDT C++ chạy rất lâu (~14 s/frame, song song 10 lõi) – chỉ để lấy z, roll, pitch	`c3s08_*_result.txt`, `c3s10T2_*`	`localization.cpp`, `run_chunk.sh`
①	1, 2	GNSS	Kalman CTRV + RTS; Track01: nhiễu cố định q_a=1, q_w=0.1, yaw GNSS làm mượt + bỏ lệch 0.42°; Track02: nhiễu tự chọn, yaw = hướng chạy	`phys_yawb_1.csv`, `rts_heading_2.csv`	`rts_smoother.py`, `gnss_smooth_to_csv.py`, `csv_ops.py`
②	1	①, IMU	Thay yaw bằng bộ lọc bù gyro τ 7 s (để GICP bắt đầu từ hướng tốt hơn)	`prior_1.csv`	`yaw_fusion.py`
③	1	②, 3cameras_ply, bản đồ 0.3 m, 0b	GICP mọi frame, tỉ lệ 1.646/h	`gicp_p1_Track1_debug.csv`	`gicp_refine.py`
④	1	②, ③	Ghép: lọc ≤2 m/3°, trừ lệch TB, trung vị 5, Gauss 2 s; không đổi yaw	`p1dm_pos_1.csv`	`fuse_to_csv.py`
③④	1	kết quả ④	Lặp lại GICP + ghép một lần (lượt 2)	`p2dm_pos_1.csv`	như trên
⑤	1	lượt 2, ①, IMU	Yaw cuối: gyro + hướng chạy từ vị trí Kalman (không phải vị trí sau GICP)	`result_1.csv`	`yaw_fusion.py`
②'	2	①, IMU	Giữ vị trí Kalman, yaw = gyro + hướng chạy, τ 15 s	`result_2.csv`	`yaw_fusion.py`

Ý chính

Vị trí Track01 = GNSS (đã lọc Kalman) + độ sửa từ khớp point cloud vào bản đồ LiDAR. **Vị trí** Track02 = GNSS (đã lọc Kalman), vì mọi phép khớp đều làm Track02 xấu đi.

Yaw cả hai track = gyro (phần thay đổi nhanh) + hướng chạy của quỹ đạo Kalman (phần chậm).

tại sao Track1 và track 2 khác nhau

Minh bạch: điểm cần biết

Pipeline sạch không chạy lại được chỉ từ repo. Nó cần ba thứ không có trong git: (1) dữ liệu cuộc thi; (2) `phys_yawb_1.csv` / `rts_heading_2.csv` trong `src/localization/result/` (thư mục bị .gitignore – nhưng có bản sao trong `docs/nhat-ky/submissions/`); (3) kết quả các lượt NDT C++ `c3s08` / `c3s10T2` (`*_result.txt`, gồm cả `result/chunks/`) – **không có bản sao nào trong repo**, phải chạy lại NDT nhiều giờ nếu mất.

Một nguồn "không sạch" gián tiếp: lượt NDT `c3s08` (Track01) được chạy với prior là `yawproj_v1_1.csv` – một tổ hợp yaw chọn theo điểm Kaggle (`docs/nhat-ky/03 §7`). Pipeline sạch chỉ lấy **z, roll, pitch** từ lượt đó (x, y, yaw lấy từ prior sạch), nên ảnh hưởng có lẽ rất nhỏ, nhưng repo **chưa kiểm chứng** và cũng không nhắc tới điểm này. Tương tự `c3s10T2` (Track02) chạy với prior `startfix_on_v3_2.csv` – nhưng Track02 sạch không dùng GICP nên không bị ảnh hưởng. Nếu muốn hoàn toàn chắc, có thể chạy lại NDT với prior sạch, hoặc lấy z/roll/pitch từ cách khác (ví dụ độ cao bản đồ dưới xe).

Điểm của hai file sạch **chưa được đo**: chúng đang được giữ lại (`~/kaggle-sdc-bot/held/`) / xếp hàng nộp 22/10. So với bản cũ có điểm (0.78914 / 0.27654), vị trí chỉ lệch 0.101 m / 0.054 m RMS và yaw gần như trùng.

4.3 Những dữ liệu trung gian quan trọng

4.3.1 File CSV Kaggle `idx,x,y,yaw`

Mọi bước đều đọc/ghi định dạng này, nên có thể ghép các công cụ với nhau như ống nước: đầu ra của bước này là "prior" (ước lượng trước) của bước sau.

4.3.2 File debug của scan matching

Node C++ và `gicp_refine.py` ghi cùng một định dạng, nhờ vậy `fuse_to_csv.py` xử lý được cả NDT lẫn GICP:

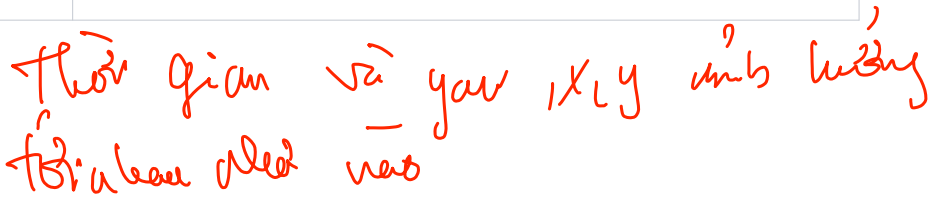
Dòng đầu của docs/nhat-ky/data/ndt_debug/c3s08_Track1_debug.csv

```
idx,timestamp_ns,converged,accepted,score,input_points,prior_x,prior_y,prior_yaw,guess_x,guess_y,guess_z
    ,guess_yaw,
ndt_x,ndt_y,ndt_z,ndt_yaw     (+ với GICP: init_*, error, inliers, inlier_ratio, scale, ground_h, info_x
    x, info_xy, info_yy)
0,1727234445100462219,1,1,1.130418651,8044,-4800.685998,2274.557006,2.327301,-4800.686035,2274.556885,10
    9.928093,
  2.327300787,-4801.308105,2274.152344,110.3203506,2.325661898
```

- `guess_*`: pose bắt đầu (từ prior); `ndt_*`: pose sau khi khớp (tên giữ là "ndt" kể cả với GICP).
- **Độ sửa** = `ndt − guess`. Ví dụ dòng trên: x sửa −0.62 m, y sửa −0.40 m.
- `converged`: thuật toán có hội tụ không; `accepted`: có nằm trong ngưỡng cho phép quanh prior không.
- `info_xx/xy/yy` (chỉ GICP): "độ chắc" của phép khớp theo từng hướng x, y (Chương 6.4).

4.3.3 Đặt tên file nộp

Mẫu tên	Ý nghĩa
`gnss_*, gnss_smooth_*, gnss_heading_*`	GNSS nội suy / làm mượt / yaw = hướng chạy
`rts_*, phys_yawb_1`	Kalman CTRV + RTS; `phys` = nhiễu quá trình cố định theo vật lý, `yawb` = bỏ lệch yaw b
`*_dt±0.1_*`	Thăm dò: dịch thời gian ±0.1 s
`ndt3c_s08/s10`	NDT trên 3cameras_ply, tỉ lệ 0.8 / 1.0
`gicp_ground_*, pass2, dm`	GICP, tỉ lệ theo mặt đường, lượt 2, có demean
`cfg*/cfz*_tauN[_al]`	Yaw bộ lọc bù: `g` = gyro chiếu theo trọng lực, `z` = gyro trục z; τ = N giây; `_al` = có hiệu chỉnh trượt (auto-lever)
`clean_*`	Dựng lại hoàn toàn bằng pipeline (bài chấm)
`if_iso/if_aniso`	Bộ ghép `info_fuse.py` đẳng hướng / theo Hessian
`probe_*, bump*, yawproj_*, posproj_*, lb_posyaw_*, corr_r*, *_bestyaw_*`	**Dò / tổ hợp theo bảng điểm** – không dùng chấm



Chương 5. Giải thích chi tiết từng file code

Đọc code: mỗi mục có (a) file làm gì, (b) các hàm/lớp chính, (c) trích đoạn quan trọng kèm giải thích dòng, (d) chỗ tạm/hack/lỗi nếu có. Trích đoạn được rút gọn bớt dòng log; số dòng ghi theo file gốc.

5.1 Node C++ `localization` (src/localization/)

Đây là template của TA viết bằng C++ cho ROS 2. Một chương trình ("node") tên `image_point_cloud_localizer`. Mọi thứ nằm trong lớp `LocalizationApp`. Repo thêm khả năng **bắt đầu mỗi frame từ một prior** (ví dụ GNSS) và **từ chối** kết quả NDT lệch quá xa prior.

5.1.1 localization.h – khai báo

Thành phần	Ý nghĩa
`struct PlyFrameInfo`	Một frame: thời điểm (ns) + đường dẫn file .ply.
`struct PriorPose` (mới)	x, y, yaw của prior cho một frame.
`struct DB3Frame`	Message GNSS và IMU mới nhất tính tới thời điểm frame (giữ lại qua các frame).
Tham số mặc định	`ply_scale_ = 3.0`, `ndt_resolution_ = 8.0`, `ndt_step_size_ = 0.8`, `ndt_transformation_epsilon_ = 0.05`, `ndt_max_iterations_ = 30`, `map_voxel_leaf_size_ = 1.0`, `input_voxel_leaf_size_ = 0.3`, `submap_half_size_ = 100`, `submap_half_height_ = 30`, `min_input_points_ = 10`. Mới: `frame_stride_ = 1`, `start_frame_ = 0`, `max_correction_xy_ = 3.0`, `max_correction_yaw_deg_ = 5.0`, `max_input_range_ = 0` (0 = tắt).

5.1.2 localization.cpp – các hàm theo thứ tự chạy

Hàm	Làm gì
`Start`	Khởi tạo ROS, đọc tham số, mở file kết quả, liệt kê PLY, đọc prior, đọc bản đồ, mở file debug, chạy `RunLocalization`, rồi (nếu `keep_alive`) giữ chương trình chạy để RViz còn xem được.
`LoadParameters`	Khai báo + đọc ~30 tham số ROS; kiểm tra hợp lệ (đường dẫn không rỗng, số dương, `frame_stride ≥ 1`...).
`SetupRosInterfaces`	Tạo các kênh phát (publisher) cho RViz: `/image_point_cloud`, `/image_point_cloud_submap`, `/map_pc`, `/transformed_image_point_cloud_pose`, `/localization_path`, và TF.
`LoadMap`	Đọc bản đồ; nếu có file cache `map_downsampled_1m.pcd` thì đọc cache (vài giây) thay vì `map.pcd` 2.7 GB (~1 phút); nếu không thì giảm mẫu voxel và lưu cache.
`OpenDB3File` / `LoadDB3Frame`	Mở rosbag, chỉ lọc 2 topic GNSS và IMU; với mỗi frame, đọc tiếp các message có thời điểm ≤ thời điểm frame (không nhìn tương lai). **Lưu ý:** dữ liệu đọc được chỉ được in log, **không được dùng** trong thuật toán (TODO của TA để sinh viên tự dùng).
`IndexPlyFiles`	Liệt kê file .ply, lấy thời điểm từ tên file (`…_ts_<nanoseconds>.ply`), sắp theo thời gian. Không đọc nội dung (cho nhanh).
`LoadPlyFrame`	Đọc một file .ply ra hai dạng: chỉ toạ độ (để khớp) và có màu (để hiển thị).
`LoadInitialPose`	Pose ban đầu từ `init_guess.txt` (4 dòng × 4 số) hoặc từ tham số `initial_pose`.
`LoadPriorFile` (mới)	Đọc CSV prior `idx,x,y,yaw`; số dòng phải bằng số frame.
`PriorGuess` (mới)	Tạo pose bắt đầu: x, y, yaw từ prior; z, roll, pitch giữ của pose trước.
`WriteDebug` (mới)	Ghi một dòng CSV debug (định dạng ở Chương 4).
`ScalePlyPointCloud` / `ScalePlyColorCloud`	Nhân mọi toạ độ với `ply_scale`.
`FilterInputCloud`	Bỏ điểm NaN/vô cực; (mới) bỏ điểm xa hơn `max_input_range`; giảm mẫu voxel 0.3 m.
`CropSubmap`	Cắt một khối bản đồ ±100 m (ngang) × ±30 m (cao) quanh pose bắt đầu – NDT chỉ làm việc trên khối này.
`RunLocalization`	**Vòng lặp chính** qua mọi frame (trích dưới).
`AlignNDT`	Gọi NDT của PCL với các tham số, trả về pose và `fitness score` (khoảng cách TB bình phương).
`PublishGlobalMap` / `PublishFrame`	Gửi bản đồ, point cloud, submap, pose, quỹ đạo cho RViz.
`WritePose`	Ghi 12 số (3 hàng đầu của ma trận 4×4) vào file kết quả.
`ToTransformStamped` / `ToPoseStamped`	Đổi ma trận 4×4 sang message ROS (quaternion).
`TimestampNsFromPlyFilename`	Tách số sau `_ts_` trong tên file.

src/localization/src/localization.cpp – RunLocalization (rút gọn)

Note Hỏi thầy Physical AI

```
591 for (index = 0; index < ply_frames_.size(); ++index) {
594     if (index < start_frame_) continue;                        // (mới) bỏ qua frame trước đoạn cần c
    hạy
595     if (max_frames_ > 0 && processed >= max_frames_) break;    // giới hạn số frame
596     LoadDB3Frame(frame.timestamp_ns, db3_frame);               // đọc GNSS/IMU tới thời điểm này (chư
    a dùng)
598     prior = priors_.empty() ? nullptr : &priors_[index];
599     guess = prior ? PriorGuess(*prior, pose) : pose;           // (mới) điểm bắt đầu = prior, hoặc po
    se frame trước
602     if (prior && index % frame_stride_ != 0) {                 // (mới) frame không cần khớp: lấy luô
    n prior
603         pose = guess; WritePose(pose); ++processed; continue; }
611     LoadPlyFrame(frame, raw_cloud, raw_color);
612     input  = FilterInputCloud(ScalePlyPointCloud(raw_cloud, ply_scale_));   // nhân tỉ lệ, lọc, voxe
    l
613     submap = CropSubmap(guess);                                              // khối bản đồ quanh điểm
     bắt đầu
631     converged = input->size() >= min_input_points_ && AlignNDT(input, submap, guess, candidate, scor
    e);
635     accepted = converged;
636     if (converged && prior) {                                  // (mới) từ chối nếu kéo quá xa prior
637         dx, dy = candidate - guess;  dyaw = wrap(yaw(candidate) - yaw(guess));
641         accepted = hypot(dx,dy) <= max_correction_xy_ && |dyaw| <= max_correction_yaw_deg_;
643     }
644     WriteDebug(...);                                           // (mới)
646     pose = accepted ? candidate : guess;                       // không chấp nhận -> dùng điểm bắt đầ
    u
657     PublishFrame(...); WritePose(pose);
}
```

- Dòng 599: đây là thay đổi quan trọng nhất. Baseline TA luôn bắt đầu từ pose frame trước → sai một lần là sai mãi. Có prior thì mỗi frame bắt đầu độc lập từ GNSS → lỗi không cộng dồn, và các frame có thể chạy song song theo đoạn (`run_chunk.sh`).
- Dòng 636–642: "gating" (cổng chặn): kết quả NDT dời > 3 m hoặc quay > 5° so với prior gần như chắc chắn là khớp sai.
- Phần giữa hai dòng `TODO` (chữ Hán trong code, của TA: "hãy tự cài thuật toán định vị ở đây; hiện có NDT cơ bản") vẫn còn nguyên trong file.

PriorGuess (dòng 450–461), viết lại dạng giả mã

```
PriorGuess(prior, reference):
    rotation       = reference[0:3,0:3]
    reference_yaw  = atan2(rotation(1,0), rotation(0,0))
    yaw_delta      = quay quanh trục z một góc (prior.yaw - reference_yaw)
    guess          = reference                    // giữ z, roll, pitch của frame trước
    guess.rotation = yaw_delta * rotation         // chỉ thay yaw
    guess.x, guess.y = prior.x, prior.y
```

Minh bạch: điểm cần biết

Ba thư mục build khác nhau: `run_baseline.sh` nạp `install/`, `run_fusion.sh` và `run_image_only.sh` nạp `install_dev/`, `run_chunk.sh` nạp `install_par/`. Hướng dẫn chạy chỉ build `install_dev`. Muốn dùng `run_chunk.sh` / `run_baseline.sh` phải build thêm với `--install-base install_par` / mặc định, hoặc sửa script.

`launch/localization.launch.py` không khai báo các tham số mới (`prior_file`, `frame_stride`, `debug_file`…) – muốn dùng chúng phải chạy bằng `ros2 run` như các script `run_*.sh`.

Dữ liệu GNSS/IMU được node đọc nhưng không dùng; GNSS chỉ đi vào qua file prior CSV tạo bằng Python.

5.1.3 map_modified.cpp, visualization.cpp, launch và rviz

- `map_modified.cpp` (`DownsampleMap`): đọc PCD, giảm mẫu voxel `map_voxel_leaf_size` (mặc định 1 m), lưu PCD nhị phân; kiểm tra file ra khác file vào. Tạo cùng file cache mà `LoadMap` dùng.
- `visualization.cpp`: đọc bản đồ (lọc cường độ 10–256), danh sách tên file thời gian, file pose 12 số; với mỗi pose phát point cloud "radar" (đọc từ thư mục .pcd) và quỹ đạo, cách nhau 20

ms. Repo không dùng chương trình này.

- `localization.launch.py`: chạy node với `ply_path` mặc định `raw_ply`, cache `map_downsampled_1m.pcd`; các tham số số học chỉ được truyền khi người dùng gõ trên dòng lệnh (để C++ giữ mặc định); kèm RViz.

> **Không chắc chắn**
> `visualization.cpp` và `visualization.rviz` có vẻ là phần còn sót từ một bài tập trước (radar, cấu hình RViz kiểu ROS 1). Tài liệu không chắc nó còn chạy đúng với ROS 2 Humble – repo không có dấu vết đã chạy nó.

5.2 processing_codes/ – chuyển đổi và ghép (Python)

5.2.1 txt_to_idx_x_y_yaw.py (TA)

Mỗi dòng 12 số `r11 r12 r13 x r21 r22 r23 y r31 r32 r33 z` → `x = pose[3]`, `y = pose[7]`, `yaw = atan2(pose[4], pose[0]) = atan2(r21, r11)`. Có import `pandas`, `savgol_filter`, `gaussian_filter1d` nhưng không dùng.

5.2.2 gnss_to_csv.py

- `read_gnss(bag)`: mở rosbag bằng thư viện `rosbags`, đọc mọi message `/localizer_node/gnss_pose`, lấy thời điểm, x, y, yaw (từ quaternion), trả về yaw đã `unwrap`. Được nhiều công cụ khác import lại.
- `frame_timestamps(ply_dir)`: thời điểm từng frame từ tên file .ply (sắp theo tên = theo thời gian).
- `main`: nội suy tuyến tính x, y, yaw tới thời điểm từng frame (`np.interp`, frame ngoài khoảng GNSS lấy giá trị đầu/cuối), gói yaw về −π…π, ghi CSV.

5.2.3 gnss_smooth_to_csv.py

Như trên nhưng: (1) làm mượt Gauss x, y, yaw với σ tính bằng **số mẫu GNSS** (1.5 mẫu = 0.375 s); (2) nội suy bằng spline bậc 3; (3) chế độ `heading`: yaw = hướng của vector vận tốc (đạo hàm spline), khi xe chạy > 1 m/s; đứng yên thì giữ yaw GNSS. Dòng `heading = yaw + wrap(heading − yaw)` đưa hướng chạy về cùng "vòng" 2π với yaw GNSS trước khi trộn.

5.2.4 fuse_to_csv.py – ghép độ sửa của scan matching (dùng trong pipeline cuối)

processing_codes/fuse_to_csv.py – hàm fuse (rút gọn)

```
def fuse(prior, dbg, weight, yaw_weight, sigma_s, max_xy, max_yaw_deg, demean=False, need_converged=True
    ):
    dx, dy, dyaw, along, lat = corrections(dbg)              # độ sửa = ndt - guess; along/lat = theo hư
    ớng xe
    good = (hypot(dx, dy) <= max_xy) & (|deg(dyaw)| <= max_yaw_deg)   # cổng chặn: ≤ 2 m, ≤ 3° trong pip
    eline
    if need_converged: good &= converged
    if demean:                                               # trừ độ sửa trung bình (theo hệ xe)
        along -= mean(along[good]);  lat -= mean(lat[good])
        dx, dy = along*cos - lat*sin, along*sin + lat*cos     # quay về hệ bản đồ
    t = thời điểm (s) của các frame tốt
    all_t = nội suy thời điểm cho MỌI frame từ (idx, t) của các frame đã khớp
    out = prior.copy()
    step = median(diff(t))                                   # khoảng thời gian giữa 2 frame khớp
    for col, values, w in ((x, dx, weight), (y, dy, weight), (yaw, dyaw, yaw_weight)):
        v = median_filter(values[good], size=5)              # bỏ giá trị nhảy bất thường
        v = gaussian_filter1d(v, sigma_s / step)             # làm mượt σ = 2 s
        out[:, col] += w * interp(all_t, t, v)               # cộng vào prior ở mọi frame
    return out
```

- Pipeline gọi với `--yaw-weight 0 --sigma-s 2 --max-xy 2 --max-yaw-deg 3 --demean`: chỉ sửa vị trí, yaw giữ nguyên.

- Vì sao làm mượt 2 s: độ sửa từng frame rất nhiễu (~0.5–0.7 m), còn lỗi GNSS thay đổi chậm → lấy trung bình theo thời gian giữ lại phần chậm (đúng) và bỏ phần nhanh (nhiễu).
- Vì sao `--demean`: xem Chương 6.5 – GICP có một độ lệch cố định (~0.2 m) không phải lỗi của GNSS.
- Chế độ `--stats` chỉ in thống kê độ sửa (dùng cho các bảng ở `docs/nhat-ky/data/ndt_correction_stats*.txt`).

> **Ghi chú**
> Thời điểm được tính bằng `np.interp(all_idx, dbg['idx'], …)`: nếu file debug chỉ khớp mỗi 4 frame thì thời điểm các frame ở giữa được nội suy theo chỉ số – xấp xỉ hợp lý vì camera chụp đều ~28 Hz.

5.3 Các script shell ở thư mục gốc

Cả bốn đều chạy **trong container ROS**, nạp môi trường ROS, rồi gọi `ros2 run localization localization --ros-args -p …` với bản đồ `/root/data/TrackN/map.pcd`, cache `map_downsampled_1m.pcd`, PLY `raw_ply` (mặc định), `init_guess.txt`, `keep_alive:=false` (tự thoát). Tham số thêm truyền sau tên track/tag (`"$@"`).

Script	Khác biệt
`run_baseline.sh N`	Cấu hình TA mặc định; sau đó gọi `txt_to_idx_x_y_yaw.py` → `result_N.csv`.
`run_fusion.sh N TAG`	Thêm `prior_file` (mặc định `gnss_smooth_N.csv`, đổi bằng biến `PRIOR`), `debug_file`; `chown 1000:1000` để file ra thuộc người dùng máy host (container chạy bằng root).
`run_chunk.sh N TAG START COUNT`	Như fusion nhưng chỉ chạy `COUNT` frame từ `START`; ghi vào `result/chunks/`. Mở 10 tiến trình với START khác nhau → nhanh ~2.5 lần.
`run_image_only.sh N TAG`	Không prior: `ply_scale 9, ndt_resolution 4, input_voxel 1 m, submap ±60 m, max_input_range 40 m`; đổi sang CSV. Kết quả 150 frame đầu: trôi tới ~9 m (Track01), yaw sai ~30–40° (Track02) – `data/image_only_test.txt`.

> **Không chắc chắn**
> Chạy `run_chunk.sh` có thể đặt `ply_path` khác (ví dụ `3cameras_ply`) qua tham số thêm. Các lượt `c3s08`/`c3s10T2` dùng `3cameras_ply`, tỉ lệ 0.8 / 1.0, NDT 4 m, voxel 0.5 m, tầm 40 m, 50 vòng lặp, submap ±60 m (`docs/nhat-ky/03 §7`) – lệnh chính xác không được lưu trong repo.

5.4 tools/ – từng công cụ

5.4.1 rts_smoother.py – Kalman CTRV + RTS (bước ①)

Trạng thái 6 số `[x, y, θ, v, ω, b]`: vị trí, hướng chạy, tốc độ, tốc độ quay, và **b** = độ lệch cố định giữa yaw GNSS và hướng chạy. GNSS đo `(x, y, θ + b)`. Toán: Chương 6.2.

tools/rts_smoother.py (giả mã)

```
def predict(s, P, dt, q_a, q_w):
    x, y, th, v, w, b = s
    s2 = [x + v*cos(th)*dt, y + v*sin(th)*dt, th + w*dt, v, w, b]   # CTRV (xấp xỉ bậc 1): đi thẳng theo
     hướng, quay đều
    F  = Jacobian (đạo hàm của s2 theo s)                            # để lan truyền độ không chắc chắn
    Q[v,v] = (q_a*dt)^2;  Q[w,w] = (q_w*dt)^2                         # v, ω thay đổi ngẫu nhiên
    Q[x,x] = Q[y,y] = (0.5*q_a*dt^2)^2;  Q[th,th] = (0.5*q_w*dt^2)^2;  Q[b,b] = (1e-5*dt)^2
    return s2, F P F^T + Q, F

def run(times, meas, is_meas, params, smooth=True):
    R = diag(σp², σp², σyaw²)
    for mỗi thời điểm (GNSS hoặc frame, đã sắp xếp):
        dự đoán tới thời điểm này
        nếu là GNSS: đổi mới (innovation) = đo - H s (yaw gói về ±π); S = H P H^T + R; K = P H^T S^-1
                     s += K·innov;  P = (I - K H) P;  loglik += log N(innov; 0, S)
    chạy ngược (RTS): C = Pf[i] F[i+1]^T Pp[i+1]^-1;  xs[i] = xf[i] + C (xs[i+1] - xp[i+1])
```

- Chọn nhiễu: lưới 7 × 5 × 5 × 4 = 700 bộ (σ_p, σ_yaw, q_a, q_w), mỗi bộ chạy bộ lọc chỉ trên các mốc GNSS và cộng log-likelihood; lấy bộ lớn nhất. `--qa/--qw` cố định nhiễu quá trình (Track01 dùng q_a = 1 m/s², q_w = 0.1 rad/s² vì ML tự do chọn q_a = 3, làm hướng chạy rung theo GNSS giật).
- Sau đó gộp mốc GNSS và mốc frame thành một trục thời gian, chạy bộ lọc + RTS, đọc trạng thái ở các mốc frame → đúng thời điểm từng ảnh.
- `--yaw heading`: xuất θ (hướng chạy); `--yaw gnss`: xuất θ + b. `--time-offset`: dịch thời điểm frame (chỉ để thăm dò).
- Kết quả ghi trong `data/rts_smoother_output.txt`: Track01 b ≈ −0.42°, Track02 b ≈ +0.65°; σ_p chỉ vài mm → GNSS gần như không có nhiễu trắng.

Tạo phys_yawb_1.csv (Track01) là chuỗi 3 lệnh: `rts_smoother.py … --yaw gnss --qa 1.0 --qw 0.1` → lấy vị trí; `csv_ops.py merge` với yaw của `gnss_smooth_1.csv`; `csv_ops.py offset --yaw-deg 0.42` (bỏ lệch b mà bộ lọc ước lượng).

> **Ghi chú**
> Yaw của `phys_yawb_1` không đi vào kết quả cuối: bước ② và ⑤ thay yaw hoàn toàn bằng gyro + hướng chạy tính từ **vị trí** của file này. Chỉ vị trí Kalman là được dùng.

5.4.2 csv_ops.py

`merge OUT --pos A --yaw B`: lấy x, y của A và yaw của B, chép nguyên văn từng dòng. `offset IN OUT --lateral M --yaw-deg D`: dời mỗi pose M mét sang trái (theo yaw của dòng: `x −= M·sin(yaw), y += M·cos(yaw)`) và/hoặc cộng D độ vào yaw. Chỉ dùng thư viện chuẩn (chạy được không cần numpy).

5.4.3 yaw_fusion.py – bộ lọc bù yaw (bước ②, ⑤)

tools/yaw_fusion.py (giả mã)

```
t = thời điểm frame (từ 3cameras_ply/index.csv); pos = file vị trí
# 1) Hướng chạy: làm mượt x, y (Gauss σ = heading_sigma giây), đạo hàm → vận tốc → heading = unwrap(atan
    2(vy, vx))
moving = speed > min_speed (2 m/s)
# 2) Gyro: đọc /imu/data; trục z (--gyro-axis z) hoặc chiếu lên hướng trọng lực (mặc định)
cum = tích phân hình thang của gyro theo thời gian;  psi_raw = cum nội suy tại t
# 3) Bias: khớp (heading - psi_raw) ≈ c + b·t trên các frame đang chạy, 2 lượt, bỏ điểm lệch > 3σ
psi_g = psi_raw + c + b·t                      # gyro đã bỏ trôi tuyến tính và đặt đúng góc ban đầu
# 4) (tuỳ chọn) trượt: h_body = heading - atan(lever·ω / v)
# 5) Bộ lọc bù:
corr = weighted_smooth(unwrap(heading - psi_g), weight = moving, sigma = tau·fps)
yaw  = wrap(psi_g + corr)                      # gyro + phần chậm của (hướng chạy - gyro)
ghi idx, x, y lấy từ --positions, yaw mới
```

- `weighted_smooth` là làm mượt Gauss **có trọng số**: frame xe đứng yên (hướng chạy vô nghĩa) có trọng số 0, nên phần chậm ở đó được "mượn" từ các frame lân cận đang chạy.
- `--heading-from`: tính hướng chạy từ một file khác với file lấy vị trí (bước ⑤: vị trí sau GICP, nhưng hướng chạy từ quỹ đạo Kalman – vì vị trí sau GICP có thể rung hơn).
- `--lf-from-debug`: dùng yaw của scan matching thay cho hướng chạy làm phần chậm (thử nghiệm, không dùng).
- Cuối hàm in các chẩn đoán: bias gyro, tương quan tốc độ quay hướng chạy với gyro ở các độ trễ −0.2/0/+0.2 s, phần dư.

> **Minh bạch: điểm cần biết**
> Trong pipeline: Track01 `--gyro-axis z --tau 7 --heading-sigma 1.5`; Track02 `--gyro-axis z --tau 15 --heading-sigma 1.0`. τ, trục z (thay vì chiếu theo trọng lực – mặc định của code!) và việc tắt hiệu chỉnh trượt đều được **chọn bằng điểm Kaggle** (`docs/nhat-ky/08 §9.1`). Lưu ý: giá trị mặc định trong code (`tau 5`, `gravity`) không phải giá trị pipeline dùng.

5.4.4 gicp_refine.py – khớp point cloud vào bản đồ (bước ③)

Hàm	Làm gì
`read_pcd_binary, read_ply_xyz`	Đọc bản đồ (PCD nhị phân x y z) và PLY (ASCII cho raw_ply, nhị phân cho 3cameras_ply).
`prior_guess`	Bản Python của `PriorGuess` C++: x, y, yaw từ prior, z/roll/pitch từ pose tham chiếu.
`load_run_poses`	Đọc pose 4×4 của một lượt C++ cũ (file chính + các file chunk; chunk sau ghi đè).
`frame_list`	Danh sách (chỉ số, thời điểm, đường dẫn) từ `3cameras_ply/index.csv` hoặc tên file raw_ply.
`build_target`	Tiền xử lý bản đồ bằng small_gicp (voxel `--map-res`, 20 lân cận để tính hiệp phương sai/pháp tuyến). PLANAR: chỉ giữ điểm có pháp tuyến gần nằm ngang (tường, cột) + cây KD. VGICP: thêm lưới voxel Gauss.
`load_hood, in_hood`	Mặt nạ nắp capo: chiếu điểm lên ảnh camera trước bằng mô hình fisheye (hệ số méo D, ma trận K từ `camera_info`), bỏ điểm rơi vào vùng trắng của mặt nạ.
`statistical_outliers`	Lọc điểm lạc: điểm có khoảng cách TB tới K lân cận lớn hơn TB + STD × độ lệch chuẩn.
`source_cloud`	Point cloud nguồn: (hood) → bỏ điểm xa hơn `max_range` (tính trước khi nhân tỉ lệ) → nhân tỉ lệ → (SOR) → (bỏ mặt đường) → small_gicp tiền xử lý voxel 0.5 m.
`planar_icp`	ICP point-to-plane tự viết chỉ cho x, y, yaw; trọng số Geman-McClure `w = 1/(1+(r/k)²)²` giảm ảnh hưởng điểm sai.
`xy_information`	Rút Hessian 6×6 của GICP về ma trận 2×2 cho x, y thế giới (bổ sung Schur). Chương 6.4.
`register`	Xử lý một frame (chạy trong tiến trình con): dựng nguồn, trừ gốc toạ độ, gọi `small_gicp.align` hoặc `planar_icp`, cộng lại gốc, trả về một dòng debug.
`main`	Đọc tham số; tạo danh sách việc (từ `--init-run` hoặc `--prior` + `--pose-source`); đo độ cao mặt đường mọi frame (song song) → tỉ lệ; dựng bản đồ đích; chạy song song `--workers` tiến trình; ghi CSV.

tools/gicp_refine.py – register (rút gọn)

```
def register(job):
    idx, ts, path, init_T, guess_T, prior, scale, ground_h = job
    src = source_cloud(path, scale, a.max_range, a.voxel, opts)   # point cloud đã nhân tỉ lệ, giảm mẫu 0.5 m
    init_local = init_T.copy(); init_local[:3, 3] -= origin       # trừ gốc: toạ độ -5000 dùng float dễ mất chính xác
    res = small_gicp.align(target, src, tree, init_local, registration_type="GICP",
                           max_correspondence_distance=1.0, max_iterations=100, num_threads=1)
    T = res.T_target_source; T[:3, 3] += origin                   # pose kết quả trong hệ bản đồ
    info = xy_information(res.H, T[:3, :3])                       # độ chắc theo x, y
    return {..., "ndt_x": T[0,3], "ndt_y": T[1,3], "ndt_yaw": yaw_of(T), "scale": scale, "info_xx": info[0], ...}
```

- **Trừ gốc toạ độ** (`origin = round(mean(map))`): toạ độ bản đồ cỡ −5000 m; tính trong hệ có gốc gần đó tránh sai số làm tròn.
- **Tỉ lệ theo mặt đường** (`--ground-height 1.646 --ground-window 57`): đo h mỗi frame bằng `ground_scale.ground_height`, lọc trung vị 57 frame (~2 s) để bớt nhảy, tỉ lệ = 1.646 / h.
- Pipeline: `--map map_voxel_0.3m.pcd --map-res 0.3 --prior PRIOR --pose-source c3s08 --stride 1 --ground-height 1.646 --ground-window 57 --method GICP`; các giá trị khác lấy mặc định: `--voxel 0.5 --max-range 40 --max-corr 1.0 --iterations 100 --workers 10`.
- Cột `accepted` luôn ghi 1 (không gating ở đây) – gating làm ở `fuse_to_csv.py`. Cột `score` = error / số inlier.

Minh bạch: điểm cần biết

Khi dùng `--prior` mà quên `--pose-source` chương trình sẽ lỗi (`None.endswith`); khi dùng `--init-run` mà lượt C++ thiếu pose của một frame có trong file debug sẽ lỗi `KeyError`. Không có thông báo rõ ràng.

Bẫy đã gặp: Track02 phải dùng `--pose-source c3s10T2`; `c3s10` là của Track01 (độ cao z ~110 so với ~17) → GICP bắt đầu ở độ cao sai và không chạy.

`--map` mặc định là `map_downsampled_1m.pcd` nhưng hàm `read_pcd_binary` yêu cầu đúng 3 trường x y z – file cache của C++ (PCL) có thể có thêm trường đệm; tài liệu không chắc mặc định này chạy được (pipeline luôn truyền `map_voxel_0.3m.pcd` nên không bị ảnh hưởng).

5.4.5 ground_scale.py, pc_io.py, map_voxel.py

- `ground_height(pts)`: lấy điểm phía trước (x > 0) và gần (60 % gần nhất theo khoảng cách ngang), lấy 30 % thấp nhất theo z, histogram 60 ô, độ cao = tâm ô đông nhất; trả về `-z` (khoảng cách camera tới mặt đường). `main` in bảng h, tầm nhìn và độ cao mặt đất bản đồ dưới vị trí GNSS cho mỗi frame thứ STEP, rồi tỉ lệ ứng với các chiều cao camera 1.5/1.8/2.0/2.2 m.
- `pc_io.py`: đọc PLY/PCD (ASCII hoặc nhị phân) bằng numpy, không cần PCL.
- `map_voxel.py`: đọc map.pcd 2.7 GB **theo khối** (5 triệu điểm hoặc 80 MB chữ một lần) để không tràn RAM; mỗi khối tính chỉ số voxel `floor(p / leaf)`, cộng dồn tổng toạ độ và số điểm mỗi voxel; cuối cùng gộp các khối và lấy trọng tâm; ghi PCD nhị phân.

5.4.6 info_fuse.py – bộ ghép bình phương tối thiểu (ứng viên mới, chưa có điểm)

Thay cho "lọc trung vị + Gauss 2 s" của `fuse_to_csv.py`: giải một bài toán tối ưu trên **toàn bộ quỹ đạo** cùng lúc, tin mỗi phép khớp nhiều hơn ở hướng nó đo chắc (Chương 6.4). Hàm `weights` tạo ma trận trọng số 2×2 mỗi frame từ Hessian; `solve` lập hệ phương trình thưa 2n ẩn và giải bằng `scipy.sparse.linalg.spsolve`. `--vehicle-noise` thay Hessian bằng một mô hình nhiễu dọc/ngang đo từ dữ liệu. Đầu ra giữ yaw của prior (yaw thay sau bằng `yaw_fusion.py`).

5.4.7 rts_imu_smoother.py – Kalman có gyro và yaw GNSS trễ (thí nghiệm)

Trạng thái 7 số `[x, y, θ, v, ω, b, bg]` (thêm bias gyro bg). Đo: vị trí GNSS ở thời điểm của nó; yaw GNSS = θ(t − lag) + b (mô hình trễ: đặt phép đo yaw vào trục thời gian sớm hơn `yaw_lag` giây); gyro (trung bình mỗi 50 ms) = ω + bg. Tham số (lag, lệch đồng hồ IMU, 6 mức nhiễu) chọn bằng maximum likelihood: quét lag −1...3 s, quét lệch IMU, rồi "coordinate ascent" (lần lượt tối ưu từng tham số, 3 vòng). Kết quả: trễ 1.11 s (T1), 1.41 s (T2). Yaw bù trễ **làm điểm xấu đi** nên không dùng.

Ghi chú

Hàm `run` không lưu ma trận P của bước RTS (chỉ lưu trạng thái) – đủ cho mục đích xuất trạng thái, không ảnh hưởng kết quả.

Tại sao

Viết vào report

5.4.8 Các công cụ phân tích

File	Làm gì / kết quả chính
analyze_gnss_imu.py	Thống kê GNSS (tần số, quãng đường, độ giật, lệch hướng chạy–yaw), init_guess so với GNSS, IMU (gia tốc TB, bias gyro, tương quan với tốc độ quay GNSS theo độ trễ). → data/gnss_imu_analysis.txt.
analyze_yaw_lag.py	So yaw GNSS với hướng chạy của chính vị trí GNSS ở các độ trễ −3…3 s → yaw GNSS trễ +1.20 s / +1.40 s.
check_frame_alignment.py	Đặt point cloud vào bản đồ tại pose GNSS, dò lưới tỉ lệ × góc × dịch, chấm bằng khoảng cách tới bản đồ (chỉ điểm trên mặt đường). Kết luận: hệ FLU đúng, ×9 nhất quán. Ghi chú trong repo: thước đo này thiên vị tỉ lệ nhỏ – phải nhìn tính nhất quán của góc.
time_offset.py	So gia tốc dọc đường từ GNSS với gia tốc kế IMU theo độ trễ, bootstrap khối 10 s → ±75–220 ms: không đo được độ lệch cỡ 20 ms.
map_fit.py	Thước đo không cần đáp án: đặt point cloud (điểm cao hơn mặt đường 0.3 m) theo pose của file, đo khoảng cách tới bản đồ; --validate so thứ hạng với điểm Kaggle (Spearman 0.92 / 0.67). Bị GICP đánh lừa → chỉ dùng loại cấu hình hỏng. Cần file reg/clean/clean_gicp_p1_TrackN_debug.csv (không có trong git).
three_hat.py	Ước lượng sai số từng nguồn từ phương sai hiệu từng cặp (cần ≥ 3 nguồn độc lập). Thất bại: các phép khớp đều bắt đầu từ GNSS nên không độc lập (phương sai âm).
segment_analysis.py	(1) Hessian có dự báo nhiễu từng frame không → tương quan ≈ 0; (2) bảng 10 đoạn: tốc độ, độ cong, tỉ lệ, % hội tụ, nhiễu.
track_error.py	So quỹ đạo với một quỹ đạo tham chiếu, in lỗi theo cửa sổ và "điểm" theo công thức Kaggle (dùng GNSS làm tham chiếu).
compare_csv.py	RMS chênh vị trí và yaw giữa từng cặp file → data/file_differences.txt.
submission_stats.py	Đọc lịch sử nộp (JSON từ Kaggle API + CSV log + điểm chính xác), phân loại bằng biểu thức chính quy theo tên file, vẽ biểu đồ, sinh 09-thong-ke-nop-bai.md. Cần matplotlib.
generate_result.py	Tự động: GNSS làm mượt → (tuỳ chọn) run_fusion.sh → fuse_to_csv.py theo cấu hình JSON. Thuộc giai đoạn NDT, không dùng nữa.
merge_debug.py	Gộp nhiều file debug (theo chỉ số frame, file sau thắng).
start_yaw_fix.py	Trước thời điểm xe vượt 4 m/s: yaw = hướng chạy lúc đó trừ ngược tích phân gyro (bias đo lúc đứng yên); hoà trộn 1 s. Track02 tốt lên 0.010 (nhưng chạy trên nền file tổ hợp).
curvature_yaw.py	yaw = hướng chạy − atan(l·κ) (κ = độ cong): điểm đo trước cầu sau l mét thì hướng vận tốc lệch thân xe khi vào cua. Không có ích.

5.4.9 Các công cụ dò bảng điểm (chỉ còn giá trị phân tích)

File	Làm gì
analyze_probes.py	Từ điểm của các cặp file thăm dò ± (dịch thời gian, dời ngang, cộng yaw) giải ra phần điểm của yaw c = 0.2·Y bằng chia đôi (bisection).
yaw_projection.py	Giữ vị trí cố định → mỗi file chỉ đổi yaw cho biết chính xác lỗi yaw của nó → tìm tổ hợp tuyến tính các hướng thay đổi yaw tốt nhất a = $-G^{-1}m$.
pos_projection.py	Như trên cho vị trí.
make_probes.py	Sinh file = file nền + biên độ × cos(πkt/T) cộng vào x, y hoặc yaw (dao động DCT bậc k).
bump_yaw.py	Cộng một "bướu" yaw $\cos^2$ vào một cửa sổ thời gian.

là sao

5.4.10 reproduce_submissions.sh và run_clean_pipeline.sh

run_clean_pipeline.sh chạy trên máy host (ngoài container): mỗi lệnh Python chạy trong một container sdc-reg dùng một lần (docker run --rm -u uid:gid -v ~/SDC-midterm01:/w). Hàm fuse gói sẵn các cờ --yaw-weight 0 --sigma-s 2 --max-xy 2 --max-yaw-deg 3 --demean. Vòng for k in 1 2 chính là "2 lượt GICP". Repo ghi đã kiểm tra: kết quả trùng 100 % với clean_*.

Minh bạch: điểm cần biết

`reproduce_submissions.sh` chỉ tạo lại 34 file nộp đầu tiên (17–18/10), nhưng ở cuối nó so **mọi** file trong `docs/nhat-ky/submissions/` (nay 111 file). Chạy bây giờ sẽ in `DIFFERENT` cho ~77 file chưa được tạo lại và trả mã lỗi 1. Script lỗi thời, không phải dấu hiệu code hỏng. Tương tự, `06-nhan-xet.md` ghi "19/19" còn `07` ghi "34/34".

5.5 docs/report/make_report.py và video/

`make_report.py`: dùng reportlab + matplotlib (font Noto, ký hiệu toán bằng DejaVu) để dựng `bao_cao_tong_hop.pdf` từ lịch sử nộp và nhật ký; vẽ 4 hình (dòng thời gian điểm, ảnh hưởng τ, lỗi vị trí, sơ đồ pipeline). Chạy trong image `sdc-video`.

Bộ video (`video/`) đã mô tả ở Chương 3.6. Điểm kỹ thuật đáng chú ý: `scenes/c3_06_icp.py` tính ICP thật bằng numpy cho animation; `lib/tts.py` cache từng câu đọc (sửa một câu chỉ đọc lại câu đó) và tự thử lại khi dịch vụ Microsoft không trả âm thanh; `build.py` đặt mỗi câu đúng thời điểm bắt đầu "nhịp" thật mà Manim ghi ra (`beats.json`), giảm âm lượng nhạc khi có giọng (ducking), in phụ đề vào một dải riêng dưới hình.

Note: Tìm các bài toán tương tự xem họ đã giải quyết như nào

Chương 6. Toán học và thuật toán phía sau

Mỗi mục: trực giác trước, công thức sau, rồi chỉ ra dòng code tương ứng. Có thể bỏ qua công thức mà vẫn hiểu ý.

6.1 Scan matching như một bài toán tối ưu

Trực giác: tìm phép quay R và dịch t sao cho point cloud sau khi biến đổi "nằm sát" bản đồ nhất. "Sát" đo bằng tổng bình phương khoảng cách giữa các cặp điểm.

$$\min_{R,t} \sum_i \| R\,p_i + t - q_i \|^2$$

ICP điểm–điểm: p_i điểm point cloud, q_i điểm bản đồ gần nhất

$$\min_{R,t} \sum_i \left(n_i^\top (R\,p_i + t - q_i) \right)^2$$

point-to-plane: chỉ phạt theo pháp tuyến n_i của mặt bản đồ

$$\min_{R,t} \sum_i d_i^\top \left(C_i^Q + R\,C_i^P R^\top \right)^{-1} d_i, \quad d_i = q_i - (R\,p_i + t)$$

GICP: C là hiệp phương sai (hình dạng bề mặt) quanh mỗi điểm, ước lượng từ 20 lân cận

Vì có góc quay nên bài toán không tuyến tính; thư viện giải lặp kiểu Gauss–Newton: tuyến tính hoá quanh pose hiện tại, giải hệ H·Δ = −g (H là **Hessian** – ma trận "độ cong" của hàm sai số), cập nhật pose, lặp tới khi Δ rất nhỏ. Ma trận H còn cho biết phép khớp **chắc chắn tới đâu theo từng hướng** (mục 6.4).

ICP 3 bậc tự do tự viết (`planar_icp`) làm đúng điều này cho 3 ẩn (dx, dy, dθ): Jacobian mỗi điểm `J = [n_x, n_y, −n_x·(p_y − c_y) + n_y·(p_x − c_x)]`, trọng số Geman–McClure để điểm sai (phần dư r lớn) gần như bị bỏ qua:

$$w_i = \frac{1}{(1 + (r_i/k)^2)^2}, \qquad \Delta = -(J^\top W J)^{-1} J^\top W r$$

k = 0.3 m (--kernel)

6.2 Bộ lọc Kalman mở rộng với mô hình CTRV và RTS smoother

Trực giác ở Chương 2.8. **Trạng thái** s = [x, y, θ, v, ω, b]. **Dự đoán** sau Δt giây (code dùng xấp xỉ bậc một của CTRV):

$$x' = x + v\cos\theta\,\Delta t, \quad y' = y + v\sin\theta\,\Delta t, \quad \theta' = \theta + \omega\,\Delta t, \quad v' = v,\ \omega' = \omega,\ b' = b$$

$$P' = FPF^\top + Q, \qquad Q = \mathrm{diag}\left(\tfrac{(q_a \Delta t^2)^2}{4}, \tfrac{(q_a \Delta t^2)^2}{4}, \tfrac{(q_w \Delta t^2)^2}{4}, (q_a \Delta t)^2, (q_w \Delta t)^2, (10^{-5}\Delta t)^2 \right)$$

F = ma trận đạo hàm của phép dự đoán theo trạng thái (Jacobian); Q = nhiễu quá trình

Cập nhật khi có GNSS z = (x, y, yaw), với mô hình đo h(s) = (x, y, θ + b):

$$\nu = z - Hs, \quad S = HPH^\top + R, \quad K = PH^\top S^{-1}, \quad s \leftarrow s + K\nu, \quad P \leftarrow (I - KH)P$$

ν: "đổi mới" (innovation) – phần phép đo không dự đoán được; K: độ lợi Kalman – tin phép đo bao nhiêu

Maximum likelihood: nếu mô hình và các mức nhiễu đúng, ν phải phân bố chuẩn với hiệp phương sai S. Log-likelihood của cả chuỗi:

$$\log L = -\tfrac{1}{2} \sum_k \left(\nu_k^\top S_k^{-1} \nu_k + \log\det S_k + 3\log 2\pi \right)$$

Code thử 700 bộ (σ_p, σ_yaw, q_a, q_w) và chọn bộ có log L lớn nhất – không dùng đáp án. **RTS smoother** chạy ngược:

$$C_i = P_i^f F_{i+1}^\top \left(P_{i+1}^p\right)^{-1}, \qquad \hat{s}_i = s_i^f + C_i\left(\hat{s}_{i+1} - s_{i+1}^p\right)$$

f: sau lọc xuôi; p: sau dự đoán; ŝ: sau làm mượt (rts_smoother.py dòng ~98–104)

Ví dụ đời thường

Kết quả cho Track02: σ vị trí 0.006 m, σ yaw 1.0°, q_a = 1 m/s², q_w = 0.03 rad/s², b = +0.653°. Đọc ra: vị trí GNSS gần như không có nhiễu ngẫu nhiên (vài mm) – sai số của nó là loại **trôi chậm** mà lọc không khử được; lợi ích chủ yếu đến từ yaw.

6.3 Bộ lọc bù cho yaw

Trực giác ở Chương 2.9. Gọi $\psi_g(t)$ là tích phân gyro đã bỏ bias, h(t) là hướng chạy, G_τ là làm mượt Gauss độ rộng τ (có trọng số: chỉ các frame xe chạy > 2 m/s):

$$\psi(t) = \psi_g(t) + G_\tau[h - \psi_g](t)$$

Phần trong ngoặc chỉ chứa (lỗi nhiễu của h) + (trôi chậm của gyro). Làm mượt với τ lớn giữ lại trôi chậm và bỏ nhiễu nhanh; cộng lại vào ψ_g thì trôi bị triệt, còn các thay đổi nhanh vẫn lấy từ gyro. τ nhỏ → tin hướng chạy nhiều hơn; τ lớn → tin gyro nhiều hơn.

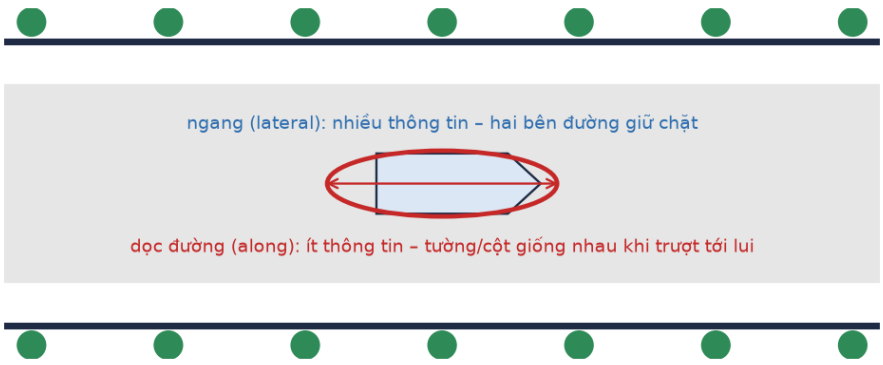
Bias được ước lượng trước bằng bình phương tối thiểu (2 lượt, bỏ điểm lệch > 3σ):

$$\min_{c,b} \sum_{t\,:\,v>2} (h(t) - \psi_{raw}(t) - c - b\,t)^2, \qquad \psi_g = \psi_{raw} + c + b\,t$$

Ý chính

Kết quả: lỗi yaw Track01 1.56° → 0.87°, Track02 0.33° → 0.25° (đo được nhờ giữ vị trí cố định khi đổi yaw). Kết luận của repo: **yaw của đáp án là yaw thân xe**, và gyro mô tả phần nhanh của nó tốt hơn mọi nguồn khác.

6.4 Hessian, thông tin theo hướng và bộ ghép bình phương tối thiểu



Hình 18. Trên đường thẳng, khớp point cloud giữ chặt hướng ngang nhưng lỏng hướng dọc.

Hessian H (6×6, cho 3 góc + 3 dịch) của GICP đo độ "cong" của hàm sai số: cong mạnh theo hướng nào thì pose bị "khoá" chặt theo hướng đó. Để có ma trận 2×2 chỉ cho x, y (thế giới), code đổi sang hệ thế giới rồi lấy **bổ sung Schur** (loại bỏ ảnh hưởng của 3 góc và z):

$$S = H_{kk} - H_{kr} H_{rr}^{+} H_{rk}, \qquad k = \{x, y\},\ r = \{\text{roll}, \text{pitch}, \text{yaw}, z\}$$

xy_information() trong gicp_refine.py; H^+ là nghịch đảo giả

Đo được: thông tin dọc đường chỉ ~0.51 lần ngang (Track01). **info_fuse.py** dùng điều này: tìm độ sửa c_i cho mọi frame i bằng cách cực tiểu

Có sai số làm tròn ko

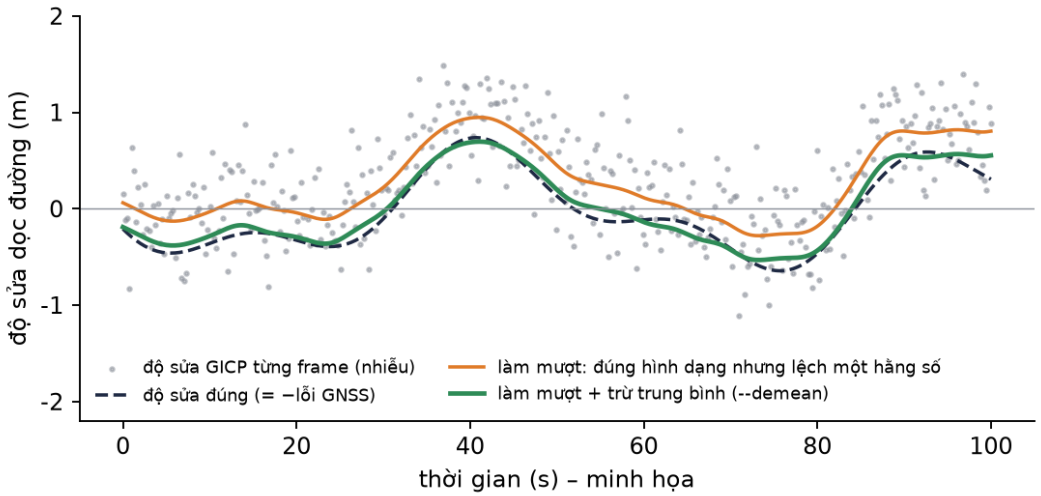
$$\sum_i \frac{\|c_i\|^2}{\sigma_{prior}^2} + \sum_i \frac{\|c_{i+1} - c_i\|^2}{q\,\Delta t_i} + \sum_{i \in \text{khớp}} (c_i - d_i)^\top W_i (c_i - d_i)$$

- Số hạng 1: lỗi GNSS có giới hạn (σ_prior). Số hạng 2: lỗi GNSS thay đổi chậm (bước ngẫu nhiên, q m²/s). Số hạng 3: bám theo độ sửa d_i của GICP với trọng số W_i.
- W_i giữ **hình dạng** Hessian nhưng chuẩn hoá: trị riêng $(1, (\lambda_{yếu}/\lambda_{mạnh})^\gamma)/\sigma_{match}^2$. γ = 0: mọi hướng như nhau (`if_iso`); γ = 1: theo Hessian (`if_aniso`).
- Đây là bài toán bình phương tối thiểu tuyến tính → hệ phương trình thưa, giải một lần.

Minh bạch: điểm cần biết

Phân tích sau đó (`segment_analysis.py`) cho thấy Hessian đúng xu hướng trung bình nhưng **không dự báo được nhiễu từng frame** (tương quan ≈ 0), và sau lọc nhiễu gần như đẳng hướng (0.505 / 0.494 m). Vì vậy repo không kỳ vọng `if_aniso` tốt hơn nhiều. Điểm thật của `if_iso_*` / `if_aniso_*` chưa có (xếp hàng nộp 22/10).

6.5 Vì sao trừ độ lệch trung bình (--demean)



Hình 19. Minh hoạ: độ sửa GICP có hình dạng đúng nhưng lệch một hằng số; trừ trung bình thì sát độ sửa đúng hơn.

Độ sửa GICP ở Track01 có trung bình khác 0: dọc +0.17 m, ngang −0.14 m (lượt 1). Thăm dò bảng điểm cho thấy GNSS Track01 **không** lệch ngang (+0.025 m) và không lệch thời gian (−3.6 ms) → độ lệch cố định này là của phép khớp ảnh (có lẽ do vị trí camera trong point cloud MapAnything), không phải của GNSS. Cách xử lý: GNSS giữ **phần trung bình**, GICP chỉ đóng góp **phần dao động** quanh trung bình. Kết quả: 0.99060 → 0.92775.

Minh bạch: điểm cần biết

Bằng chứng "GNSS không lệch ngang" đến từ một lượt **thăm dò bảng điểm** (`probe_lat±0.5_1`). Repo xếp `--demean` vào loại "chỉnh tham số bình thường" có lý do vật lý; nên nêu rõ nguồn gốc này nếu bị hỏi.

Tại sao lệch nhiều như vậy và cách mà bạn chỉnh các thông số như nào cho ít sai số nhất

6.6 Toán của việc "dò bảng điểm" (chỉ để hiểu – không được dùng chấm)

Ý tưởng: điểm S = P + 0.2Y. Nếu nộp một file chỉ khác file gốc bởi một thay đổi biết trước D (ví dụ cộng +L mét sang trái), thì

$$P(+L)^2 = P^2 + 2L\,\overline{e\cdot n} + L^2, \qquad P(+L)^2 + P(-L)^2 - 2P^2 = 2L^2$$

Với cặp ± thì số hạng chứa lỗi chưa biết e triệt tiêu → giải ra được P và Y riêng từ tổng điểm. Ba cặp thăm dò độc lập cho Track01 cùng ra 0.905 m + 1.77° (chênh 0.0014) – rất nhất quán. Tổng quát hơn, với nhiều "hướng" thay đổi D_k đã nộp, mỗi điểm cho một con số m_k = mean($e\cdot D_k$), và tổ hợp tốt nhất là $a = -G^{-1}m$ với G_{ij} = mean($D_i\cdot D_j$) (`yaw_projection.py`, `pos_projection.py`). Với các dao động cosin (`make_probes.py`) cách này gần như **đo lại đáp án** theo từng tần số → điểm 0.46 / 0.19.

> **Minh bạch: điểm cần biết**
> Đây chính là "leaderboard-based fitting" mà TA cấm dùng cho bài chấm. Trọng số thu được (ví dụ −4.58) không có ý nghĩa vật lý. Nội dung này chỉ nên xuất hiện trong báo cáo như phần phân tích chiến lược, kèm nói rõ đã dừng sau khi TA trả lời.

Viết vào report như 1 phần thú vị

Chương 7. Các tham số quan trọng và ảnh hưởng của chúng

Cột "Chọn bằng" cho biết giá trị đến từ đâu: **dữ liệu** (đo/ước lượng từ dữ liệu, không cần đáp án), **vật lý** (giá trị hợp lý theo hiểu biết), **Kaggle** (thử nhiều giá trị, giữ giá trị cho điểm tốt nhất – "chỉnh tham số bình thường", TA cho phép nhưng nên khai báo).

7.1 Pipeline sạch

Tham số	Giá trị	Ở đâu	Ảnh hưởng (số liệu từ nhật ký)	Chọn bằng
Nhiễu Kalman Track01 q_a, q_w	1 m/s², 0.1 rad/s²	`rts_smoother.py --qa --qw`	ML tự do chọn q_a = 3 → quỹ đạo rung theo GNSS giật	vật lý
Nhiễu Kalman Track02	tự chọn (σ_p 0.006 m, σ_yaw 1°, q_a 1, q_w 0.03)	`rts_smoother.py`	0.47281 → 0.40778	dữ liệu (ML)
Lệch yaw GNSS Track01	+0.42°	`csv_ops.py offset`	1.25992 → 1.25338 (không vào kết quả cuối vì yaw bị thay)	dữ liệu (Kalman ước lượng b)
Tỉ lệ point cloud	1.646 / h mỗi frame (≈ 0.8 TB)	`--ground-height 1.646`	tỉ lệ 1.0: 0.934 m; 0.8: 0.836 m (NDT); theo mặt đường: 0.803 → 0.752 m (GICP)	Kaggle (0.8) + dữ liệu (h)
Cửa sổ lọc trung vị h	57 frame (~2 s)	`--ground-window`	bớt nhảy tỉ lệ giữa các frame	chọn tay
Số lượt GICP	2	`run_clean_pipeline.sh`	xem bảng 7.2: lượt 2 tốt nhất, lượt 5 xấu hẳn	Kaggle
Phương pháp khớp	GICP	`--method`	xem bảng 7.2	Kaggle
Bản đồ / nguồn voxel	0.3 m / 0.5 m	`--map-res / --voxel`	–	vật lý / mặc định
Tầm point cloud	≤ 40 m	`--max-range`	chưa thử giá trị khác (đề xuất tiếp theo: 20/30/40 m)	vật lý
Khoảng ghép cặp tối đa	1.0 m	`--max-corr`	–	mặc định
Cổng chặn độ sửa	≤ 2 m, ≤ 3°	`fuse_to_csv.py --max-xy --max-yaw-deg`	loại phép khớp sai	vật lý
Làm mượt độ sửa	trung vị 5 + Gauss σ 2 s	`--sigma-s 2`	–	vật lý (từ giai đoạn NDT)
Trừ trung bình	bật	`--demean`	0.99060 → 0.92775	Kaggle + lý do vật lý
τ bộ lọc bù yaw	7 s (T1), 15 s (T2)	`yaw_fusion.py --tau`	xem bảng 7.2, 7.3	Kaggle
Trục gyro	z (không chiếu theo trọng lực)	`--gyro-axis z`	xem bảng 7.2	Kaggle
Hiệu chỉnh trượt	tắt	`--lever 0`	bật làm xấu đi (bảng 7.2, 7.3)	Kaggle
σ làm mượt vị trí cho hướng chạy	1.5 s (T1), 1.0 s (T2)	`--heading-sigma`	–	Kaggle / chọn tay
Tốc độ tối thiểu để tin hướng chạy	2 m/s	`--min-speed`	–	mặc định

7.2 Số lượt GICP, phương pháp khớp, τ, trục gyro (Track01, chỉ đổi đúng một thứ)

Thử nghiệm	Lỗi vị trí (m)	Lỗi yaw (°)	Điểm
GICP lượt 1 / 2 / 5 (tỉ lệ mặt đường, mọi frame)	0.714 / 0.679 / 0.801	1.570	1.02813 / 0.99283 / 1.11461
GICP lượt 2 / 3 / 4 có demean	0.616 / 0.623 / 0.655	1.559	0.92775 / 0.93564 / 0.96735
NDT C++ / GICP / NDT→GICP (tỉ lệ 0.8, mỗi 4 frame)	0.836 / 0.803 / 0.769	1.570	1.15042 / 1.11701 / 1.08265
ICP 3 bậc tự do (PLANAR) / point-to-plane / GICP lượt 1	0.719 / 0.681 / 0.714	1.570	1.03311 / 0.99504 / 1.02813
+ yaw của GICP thay vì giữ yaw	–	1.570 → ~1.627	1.02813 → 1.03955 (xấu đi)
Yaw gyro trọng lực τ 30 / 15 / 10 s (+ trượt)	0.616	1.085 / 0.952 / 0.899	0.83296 / 0.80646 / 0.79585
Yaw gyro trục z τ 15 / 10 / 7 s (không trượt)	0.616	0.940 / 0.876 / 0.865	0.80411 / 0.79127 / 0.78914

> **Ghi chú**
> Lỗi vị trí/yaw ở bảng này được suy ra từ điểm bằng cách giữ một thành phần cố định (ví dụ khi chỉ đổi yaw, lỗi vị trí đã biết = 0.616 m, nên lỗi yaw = (điểm − 0.616)/0.2). Lỗi vị trí ban đầu 0.905 m lại được đo bằng các file thăm dò – một phần của chuỗi suy luận này dựa vào dò bảng điểm.

7.3 Track02: τ của bộ lọc bù yaw

Cấu hình	Điểm	Lỗi yaw (°)
trục z, τ 5 s	0.29268	0.330
trọng lực τ 5 / + trượt τ 5	0.29756 / 0.30531	0.354 / 0.393
+ trượt, τ 2 / 15 s	0.31258 / 0.28743	0.429 / 0.304
trục z, τ 10 / 15 / 30 s	0.27949 / **0.27654** / 0.28034	0.264 / **0.249** / 0.268

7.4 Tham số của node C++ (NDT)

Tham số	Mặc định TA	Repo dùng	Ảnh hưởng
`ply_scale`	3	9 (raw_ply), 0.8/1.0 (3cameras)	×3: NDT trôi ~100 m; ×9: độ sửa trung vị 0.23 m (T2), yaw ±2°
`ndt_resolution`	8 m	4 m	8 m quá thô cho point cloud phủ ~27 m: độ sửa 1.55 m → 0.23 m
`ndt_max_iterations`	30	50	–
`submap_half_size`	100 m	60 m	nhanh hơn
`input_voxel_leaf_size`	0.3 m	0.3 / 0.5 / 1 m	nhanh hơn với 3cameras_ply (~330 nghìn điểm)
`frame_stride`	(mới) 1	2, 4, 10, 20	thí nghiệm nhanh
`max_correction_xy / yaw_deg`	(mới) 3 m / 5°	10 m / 30° khi muốn thấy độ sửa thật	loại khớp sai
`max_input_range`	(mới) 0 = tắt	40 m	độ sâu từ ảnh càng xa càng sai

Chương 8. Cài đặt và chạy lại từ đầu, từng bước

Dựa trên `docs/HUONG-DAN-CHAY.md`, bổ sung những chỗ hướng dẫn gốc ngầm hiểu. Máy dùng Ubuntu + Docker; không cài ROS hay thư viện Python lên máy thật.

8.1 Chuẩn bị (một lần)

1. Cài Docker; thêm tài khoản vào nhóm docker: `sudo usermod -aG docker $USER` rồi đăng nhập lại.
2. Tạo thư mục và đặt đúng chỗ (đường dẫn trong `docker-compose.yml` đang cứng là `/home/thuhang/SDC-midterm01`):

```
~/SDC-midterm01/
├── data/Track1/  map.pcd  Track1.db3  metadata.yaml  init_guess.txt  raw_ply/  3cameras_ply/  Track1_im
    ages/
├── data/Track2/  (tương tự)
├── midterm01/            ← clone repo vào đây:  gh repo clone mochihat/SDC-midterm-1- midterm01
└── docker-compose.yml    ← chép từ midterm01/docker/ (kèm Dockerfile, ros_entrypoint.sh)
```

1. Tải dữ liệu: trang Kaggle không có file zip gộp qua API và API bị giới hạn tốc độ (429) → tải các file zip bằng trình duyệt, giải nén như trên.
2. Build hai image:

```
cd ~/SDC-midterm01
docker compose build                                  # image sdc_ros2_dev (ROS 2 Humble), ~1 phút
docker build -t sdc-reg midterm01/docker/reg          # image Python + small_gicp
```

8.2 Build và (tuỳ chọn) xem bằng RViz

```
xhost +local:                                         # cho Docker mở cửa sổ
docker compose up -d && docker compose exec sdc_ros2_dev bash
# --- trong container ---
cd /root/midterm && source /opt/ros/humble/setup.bash
colcon build --build-base build_dev --install-base install_dev
source install_dev/setup.bash
ros2 launch localization localization.launch.py data_path:=/root/data/Track1 \
  ply_scale:=9.0 ndt_resolution:=4.0 max_frames:=100 save_file:=rviz_test_Track1_result.txt
```

Bước RViz không cần cho kết quả; hữu ích để chụp hình báo cáo (thử `ply_scale:=3.0` để thấy point cloud nhỏ hơn bản đồ và NDT trôi).

8.3 Bước 1 – GNSS + Kalman (trong container ROS)

```
O=/root/midterm/src/localization/result
cd /root/midterm/src/localization/processing_codes
python3 gnss_smooth_to_csv.py /root/data/Track1 Track1 $O/gnss_smooth_1.csv 1.5 gnss
cd /root/midterm/tools
python3 rts_smoother.py /root/data/Track1 Track1 $O/rts_phys_qa1.0_1.csv --yaw gnss --qa 1.0 --qw 0.1
python3 csv_ops.py merge  $O/rts_pos_gnssyaw_1.csv --pos $O/rts_phys_qa1.0_1.csv --yaw $O/gnss_smooth_1.
    csv
python3 csv_ops.py offset $O/rts_pos_gnssyaw_1.csv $O/phys_yawb_1.csv --yaw-deg 0.42
python3 rts_smoother.py /root/data/Track2 Track2 $O/rts_heading_2.csv --yaw heading
```

Kết quả: `phys_yawb_1.csv` (điểm đã biết 1.25338) và `rts_heading_2.csv` (0.40778). Có thể so từng byte với bản trong `docs/nhat-ky/submissions/`.

8.4 Bước 2 – Chuẩn bị cho GICP

```
cd ~/SDC-midterm01        # trên máy, ngoài container
for n in 1 2; do docker run --rm -u $(id -u):$(id -g) -v $PWD:/w -w /w/midterm01 sdc-reg \
  python3 tools/map_voxel.py /w/data/Track$n/map.pcd /w/data/Track$n/map_voxel_0.3m.pcd 0.3; done
```

Minh bạch: điểm cần biết

Pipeline còn cần pose C++ `c3s08` (Track01) / `c3s10T2` (Track02) trong `src/localization/result/` và `result/chunks/` – **không có trong repo**. Nếu máy đã mất chúng phải chạy lại NDT trên `3cameras_ply` (~14 s/frame; chia 10 đoạn bằng `run_chunk.sh`, cần build `install_par`; prior/tham số chính xác của lượt cũ không được lưu – xem Chương 5.3). Nên sao lưu các file này ngay.

8.5 Bước 3 – Pipeline sạch: một lệnh (trên máy)

```
cd ~/SDC-midterm01
bash midterm01/tools/run_clean_pipeline.sh          # ~10 phút
# → midterm01/src/localization/result/reg/rerun/result_1.csv, result_2.csv
```

8.6 Bước 4 – Kiểm tra và nộp

1. Kiểm tra định dạng: đúng số dòng (3011 / 3299 + 1 dòng tiêu đề), cột `idx,x,y,yaw`, yaw bằng radian. (Repo dùng hàm `validate` của một bot nộp bài nằm ngoài repo, `~/kaggle-sdc-bot`.)
2. Kiểm tra offline (không tốn lượt, chỉ để loại cấu hình hỏng): `docker run … sdc-reg python3 tools/map_fit.py 1 --data /w/data FILE.csv`.
3. So với file cũ: `python3 tools/compare_csv.py docs/nhat-ky/submissions track1:A.csv:B.csv`.
4. Nộp trên trang Kaggle (hoặc qua bot). Sau đó vào tab **Submissions**, đánh dấu file pipeline là **Use for Final Score** cho mỗi track.
5. Nộp E3: đổi tên thành `result_1.csv`, `result_2.csv`; kèm code (không kèm `data/`), báo cáo PDF, README.

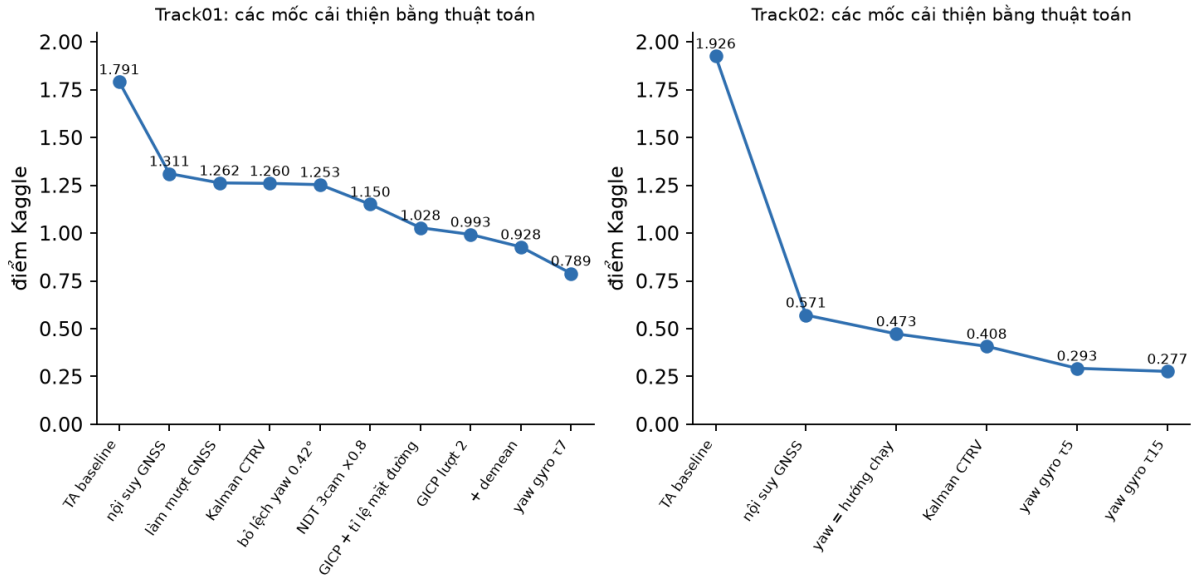
Ghi chú

Bot Kaggle (`~/kaggle-sdc-bot`, không thuộc repo) chạy bằng cron mỗi 30 phút, nộp tối đa 3 file mỗi lần từ `queue/track1|2`, ngân sách 15 lượt/ngày, reset 08:00 giờ máy. Máy phải bật lúc 08:00. Không commit token Kaggle.

Chương 9. Kết quả, điểm mạnh, hạn chế và hướng cải thiện

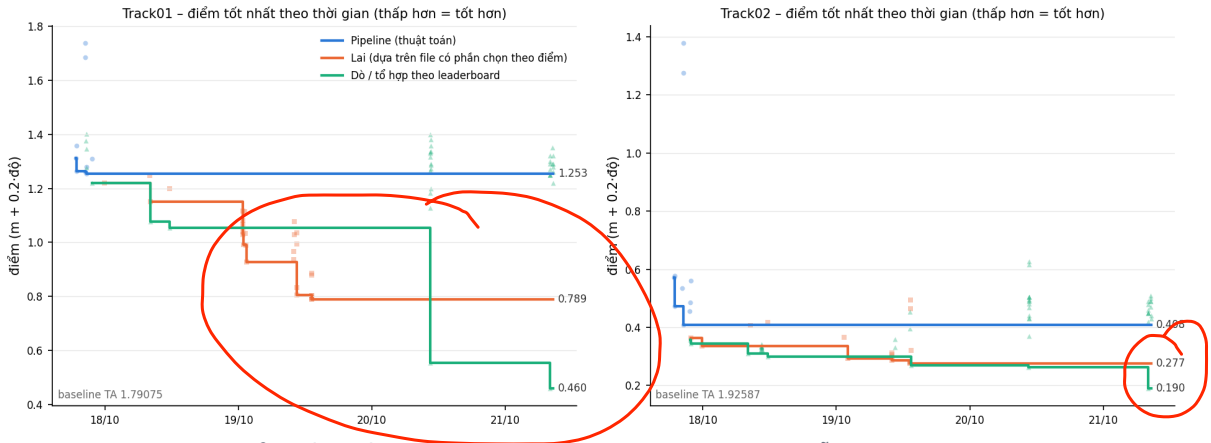
9.1 Kết quả hiện tại (21/10/2026 tối)

	Track01	Track02
Baseline TA	1.79075	1.92587
Bài chấm dự kiến (chỉ pipeline)	`clean_cfz1_tau7.csv` – **chưa có điểm**, dự kiến ~0.79	`clean_cfz2_tau15.csv` – **chưa có điểm**, dự kiến ~0.28
Cùng phương pháp, lượt cũ ("lai")	0.78914 (`cfz1_tau7`)	0.27654 (`cfz2_tau15`)
Pipeline thuần tốt nhất đã có điểm	1.25338 (`phys_yawb_1`)	0.40778 (`rts_heading_2`)
Điểm đang hiện trên bảng (file dò – KHÔNG dùng chấm)	0.46 (hạng 2)	0.19 (hạng 2)
Đội dẫn đầu (21/10)	0.40	0.17



Lưu ý: từ 'NDT 3cam' (T1) và 'yaw gyro' (T2) trở đi, các file này thuộc loại 'lai' – chạy trên nền một file có phần chọn theo điểm Kaggle. Bản dựng lại sạch (clean_cfz1_tau7 / clean_cfz2_tau15) CHƯA có điểm; dự kiến ~0.79 / ~0.28.

Hình 20. Các mốc cải thiện bằng thuật toán (số liệu từ docs/nhat-ky/04, 08, 09).



Hình 21. Điểm tốt nhất theo thời gian, tách theo loại (hình có sẵn trong repo, sinh bởi tools/submission_stats.py). Xanh dương: pipeline; cam: lai; xanh lá: dò bảng điểm (không được dùng chấm).

Sao lại có sự khác biệt này vì lúc đầu lại dẫn tooys hơn nhưng sau đó dò tụt đột ngột và giữ nguyên

Loại (theo tools/submission_stats.py)	Track01: số lần / tốt nhất	Track02: số lần / tốt nhất
Pipeline (thuật toán thuần)	10 / 1.25338	10 / 0.40778
Lai (thuật toán trên nền file có phần chọn theo điểm)	29 / 0.78914	16 / 0.27654
Dò / tổ hợp theo bảng điểm	37 / 0.46	48 / 0.19
Khác (thử, lỗi)	1	1

Minh bạch: điểm cần biết

Nhiều lượt nộp (85/152) là file dò bảng điểm. Nộp có hệ thống để đo đáp án là điều TA không cho dùng làm kết quả. Repo minh bạch về điều này (phân loại rõ, doc 09), nhưng khi thuyết trình nên chủ động nói.

Theo nhật ký, bảng xếp hạng ngày 19/10 có 7 đội; đội 11 hạng 3 với file "lai" tốt nhất. Hạng thật của bài chấm sạch sẽ chỉ biết sau khi nộp và Kaggle công bố bảng riêng (private leaderboard) – tài liệu này không đoán.

9.2 Điểm mạnh

- **Phát hiện đúng gốc rễ của baseline:** sai tỉ lệ point cloud (×3 → ×9), chứng minh bằng hai cách độc lập (mặt đường, khớp bản đồ).
- **Dùng mô hình vật lý, tham số ước lượng từ dữ liệu** (Kalman CTRV, maximum likelihood) thay vì làm mượt tuỳ ý.
- **Phân tích dữ liệu sâu:** phát hiện yaw GNSS được tính từ vị trí, trễ 1.1–1.4 s, đóng băng khi dừng; bias gyro; GNSS Track01 giật hơn.
- **Tỉ lệ theo từng frame từ mặt đường** + GICP: lần đầu scan matching thật sự sửa được GNSS (0.905 → 0.616 m ở Track01).
- **Yaw từ gyro bằng bộ lọc bù:** bước tiến lớn nhất (Track01 1.56° → 0.87°).
- **Ghi chép rất đầy đủ, tái tạo được:** nhật ký từng lần nộp, bản sao từng file nộp, script tạo lại, pipeline một lệnh, phân loại minh bạch.
- **Biết dừng đúng lúc:** dừng dò bảng điểm ngay khi TA trả lời; xây thước đo offline và thừa nhận khi chúng không dùng được.

9.3 Hạn chế, phần chưa xong, lỗi đã biết

- **Bài chấm sạch chưa có điểm** và chưa được chọn "Use for Final Score".
- **Kết quả phụ thuộc mạnh vào GNSS:** topic GNSS đã rất gần đáp án; Track02 hoàn toàn không dùng scan matching. Phần "scan matching" mà đề yêu cầu chỉ đóng góp ~0.29 m cho Track01. Đề bài nhấn mạnh tự thiết kế scan matching – nên trình bày rõ thí nghiệm NDT/GICP và lý do.
- **Kết quả chỉ dùng ảnh** (không GNSS) chưa làm xong: NDT ×9 trôi ~9 m sau 150 frame; chưa có mô hình chuyển động và thử nhiều góc ban đầu.
- **Lỗi còn lại là độ lệch có hệ thống** của GICP do point cloud ảnh bị méo (không phải nhiễu) – làm mượt hay trọng số khác không sửa được.
- **Nhiều tham số chọn bằng điểm Kaggle** (tỉ lệ 0.8, số lượt 2, τ, trục gyro, tắt trượt, demean dựa vào lượt thăm dò) – được phép nhưng phải khai báo.
- **Không có thước đo offline đáng tin:** `map_fit.py` bị GICP đánh lừa; `three_hat.py` sai giả định độc lập.
- **Phụ thuộc file ngoài git:** lượt NDT `c3s08`/`c3s10T2`, `map_voxel_0.3m.pcd`, file debug cho `map_fit.py`.

- **Ảnh hưởng gián tiếp chưa kiểm chứng:** z/roll/pitch lấy từ lượt NDT có prior là tổ hợp chọn theo điểm (Chương 4.2).
- **Code/tài liệu lỗi thời:** `README.md` (điểm 17/10), `nhat-ky/README.md` (số file), `reproduce_submissions.sh` (chỉ 34 file nhưng so 111), video (18/10), báo cáo PDF cũ (19/10).
- **Không nhất quán môi trường:** 3 thư mục build (`install`, `install_dev`, `install_par`); đường dẫn cứng trong docker-compose; `.vscode` trỏ máy TA; launch file thiếu tham số mới.
- **Code thừa của template:** `visualization.cpp/.rviz` (dấu vết bài radar, ROS 1), import thừa trong `txt_to_idx_x_y_yaw.py`, GNSS/IMU đọc trong C++ nhưng không dùng.
- **Kiểm tra lỗi đầu vào kém** trong vài công cụ Python (thiếu `--pose-source`, thiếu pose trong lượt C++ → lỗi khó hiểu).

9.4 Hướng cải thiện (đã có cơ sở trong repo)

1. **Nộp và chọn bài chấm sạch ngay** (đã xếp hàng 22/10 cùng `if_iso_*` / `if_aniso_*`); ghi điểm vào nhật ký §13.7.
2. **Giới hạn tầm point cloud 20 / 30 / 40 m** (độ sâu từ ảnh sai tăng theo bình phương khoảng cách) – chỉ đổi `--max-range`, 3 file.
3. **Kalman kết hợp GICP + IMU** thay cho làm mượt 2 s: đưa độ sửa GICP làm phép đo vị trí, gyro làm phép đo tốc độ quay, trong cùng bộ lọc.
4. Giảm độ lệch có hệ thống ở đầu vào: lọc point cloud theo độ tin cậy độ sâu, dùng `raw_ply` và `3cameras_ply` kết hợp, hiệu chỉnh vị trí camera trong point cloud MapAnything.
5. Lấy z/roll/pitch không phụ thuộc lượt NDT cũ (ví dụ từ độ cao bản đồ dưới xe) để pipeline sạch hoàn toàn và chạy lại được.
6. Kết quả "chỉ dùng ảnh" cho báo cáo: thêm mô hình vận tốc không đổi và thử nhiều góc ban đầu cho NDT/GICP.
7. Dọn repo: cập nhật README, sửa `reproduce_submissions.sh`, thống nhất thư mục build, sao lưu các file kết quả cần thiết.

9.5 Việc cần làm trước 29/10 (checklist)

Việc	Hạn	Ghi chú
Nộp `clean_cfz1_tau7.csv`, `clean_cfz2_tau15.csv` (hoặc bản sạch tốt hơn)	trước 29/10 09:00	bot 22/10 08:00
Chọn "Use for Final Score" cho file sạch, mỗi track	trước 29/10 09:00	người dùng tự làm trên web
Rút slide còn 6–8 trang (4–6 phút), cập nhật số liệu, nói rõ phần dò bảng điểm và dùng AI	29/10	slide hiện 13 trang
Báo cáo tiếng Anh theo 4 mục TA	05/11 23:59	bản nháp đã có (artifact riêng)
E3: code (không data), result_1/2.csv, báo cáo PDF, README	05/11 23:59	

Phụ lục A – Thuật ngữ Việt – Anh

Tiếng Anh	Tiếng Việt	Giải thích ngắn
Localization	định vị	tìm vị trí và hướng của xe trên bản đồ
Pose	tư thế	vị trí + hướng; ở đây (x, y, yaw)
6DoF	6 bậc tự do	x, y, z, roll, pitch, yaw
Yaw / Roll / Pitch	góc quay / lăn / chúi	quay quanh trục đứng / trục dọc / trục ngang
Coordinate frame	hệ toạ độ	gốc + các trục; map, camera FLU
FLU (Front-Left-Up)	trước-trái-lên	quy ước trục của hệ camera
Transformation matrix	ma trận biến đổi	ma trận 4×4 gồm phép quay R và dịch t
Quaternion	quaternion	4 số biểu diễn hướng, dùng trong ROS
Radian	radian	đơn vị góc; π rad = 180°
Wrap / unwrap	gói góc / nối góc	đưa góc về −π…π / bỏ bước nhảy 2π
Point cloud	đám mây điểm	tập hợp điểm 3D
PLY / PCD	định dạng file point cloud	PLY: point cloud từ ảnh; PCD: bản đồ (PCL)
Voxel / voxel grid downsampling	khối / giảm mẫu theo lưới khối	mỗi khối lập phương giữ một điểm
LiDAR	LiDAR	cảm biến laser đo khoảng cách, dùng làm bản đồ
GNSS / GPS	định vị vệ tinh	GPS là một hệ GNSS
IMU	cảm biến quán tính	gyro + gia tốc kế
Gyroscope (gyro)	con quay hồi chuyển	đo tốc độ quay
Accelerometer	gia tốc kế	đo gia tốc (kể cả trọng lực)
Bias	độ lệch (của cảm biến)	sai số không đổi; gyro z ≈ −0.25…−0.30 °/s
Drift	trôi	sai số tích luỹ dần theo thời gian
Depth estimation	ước lượng độ sâu	đoán khoảng cách từ ảnh
Scale	tỉ lệ	hệ số nhân point cloud để ra mét
Map / submap	bản đồ / bản đồ con	submap: khối bản đồ quanh xe
Scan matching / registration	khớp quét / đăng ký	khớp point cloud vào bản đồ
ICP	lặp điểm gần nhất	Iterative Closest Point
Point-to-plane	điểm – mặt phẳng	biến thể ICP đo theo pháp tuyến
GICP / VGICP	ICP tổng quát / bản voxel	dùng hiệp phương sai mỗi điểm
NDT	biến đổi phân bố chuẩn	Normal Distributions Transform
Correspondence	cặp tương ứng	điểm nguồn ghép với điểm đích
Local minimum	cực tiểu địa phương	nghiệm sai mà thuật toán lặp không thoát được
Covariance	hiệp phương sai	mức độ trải/không chắc theo từng hướng
Hessian	ma trận Hessian	đạo hàm bậc hai; độ chắc của phép khớp theo hướng
Schur complement	bổ sung Schur	rút ma trận về một nhóm biến
Information (matrix)	(ma trận) thông tin	nghịch đảo hiệp phương sai
Robust kernel (Geman-McClure)	hàm phạt bền vững	giảm trọng số điểm sai lớn
Kalman filter / EKF	bộ lọc Kalman / mở rộng	trộn dự đoán và phép đo theo độ tin cậy
State	trạng thái	các đại lượng cần ước lượng
Predict / update	dự đoán / cập nhật	hai bước của Kalman
Innovation	đổi mới	phép đo trừ dự đoán
Kalman gain	độ lợi Kalman	tin phép đo bao nhiêu
Process noise / measurement noise	nhiễu quá trình / nhiễu đo	q / σ

Tiếng Anh	Tiếng Việt	Giải thích ngắn
CTRV	tốc độ và tốc độ quay không đổi	mô hình chuyển động xe
RTS smoother	bộ làm mượt RTS	lọc ngược để dùng cả thông tin tương lai
Maximum likelihood (ML)	hợp lý cực đại	chọn tham số làm dữ liệu "dễ xảy ra" nhất
Log-likelihood	log hợp lý	thước đo mô hình khớp dữ liệu
Complementary filter	bộ lọc bù	nhanh từ gyro, chậm từ hướng chạy
Motion heading	hướng chạy	hướng của vector vận tốc
Lever arm / slip	cánh tay đòn / trượt	lệch giữa hướng vận tốc và thân xe khi vào cua
Interpolation / spline	nội suy / đường cong spline	ước lượng giá trị giữa các mẫu
Gaussian smoothing / median filter	làm mượt Gauss / lọc trung vị	trung bình có trọng số / trung vị cửa sổ
Gating	cổng chặn	loại kết quả lệch quá ngưỡng
Demean	trừ trung bình	bỏ độ lệch cố định
Prior	ước lượng trước / tiền nghiệm	điểm bắt đầu cho bước sau
Fusion	ghép / hợp nhất	kết hợp nhiều nguồn
Least squares	bình phương tối thiểu	cực tiểu tổng bình phương sai số
RMSE	căn trung bình bình phương sai số	thước đo của Kaggle
Baseline	mốc cơ sở	kết quả của code mẫu TA
Leaderboard / probing	bảng xếp hạng / thăm dò	nộp file lệch có chủ đích để đo đáp án
Leaderboard-based fitting	khớp theo bảng điểm	điều TA cấm cho bài chấm
Ground truth	đáp án / giá trị thật	Kaggle giữ kín
Offline metric	thước đo không cần đáp án	map_fit, three_hat
Three-cornered hat	phương pháp "mũ ba góc"	ước lượng sai số từ ≥ 3 nguồn độc lập
Spearman correlation	tương quan hạng Spearman	hai thứ tự có giống nhau không
ROS 2 / node / topic	ROS 2 / nút / kênh	khung phần mềm robot
rosbag	bản ghi ROS	file .db3 chứa dữ liệu cảm biến
RViz	RViz	chương trình xem 3D của ROS
Docker image / container	ảnh / hộp chạy	môi trường phần mềm đóng gói
Fisheye	ống kính mắt cá	góc rất rộng, méo mạnh
Hood mask	mặt nạ nắp capo	che phần nắp xe trong ảnh
SOR (statistical outlier removal)	lọc điểm lạc thống kê	bỏ điểm xa lân cận

Phụ lục B – Danh sách toàn bộ file CSV đã nộp / định nộp

Thư mục `docs/nhat-ky/submissions/`. Điểm lấy từ `docs/nhat-ky/data/submission_stats.csv` (5 chữ số = điểm chính xác lúc nộp; 2 chữ số = điểm Kaggle hiện tại, đã cắt bớt; "–" = chưa nộp hoặc không có điểm). Loại: P = pipeline, L = lai, D = dò/tổ hợp bảng điểm.

Track01

File	Điểm	Loại	File	Điểm	Loại
cfg1_tau10_al	0.79585	L	phys_yawb_ndt_1	1.27855	P
cfg1_tau15	0.80359	L	planar_clean_pos_1	1.03535	L
cfg1_tau15_al	0.80646	L	planar_ground_all_pos_1	1.03311	L
cfg1_tau30_al	0.83296	L	planar_pass2dm_bestyaw_1	1.02780	L
cfz1_tau10	0.79127	L	plane_icp_ground_pos_1	0.99504	L
cfz1_tau15_al	0.80411	L	plane_icp_pass2dm_cfyaw_1	0.87976	L
cfz1_tau7	0.78914	L	plane_icp_pass2plain_cfyaw_1	0.88523	L
clean_cfz1_tau7	–	–	posproj_v1_1	1.07618	D
corr_r2_1	–	D	probe_lat+0.5_1	1.37688	D
gicp_ground_all_pos_1	1.02813	L	probe_lat-0.5_1	1.40077	D
gicp_ground_all_posyaw_1	1.03955	L	probe_x0_1	1.32969	D
gicp_ground_pass2_pos_1	0.99283	L	probe_x1_1	1.40012	D
gicp_ground_pass3dm_bestyaw_1	0.93564	L	probe_x2_1	1.26597	D
gicp_ground_pass4dm_bestyaw_1	0.96735	L	probe_x3_1	1.12730	D
gicp_ground_pass5_pos_1	1.11461	L	probe_x4_1	1.18208	D
gicp_ground_pos_1	1.06593	L	probe_x5_1	1.33500	D
gicp_hood_sor_pos_1	1.07806	L	probe_x6_1	1.31574	D
gicp_s08_pos_1	1.11701	L	probe_y0_1	1.33648	D
gnss_1	1.31094	P	probe_y1_1	1.19880	D
gnss_heading_1	1.35783	P	probe_y2_1	1.28668	D
gnss_smooth_1	1.26235	P	probe_y3_1	1.38082	D
if_aniso_1	–	–	probe_y4_1	1.35677	D
if_iso_1	–	–	probe_y5_1	1.27174	D
lagyaw_pos_best_1	1.30875	P	probe_y6_1	1.29067	D
lb_posyaw_v1_1	0.55496	D	probe_yaw+1_1	1.27676	D
ndt3c_s08_pos_1	1.15042	L	probe_yaw-1_1	1.34505	D
ndt3c_s10_pos_1	1.24807	L	rts_pos_gnssyaw_1	1.25992	P
ndt_gicp_s08_pos_1	1.08265	L	rts_pos_gnssyaw_dt-0.1_1	1.68409	P
pass2dm_bestyaw_1	0.92775	L	rts_pos_gnssyaw_dt0.1_1	1.73784	P
pass2pos_bestyaw_1	0.99060	L	startfix_on_v1_1	1.21871	L
phys_yawb_1	1.25338	P	yawproj_v1_1	1.21861	D

Track02

File	Điểm	Loại	File	Điểm	Loại
cf_tau5_2	0.29268	L	probe_cf5_yaw+0.5_2	0.33197	D
cfg2_tau15_al	0.28743	L	probe_cf5_yaw-0.5_2	0.35944	D
cfg2_tau2_al	0.31258	L	probe_lat+0.3_2	0.45373	D
cfg2_tau5	0.29756	L	probe_lat-0.3_2	0.39579	D
cfg2_tau5_al	0.30531	L	probe_x0_2	0.61644	D
cfz2_tau10	0.27949	L	probe_x1_2	0.43173	D
cfz2_tau15	0.27654	L	probe_x2_2	0.43041	D
cfz2_tau30	0.28034	L	probe_x3_2	0.50500	D
clean_cfz2_tau15	–	–	probe_x4_2	0.46752	D
corr_r2_2	–	D	probe_x5_2	0.36959	D
curv1.5_on_proj_2	0.36280	L	probe_x6_2	0.44091	D
fusion_s9r4_headingyaw_2	0.53520	P	probe_y0_2	0.62749	D
gicp_cf_dm_2	0.46400	L	probe_y1_2	0.50550	D
gicp_dm_w05_on_best_2	0.36530	L	probe_y2_2	0.49559	D
gnss_2	0.57111	P	probe_y3_2	0.49202	D
gnss_heading_2	0.47281	P	probe_y4_2	0.48105	D
gnss_smooth_2	0.57811	P	probe_y5_2	0.50504	D
gnssyaw_pos_rts_2	0.56160	P	probe_y6_2	0.49073	D
if_aniso_2	–	–	rts_heading_2	0.40778	P
if_iso_2	–	–	rts_heading_dt-0.1_2	1.27503	P
lagyaw_pos_best_2	0.45496	P	rts_heading_dt0.1_2	1.37861	P
lb_posyaw_v1_2	0.26291	D	startfix_on_v3_2	0.33504	L
ndt3c_cf_dm_2	0.32011	L	yawproj_v1_2	0.35696	D
ndt3c_s10_pos_2	0.40544	L	yawproj_v3_2	0.34530	D
plane_icp_cf_dm_2	0.49383	L	yawproj_v4_2	0.30984	D
posproj_t2_v1_2	0.26849	D	yawtau0.75_pos_best_2	0.48625	P

Không chắc chắn

Một số file không có điểm trong thống kê vì chưa từng được nộp (ví dụ `clean_*`, `if_*`, `corr_r2_*` – đã bị rút khỏi hàng nộp) hoặc vì tên trên Kaggle khác tên file. Loại "D" của file không có điểm được suy ra từ tên file.

Phụ lục C – Lịch sử commit

36 commit từ 17/10 19:10 tới 21/10 19:11 (giờ máy, UTC+8). Commit đầu sau "Initial commit" là **bản gốc của TA**, nên có thể so mọi thay đổi bằng `git diff 65d10cb HEAD`.

Commit	Thời gian	Nội dung (dịch / tóm tắt)
8d5f06a	17/10 19:23	Initial commit (GitHub)
65d10cb	17/10 19:10	Template gốc của TA + script baseline/GNSS
cd985a4	17/10 19:13	NDT có prior GNSS, cổng chặn, file debug, script ghép
64fa10c	17/10 19:27	Docker và README
49de295	17/10 19:53	Công cụ phân tích point cloud (tỉ lệ/hệ trục), generate_result.py, PRIOR
f66e1c1	17/10 20:28	Kalman CTRV + RTS, nhiễu chọn bằng ML
dcb9f8d	17/10 20:32	rts_smoother: --time-offset
7a1bfda	17/10 21:14	Nhật ký, script tái tạo bài nộp
8b092ce	17/10 21:45	Smoother có gyro + yaw trễ, tổ hợp yaw, NDT chỉ ảnh, track_error
9bccac2	17/10 22:23	Nhật ký: yaw GNSS trễ, tổ hợp yaw, NDT 3 camera thử
116f7c0	17/10 23:56	Sửa yaw đoạn khởi hành bằng gyro, công cụ tổ hợp vị trí
77ea2a9	18/10 00:08	start_frame, chạy NDT song song theo đoạn
4346f1e…60c6afd	18/10 08:15–08:36	Nhật ký sáng 18/10, tái tạo 34/34
383e15f	18/10 11:39	bump_yaw.py (dò yaw theo cửa sổ)
cff7ee3…eaed405	18/10 12:30–23:24	Video bài giảng (5 commit)
400f7da	19/10 01:30	GICP offline bằng small_gicp, tỉ lệ theo mặt đường
b746a2c	19/10 01:46	Nhật ký 08: GICP, demean (0.92775)
daad0e9	19/10 02:22	Tiền xử lý TA gợi ý (nắp capo, SOR, bỏ mặt đường); yaw bộ lọc bù
e2b1398	19/10 02:40	Nhật ký 08: kết quả tiền xử lý, ICP point-to-plane
7793b90	19/10 13:36	Yaw gyro (0.78914 / 0.27654), tổ hợp Track02 (0.26849)
fa3f102	19/10 13:46	make_probes.py: dò dao động cosin
a19eb45	19/10 14:14	Báo cáo tổng hợp PDF tiếng Việt
ea6591b	20/10 10:43	Dò đợt 1, tổ hợp 0.55496 / 0.26291
94c854c	21/10 08:49	Dò đợt 2, corr_r1 0.46 / 0.19
9d0ca5e	21/10 09:07	DỪNG dò sau trả lời TA; dựng lại bài chấm sạch
2997bfc	21/10 09:50	Thống kê nộp bài (doc 09)
95edeb0	21/10 12:18	Bộ ghép theo Hessian, kiểm tra lệch thời gian GNSS–IMU
4450751	21/10 14:27	Thước đo offline map_fit, three_hat
dfbf356	21/10 16:05	Hướng dẫn chạy + pipeline sạch một lệnh
2c3f3ae	21/10 17:16	Hessian so với nhiễu đo được, phân tích theo đoạn
0ae197f	21/10 19:11	CLAUDE.md: trạng thái dự án và việc tiếp theo

> **Ghi chú**
> Thời gian của commit "Initial commit" (19:23) muộn hơn commit TA (19:10) – có lẽ do đồng hồ hoặc do commit được tạo trên GitHub rồi ghép lịch sử (không chắc) – không ảnh hưởng gì.